



FCES

2026 CCF未来计算机教育峰会
FUTURE COMPUTER EDUCATION SUMMIT

未来教育:颠覆还是改良?

分论坛十·报告

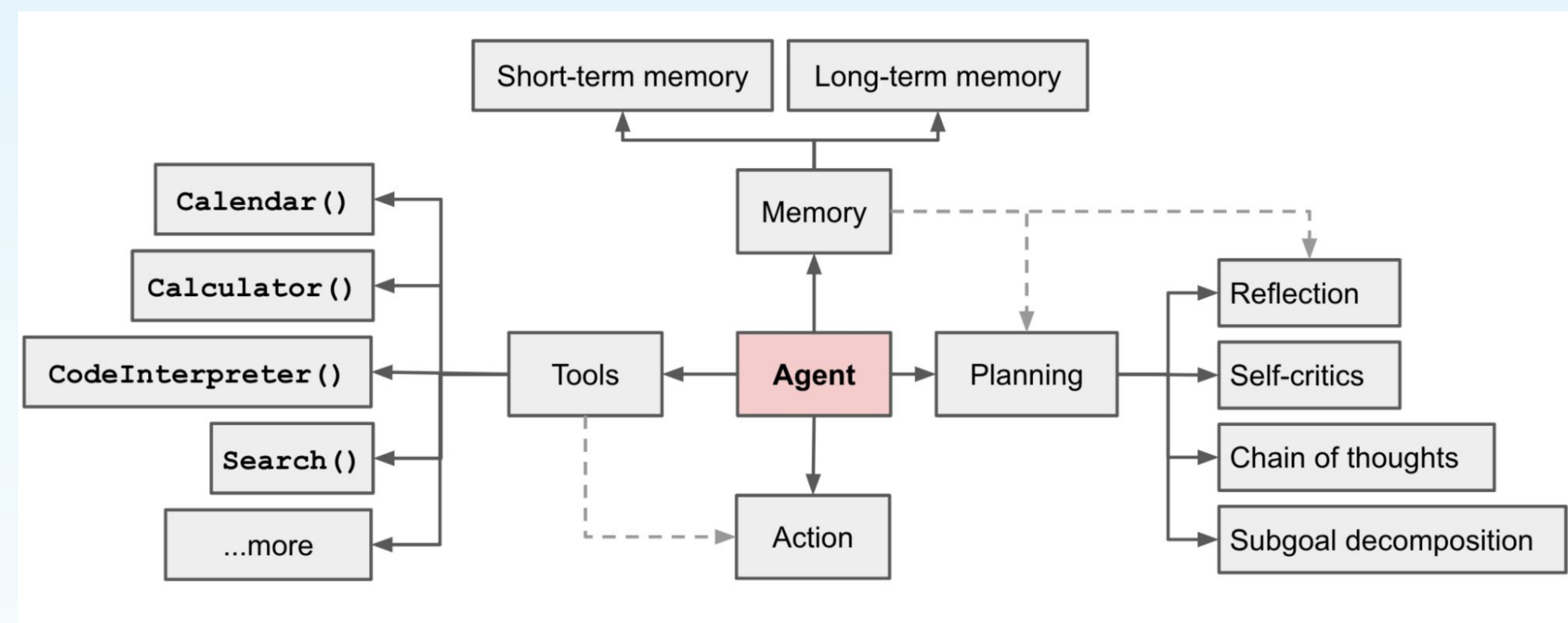
面向国产算力生态的智能计算系统课程

罗国杰

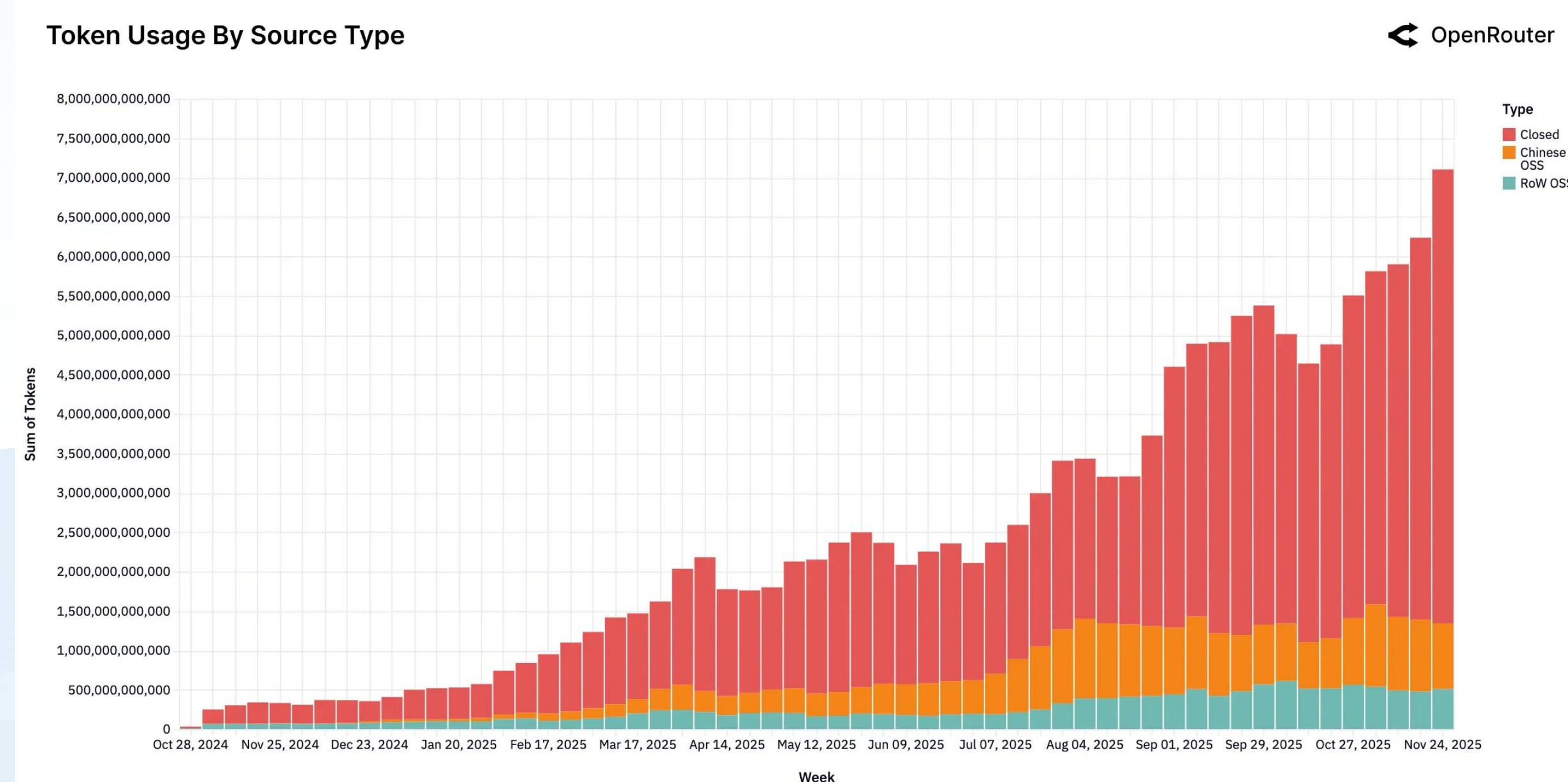
北京大学 计算机学院

Token密集型生成式AI应用

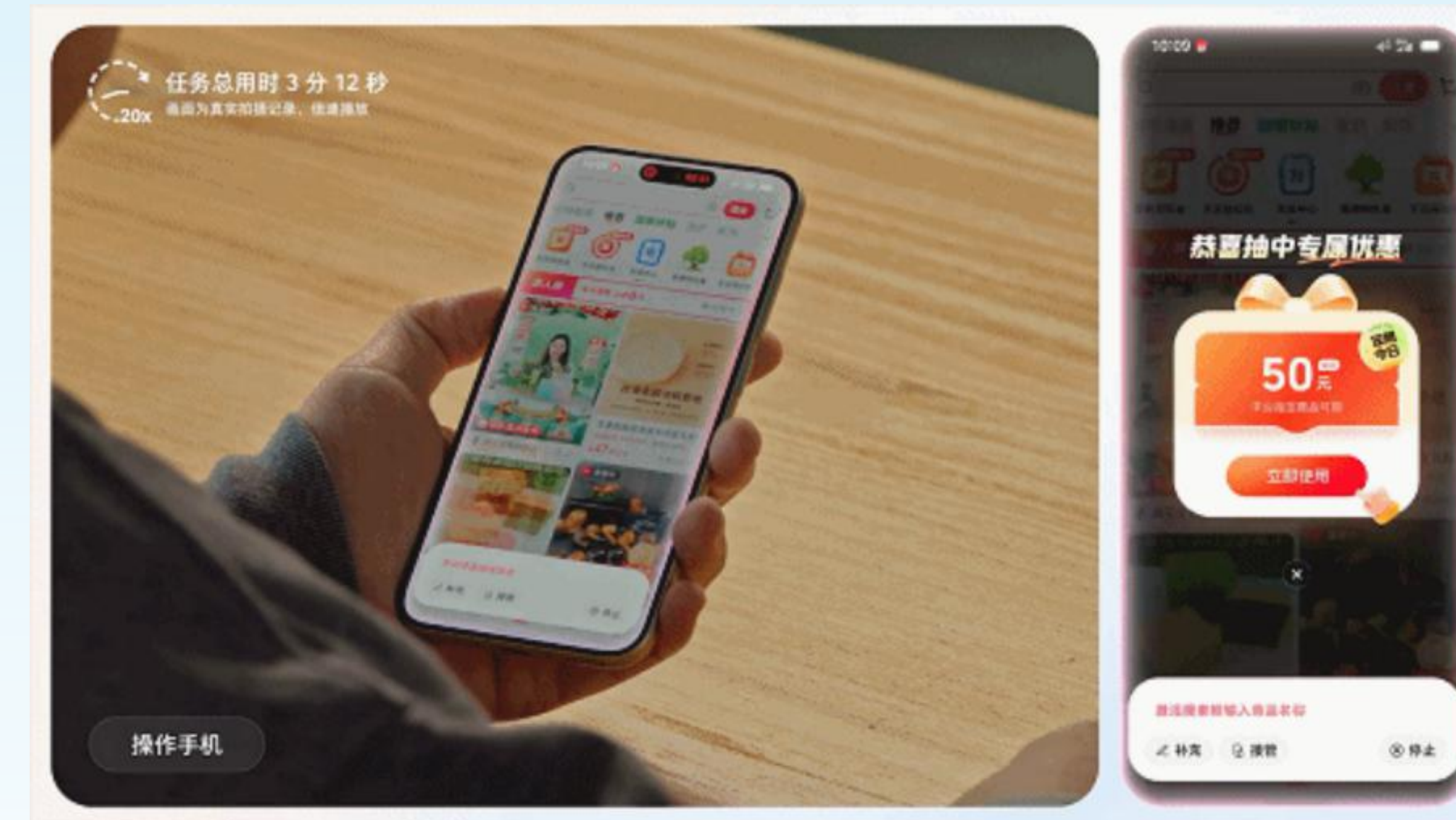
- 基于大语言模型的自主智能体



- Token消耗趋势



- AI手机与AI PC

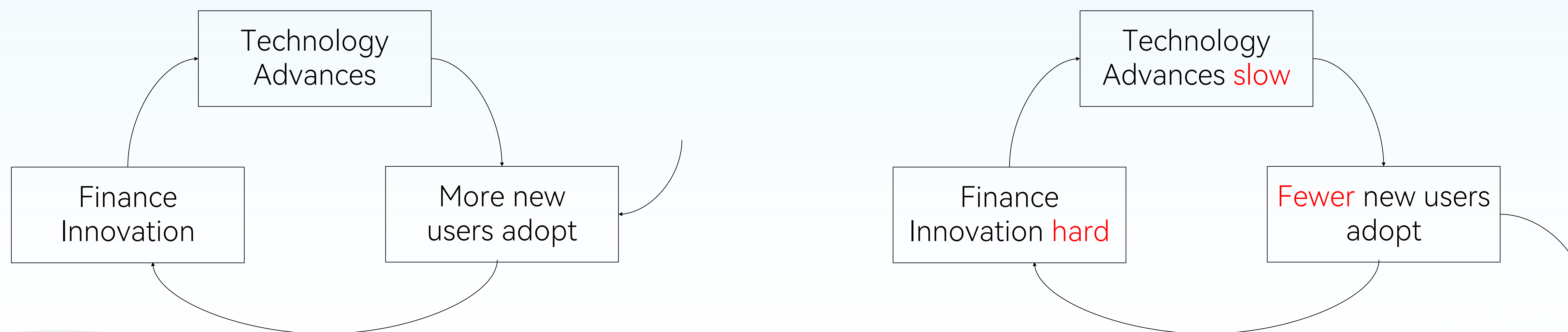


- 视频生成



What is “General-Purpose” Computing?

- in terms of #programs → CPU is (still) general-purpose
- in terms of TOP/s → GPU/NPU are general-purpose
- Specialization fragments computing into “fast” and “slow lane”





相关课程

- 【研究生】智能计算系统（25-26学年第1学期）
- 2026年北京智源“中非人工智能教师人才联合培养计划”
- 【本科生】并行与分布式计算导论（25-26学年第2学期）
- 【本科生】算法设计与分析（实验班）（26-27学年第2学期）



研究生“智能计算系统” Syllabus and Schedule

- (Sep. 11) L01: Introduction
- (Sep. 18) L02: ML System Foundations
- (Sep. 25) L03: Frameworks (1/2)
- (Oct. 09) L03: Frameworks (2/2)
- (Oct. 16) L04: Comp. Subsys (1/2)
- (Oct. 23) L04: Comp. Subsys (2/2)
- (Oct. 30) L05: AI Compilation (1/2)
- (Nov. 06) L05: AI Compilation (2/2)
- (Nov. 13) L06: Mem. Subsystem (1/2)
- (Nov. 20) L06: Mem. Subsystem (2/2)
- (Nov. 27) L07: Comm. Subsystem (1/2)
- (Dec. 04) L07: Comm. Subsystem (2/2)
- (Dec. 11) L08: Mod.-Sys. Co-opt. (1/2)
- (Dec. 18) L08: Mod.-Sys. Co-opt. (2/2)
- (Dec. 25) L09: Conclusions

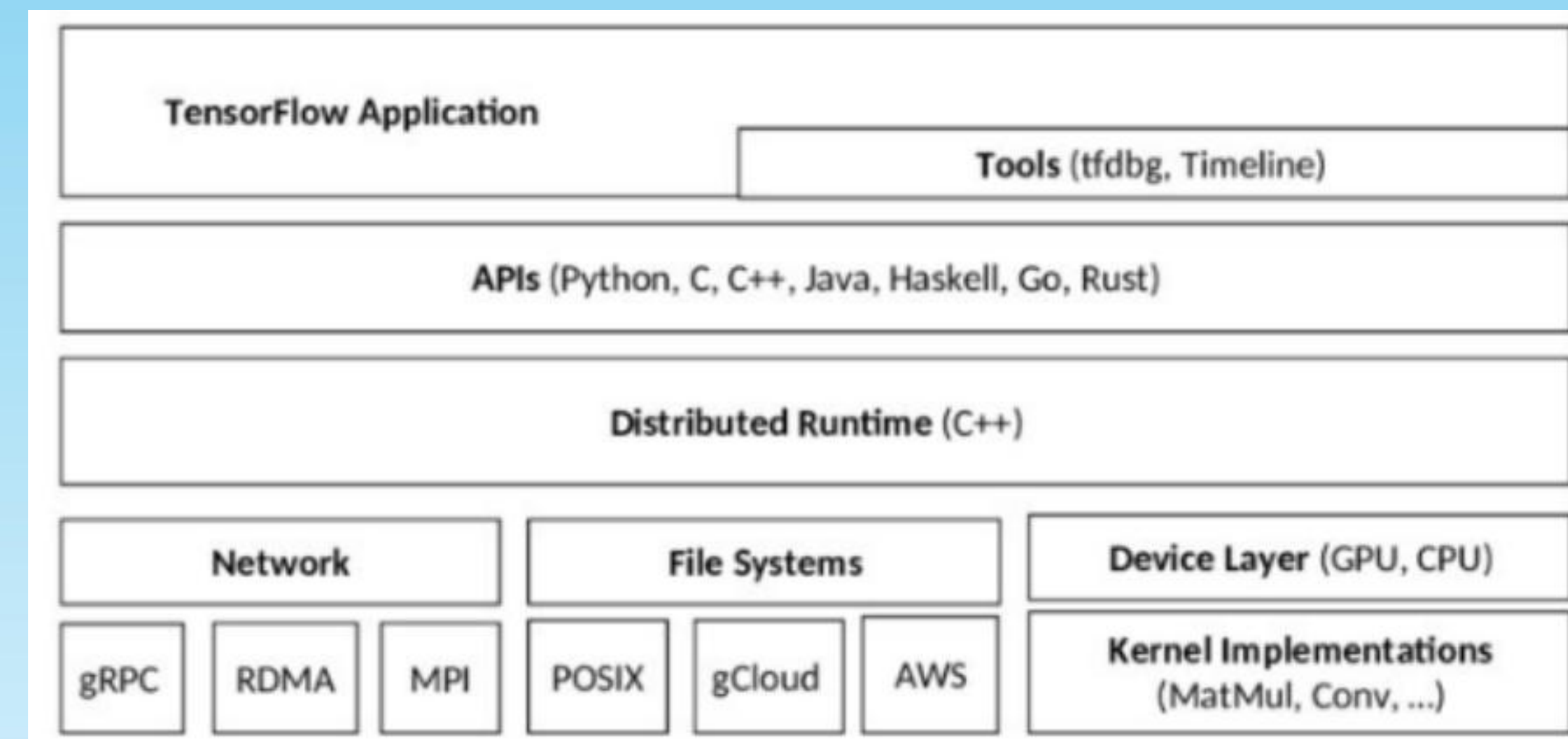
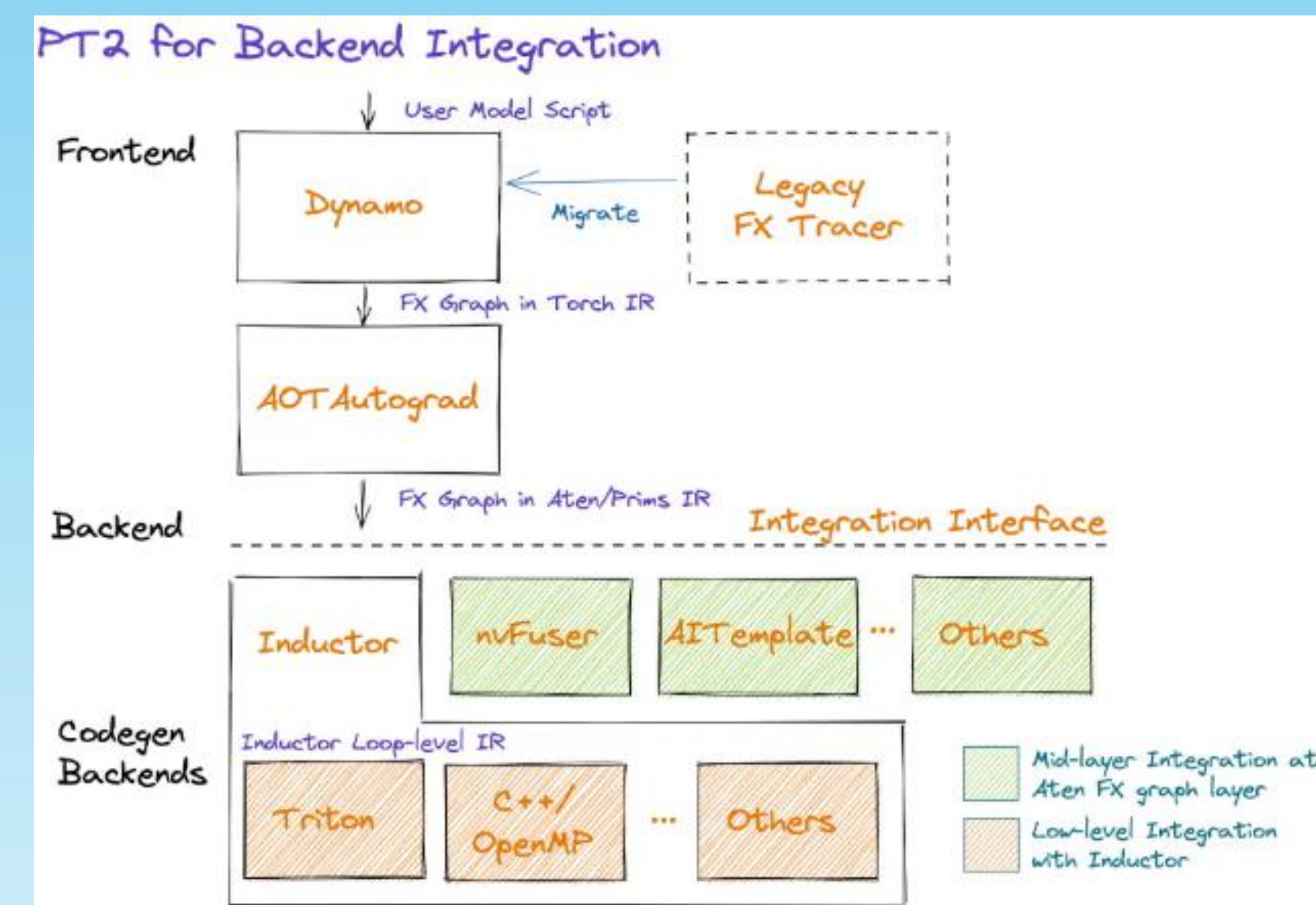


第2讲预览: ML System Foundations

- Introduction to ML Systems
- Basics of ML, DL, and LLM
- ML System Spectrum
 - (hardware-oriented)
 - AIDC (Cloud ML), Edge ML, Mobild ML, TinyML
- ML System Simulators
 - Introducing architecture and micro-architecture simulators to help understand system architecture.
- Performance Benchmarking
 - Discussing the principles, metrics, and frameworks for ML system evaluation, and their role in design and optimization.

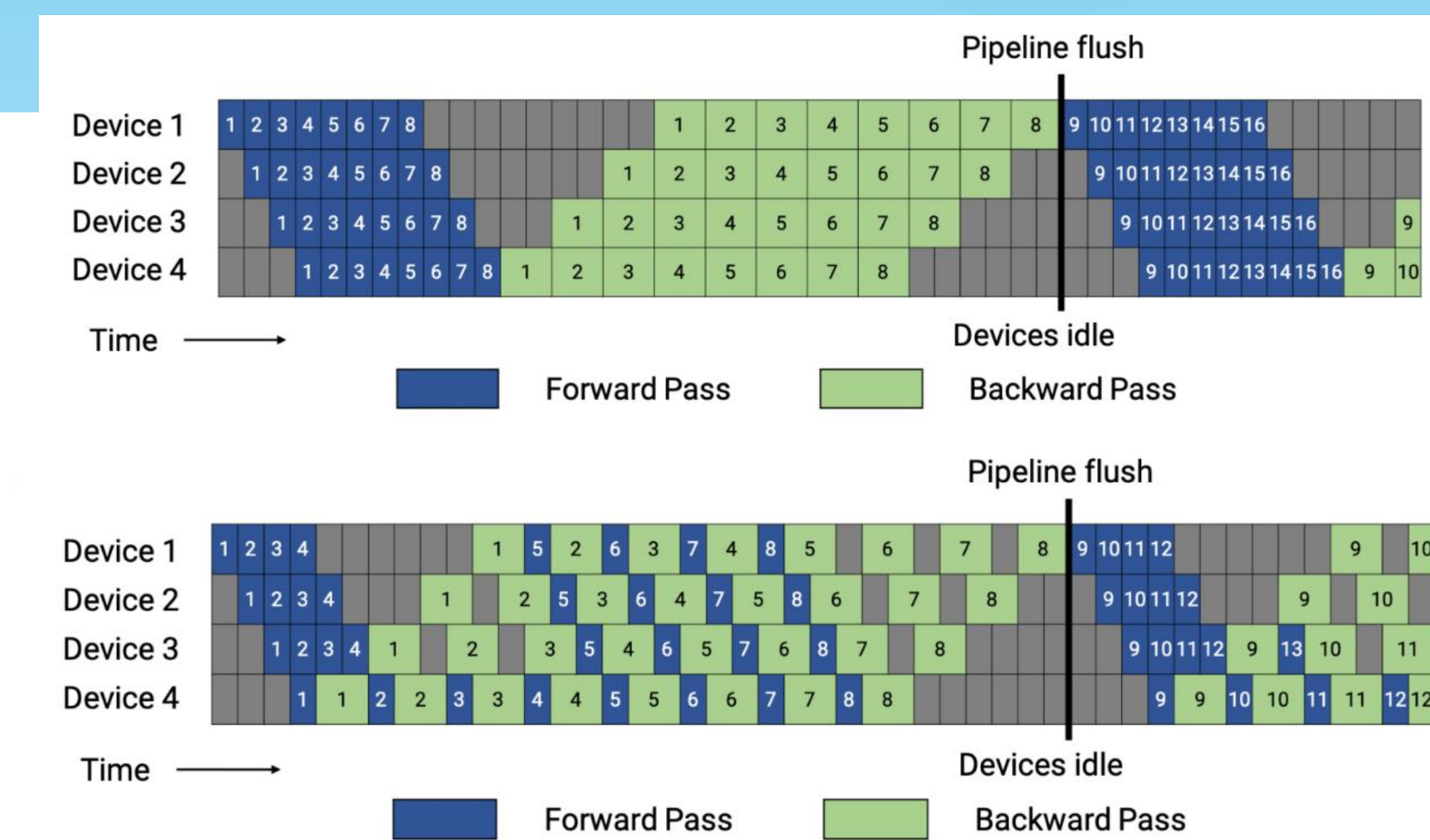
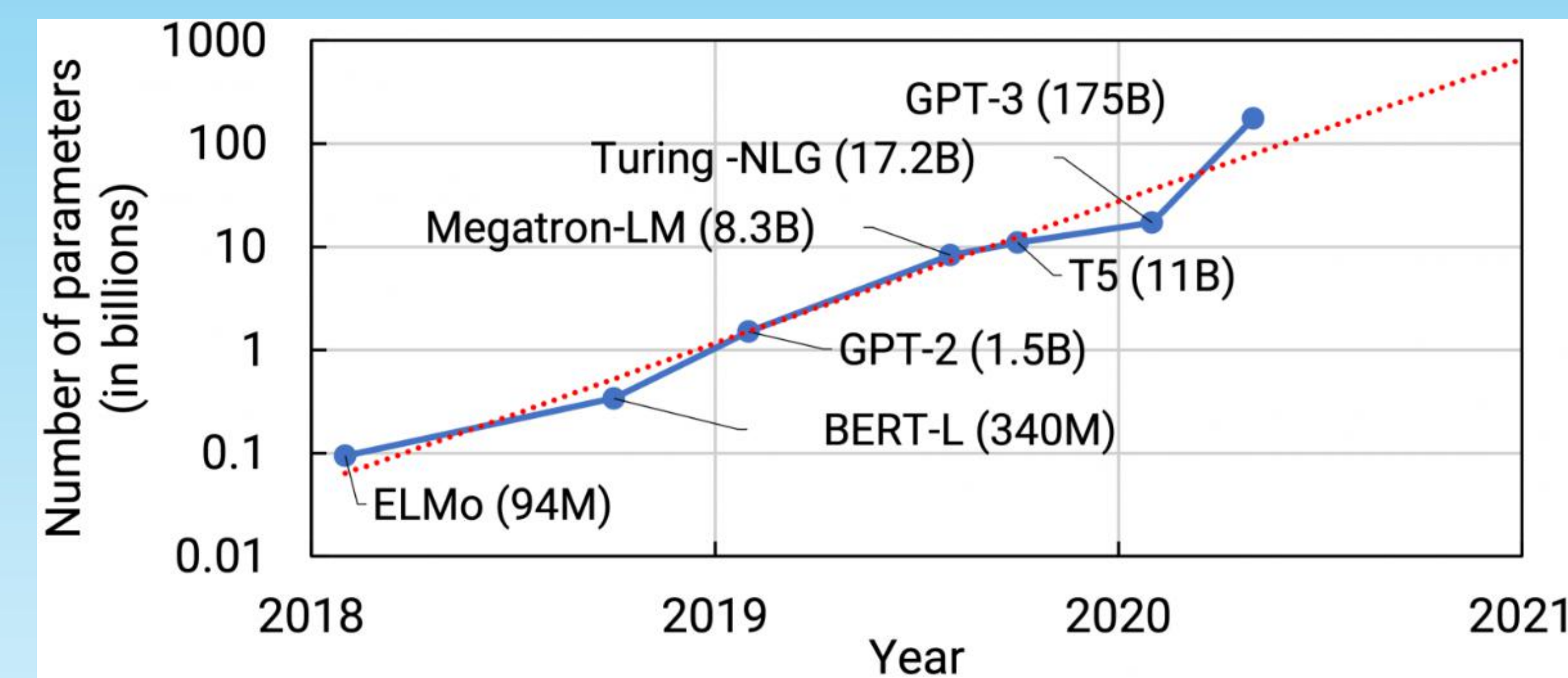
第3讲预览 (1/3): Overview of LLM Software Framework

Basics



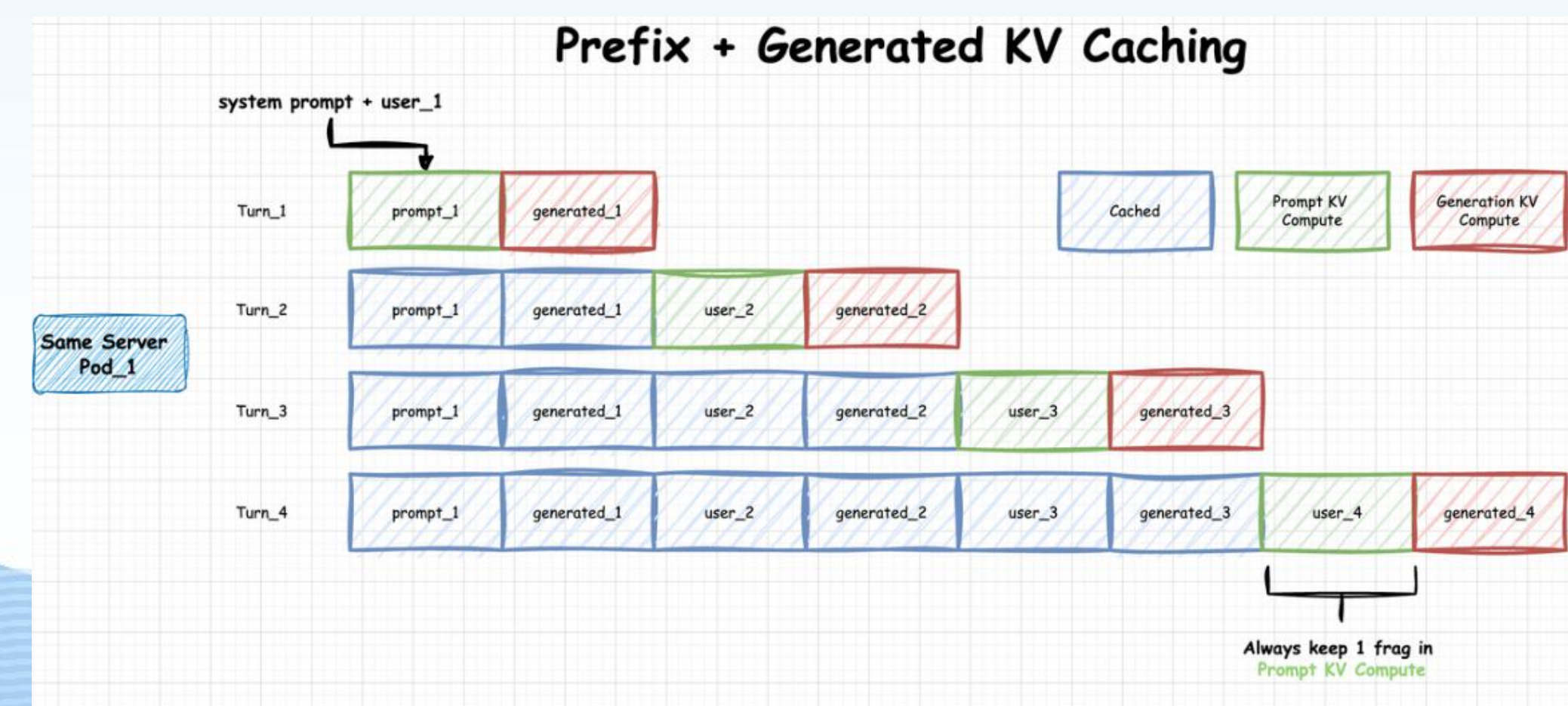
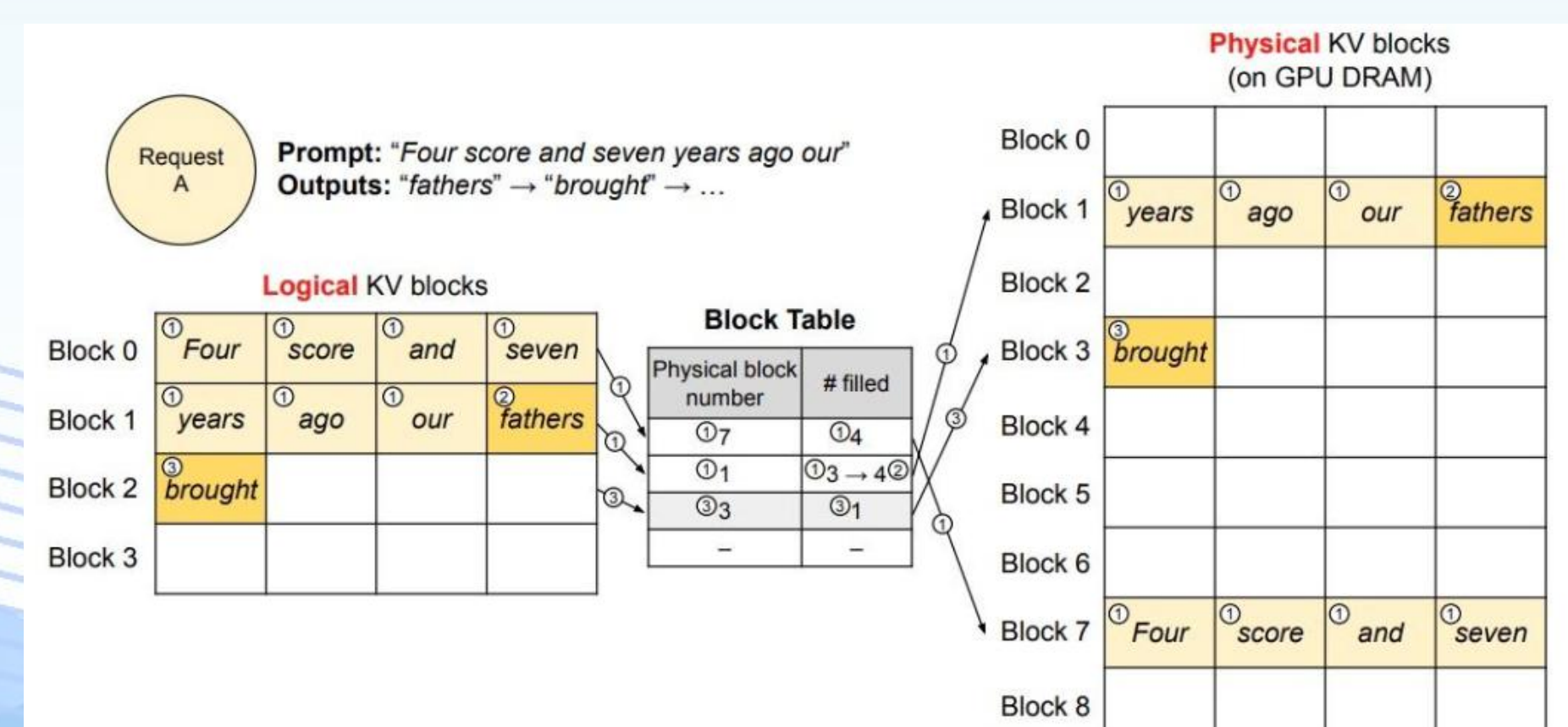
PyTorch & Tensorflow
DSL&API, Graph, AutoDiff, Compiler, CodeGen

Pre-train



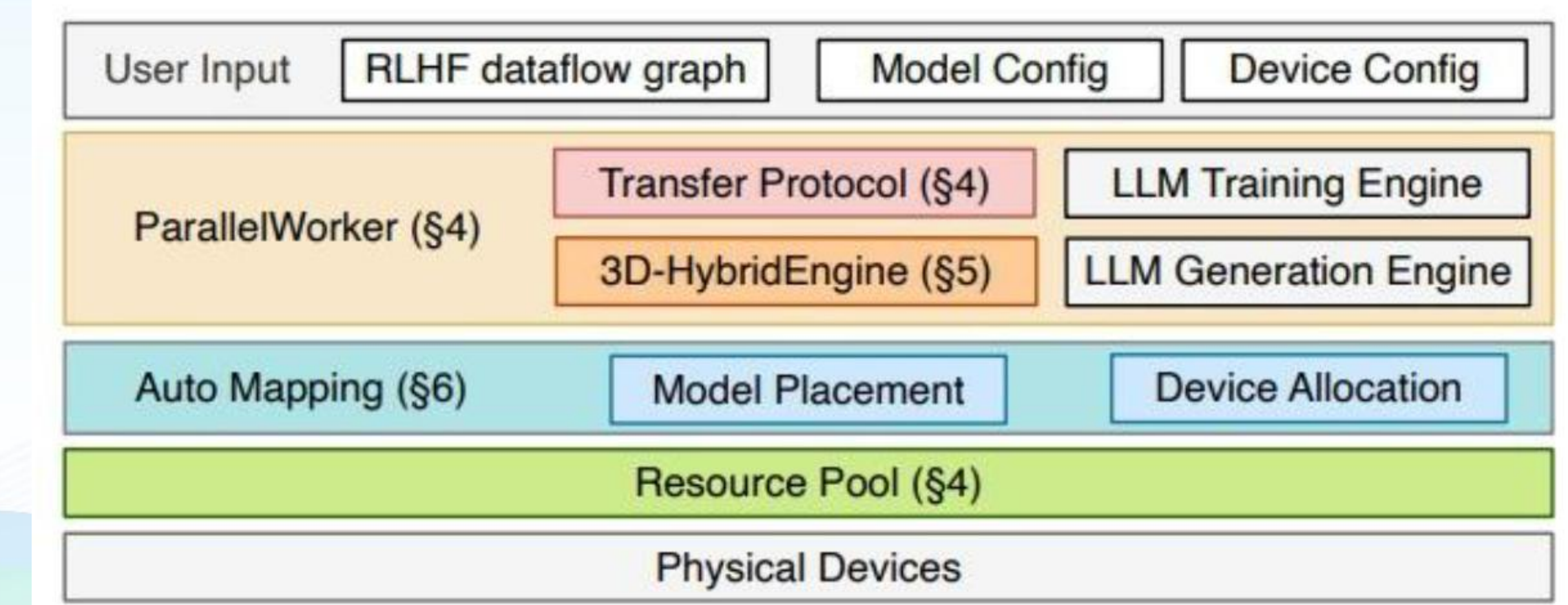
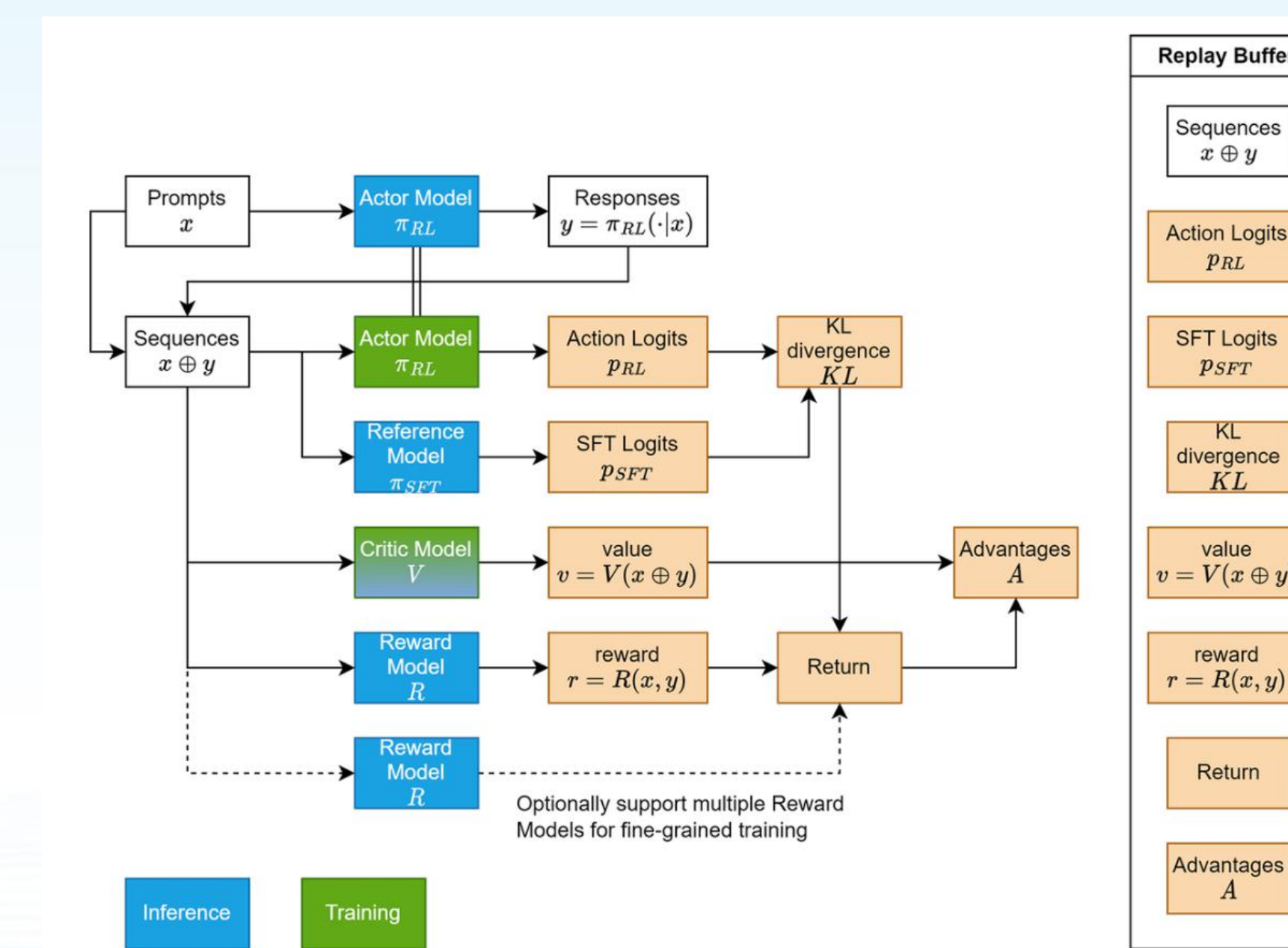
Megatron-LM & Accelerate
Parallelism Settings, FP/BP Optimization, Redundancy

Serve



vLLM & SGLang
Paged Attention, Prefix Caching, Speculative Decoding

Post-train

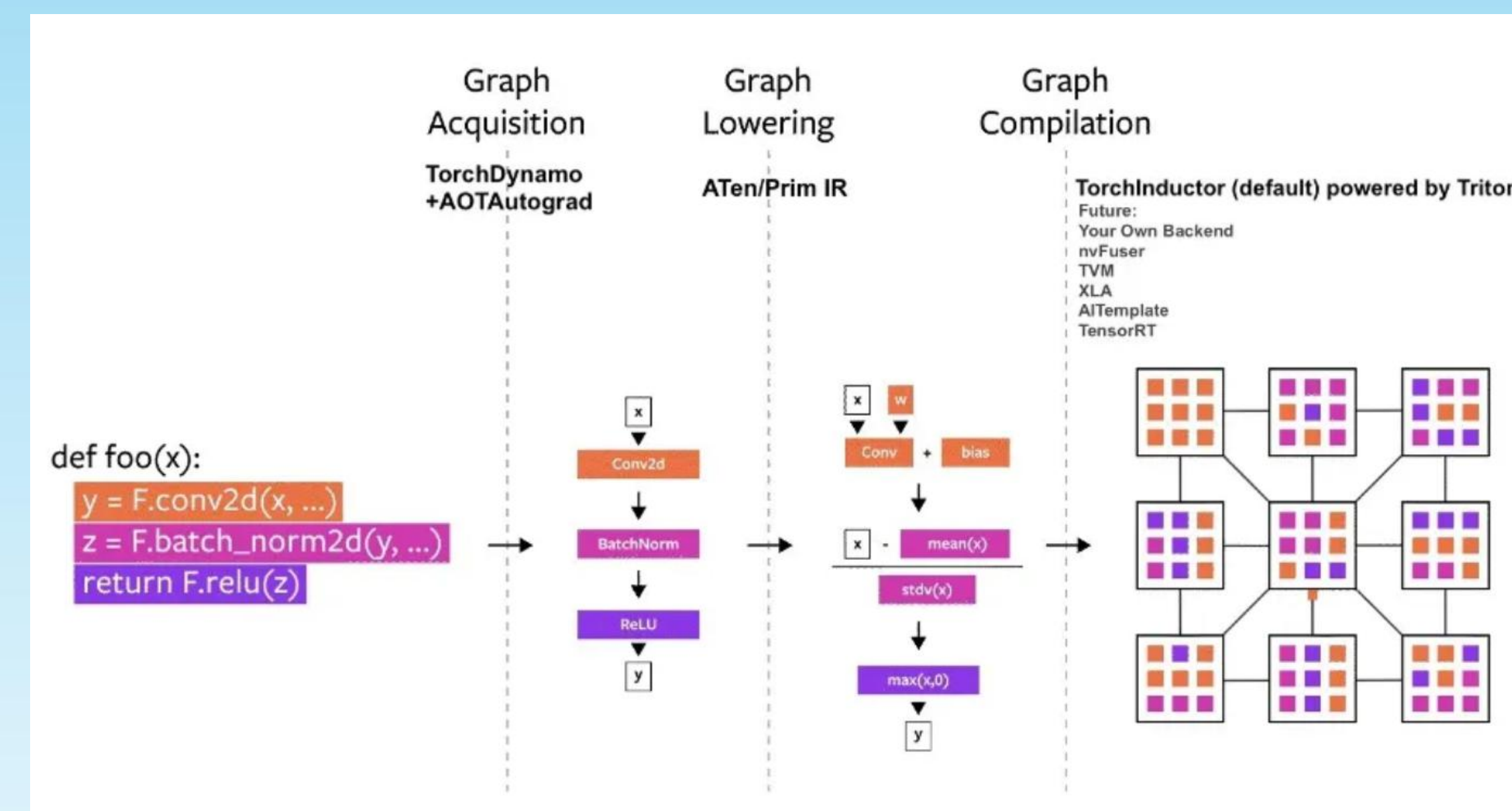
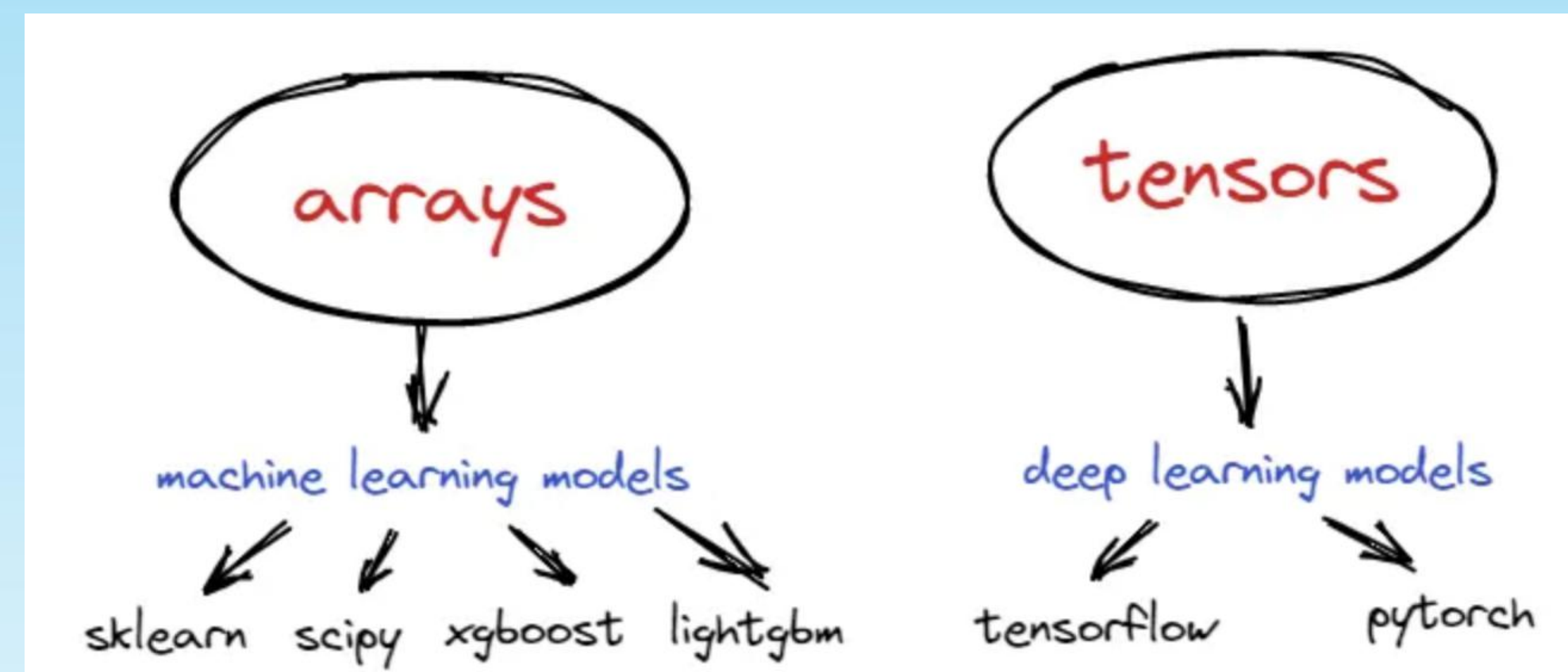


OpenRLHF & veRL
Controller & Dataflow Opt, Efficiency & Flexibility

第3讲预览 (2/3): Key Concepts of LLM Software Framework

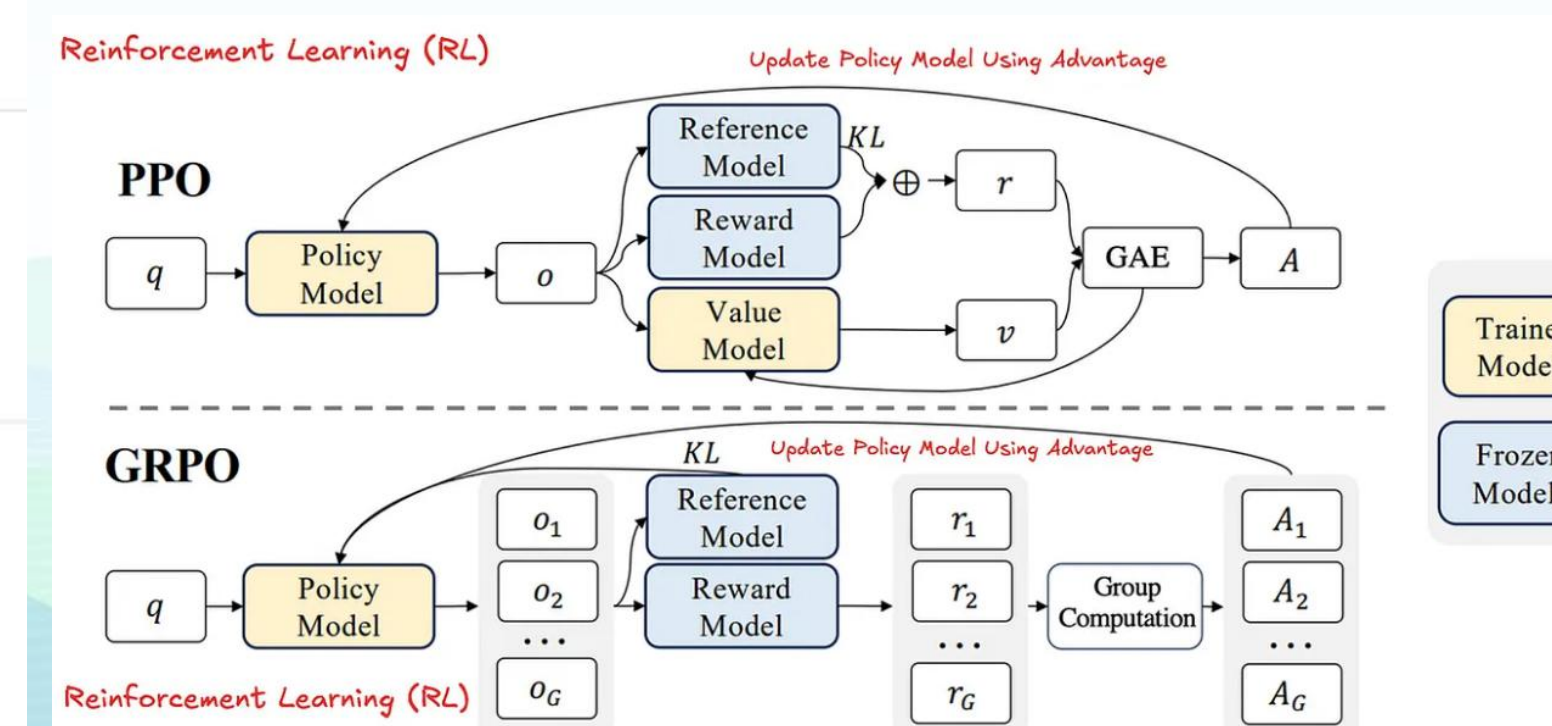
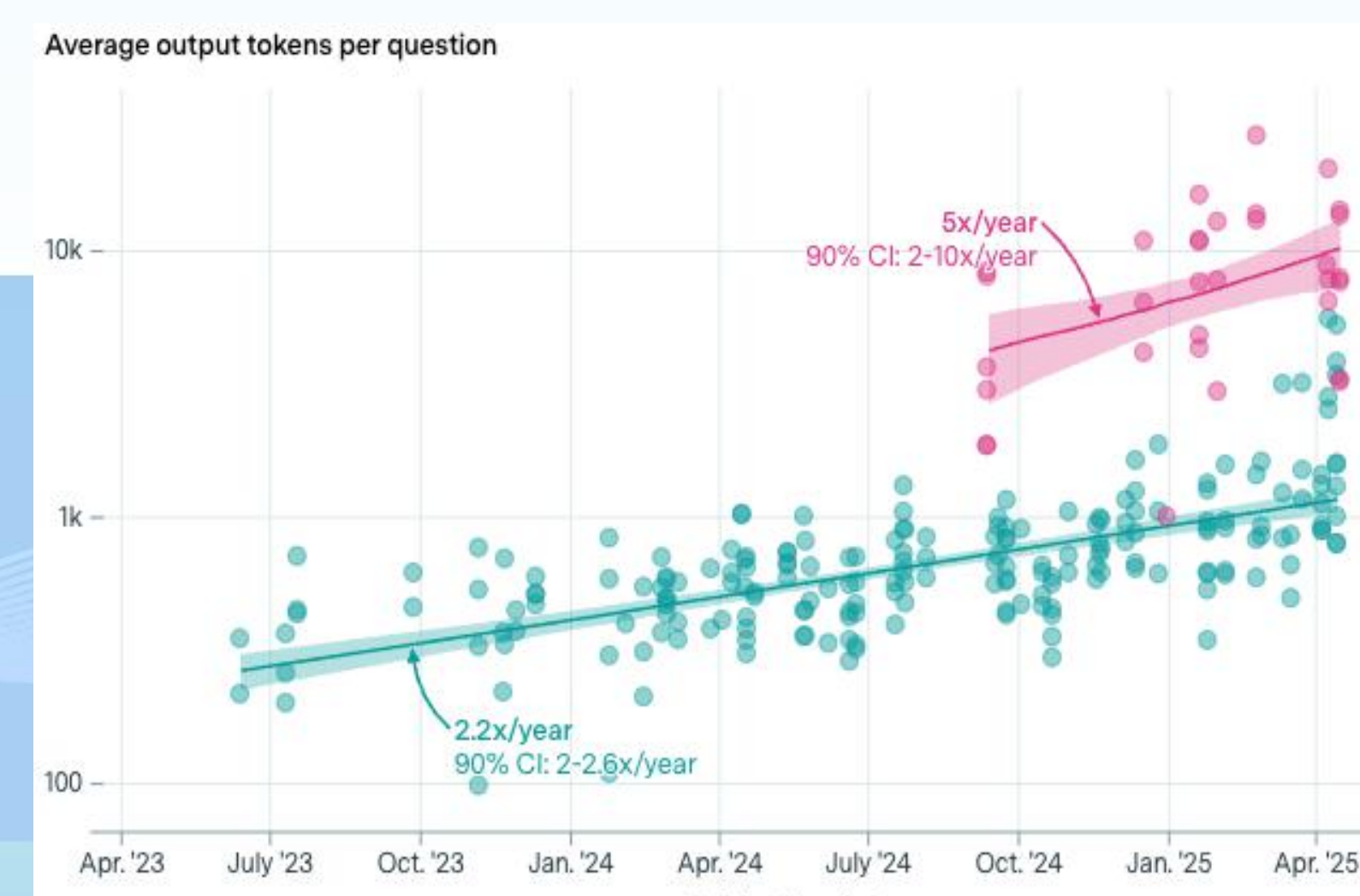
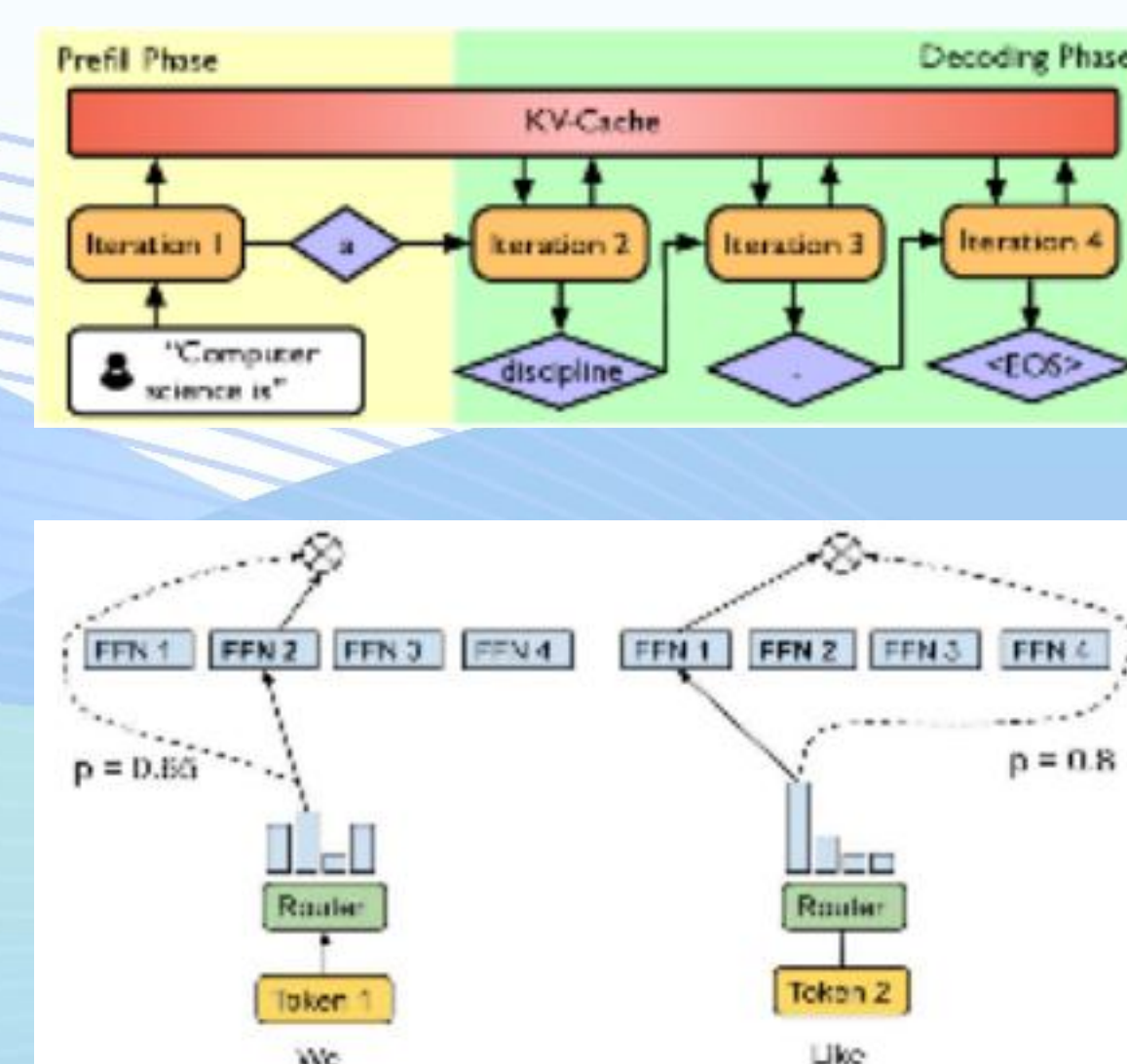
Basics

Features: Automation, Abstraction, Backend Portability
Programming Model: Eager Execution v.s. Static Graph



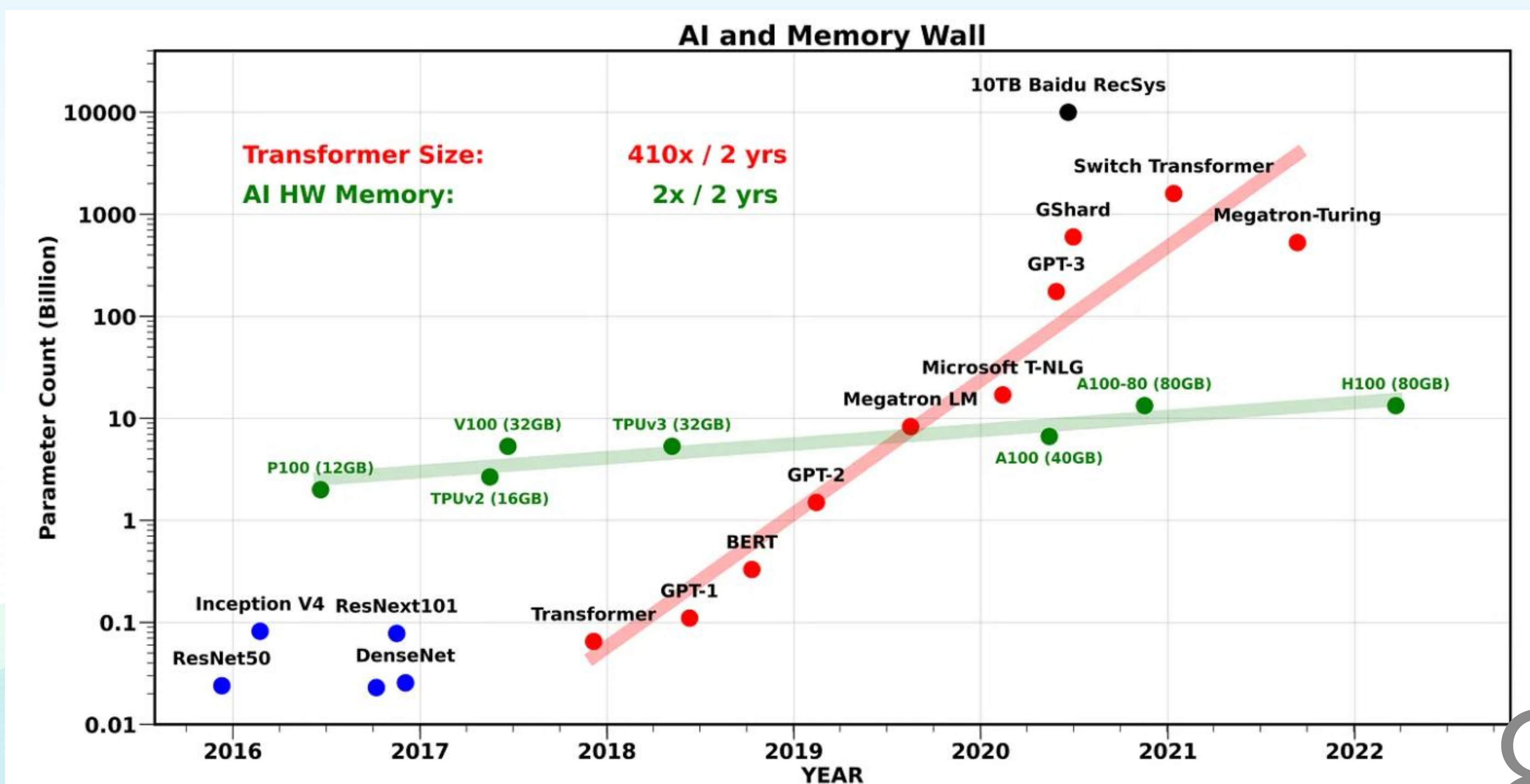
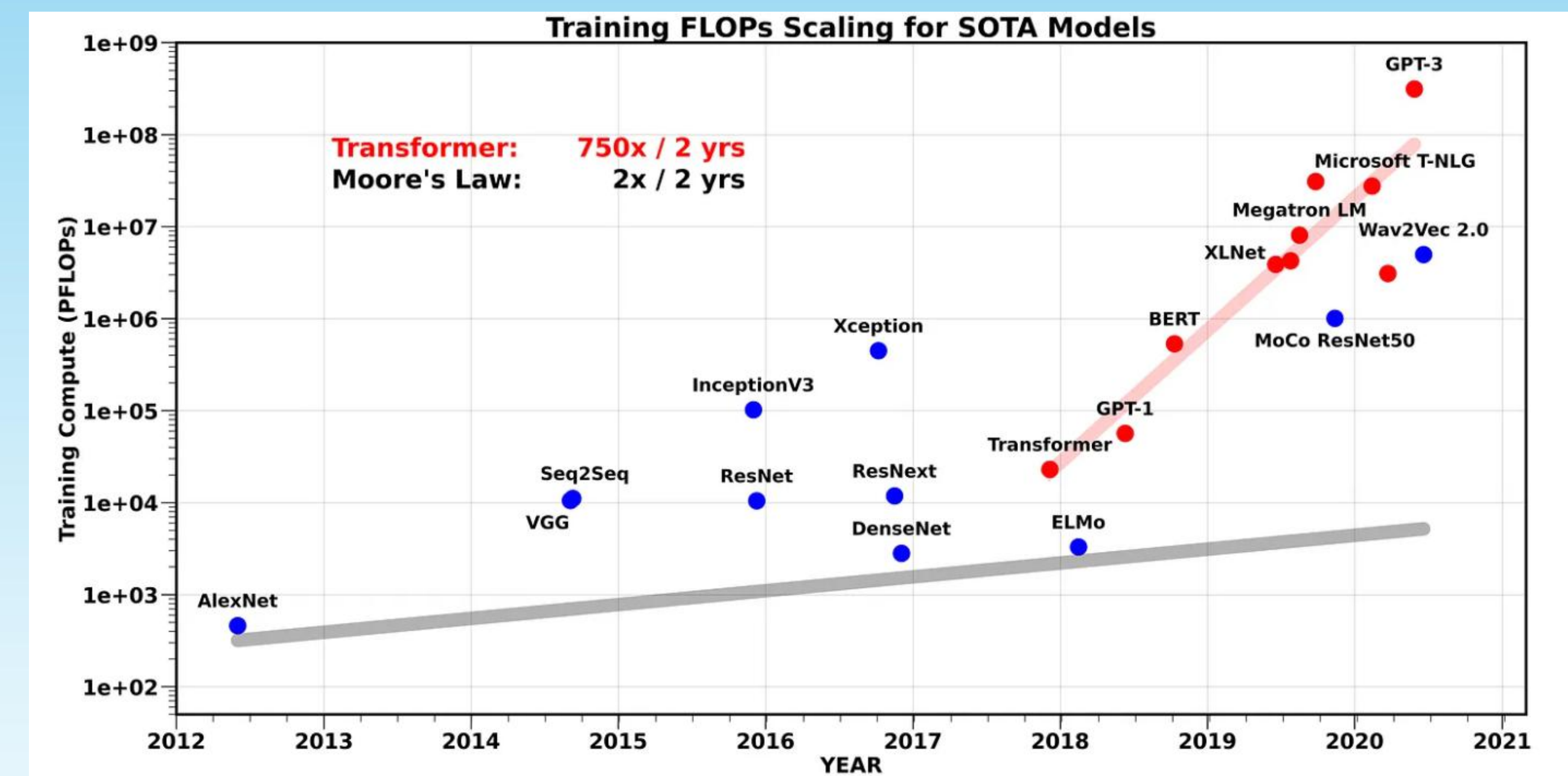
Algo.

Runtime: Dynamicity at the inference time.
Post-training: DPO, PPO, GRPO, and etc.



System

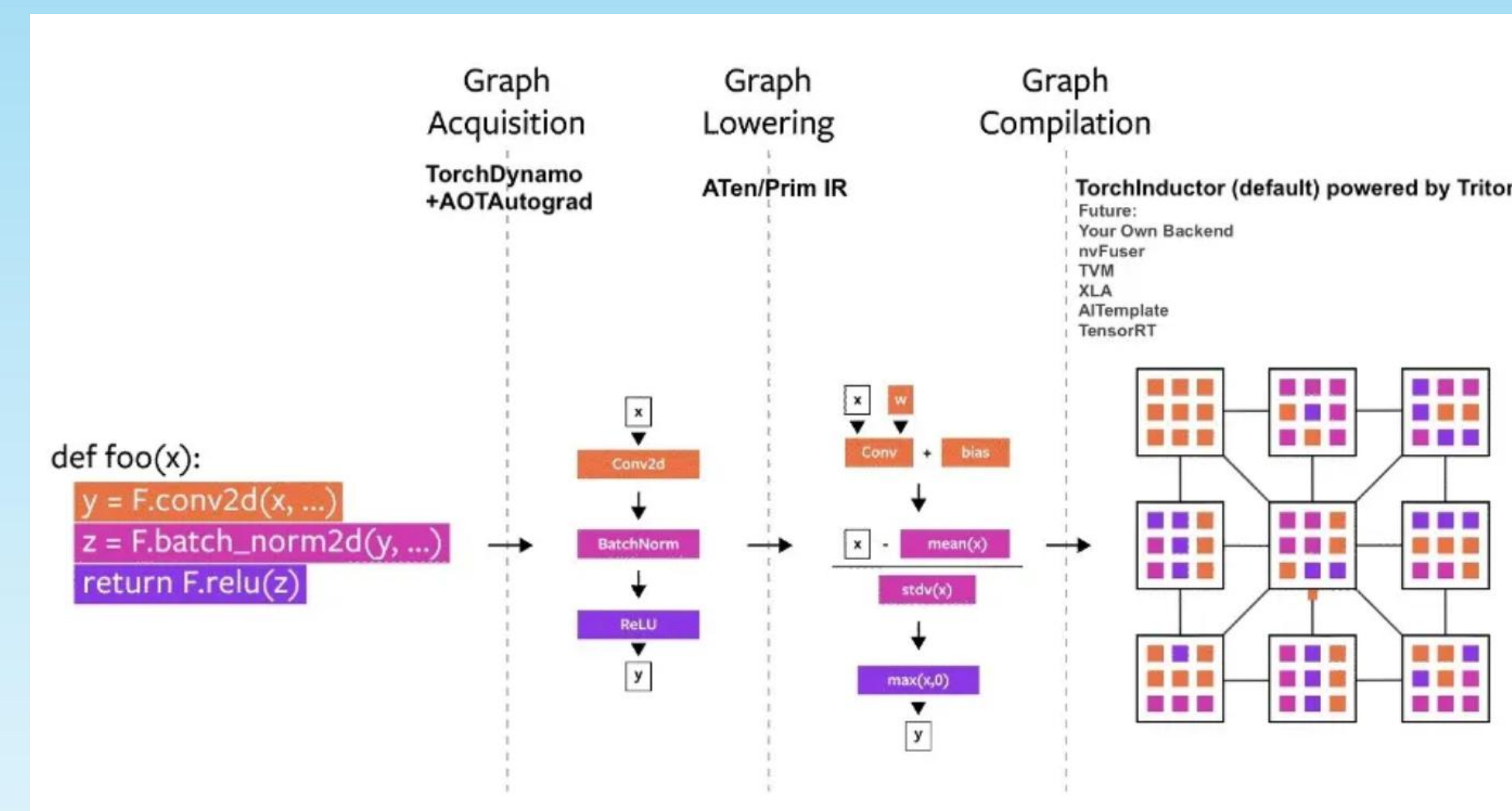
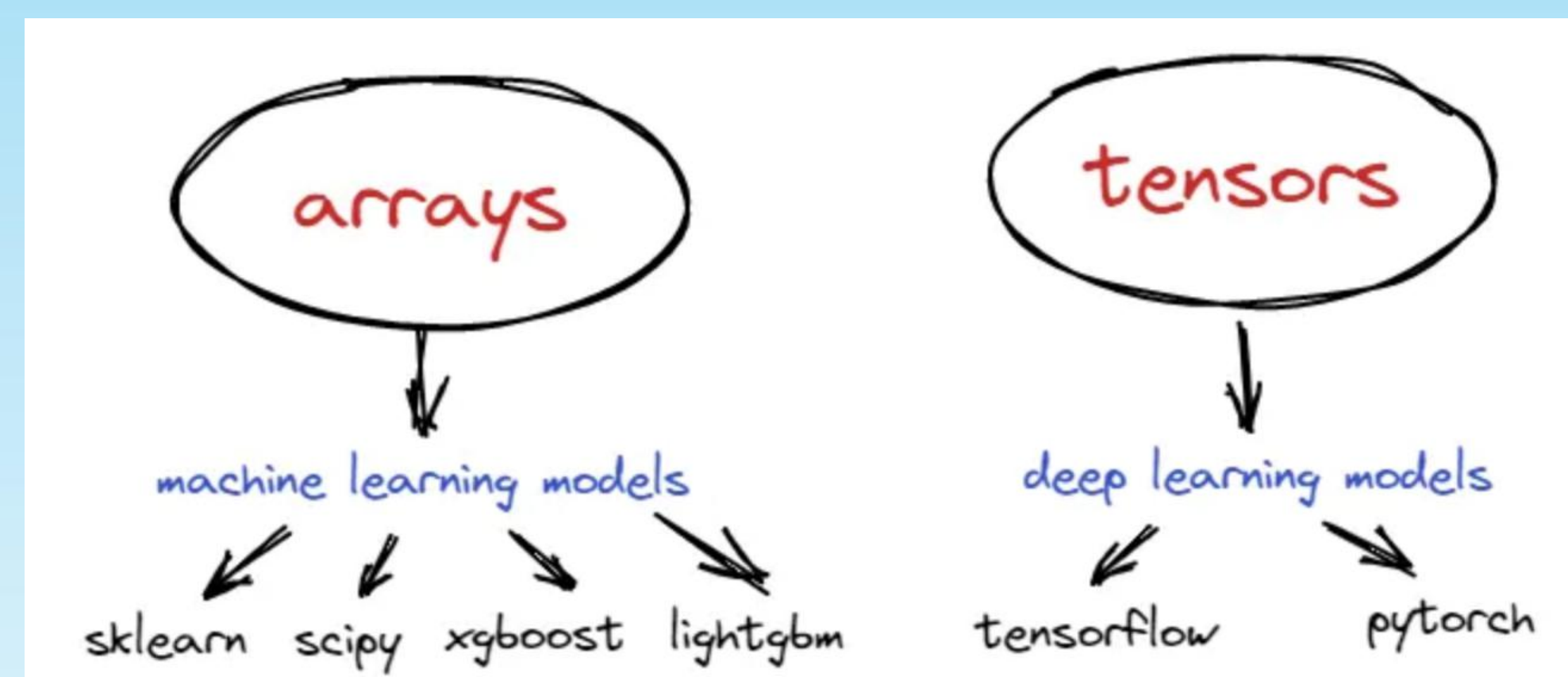
Computation, Memory, and Communication Wall



第3讲预览 (2/3): Key Concepts of LLM Software Framework

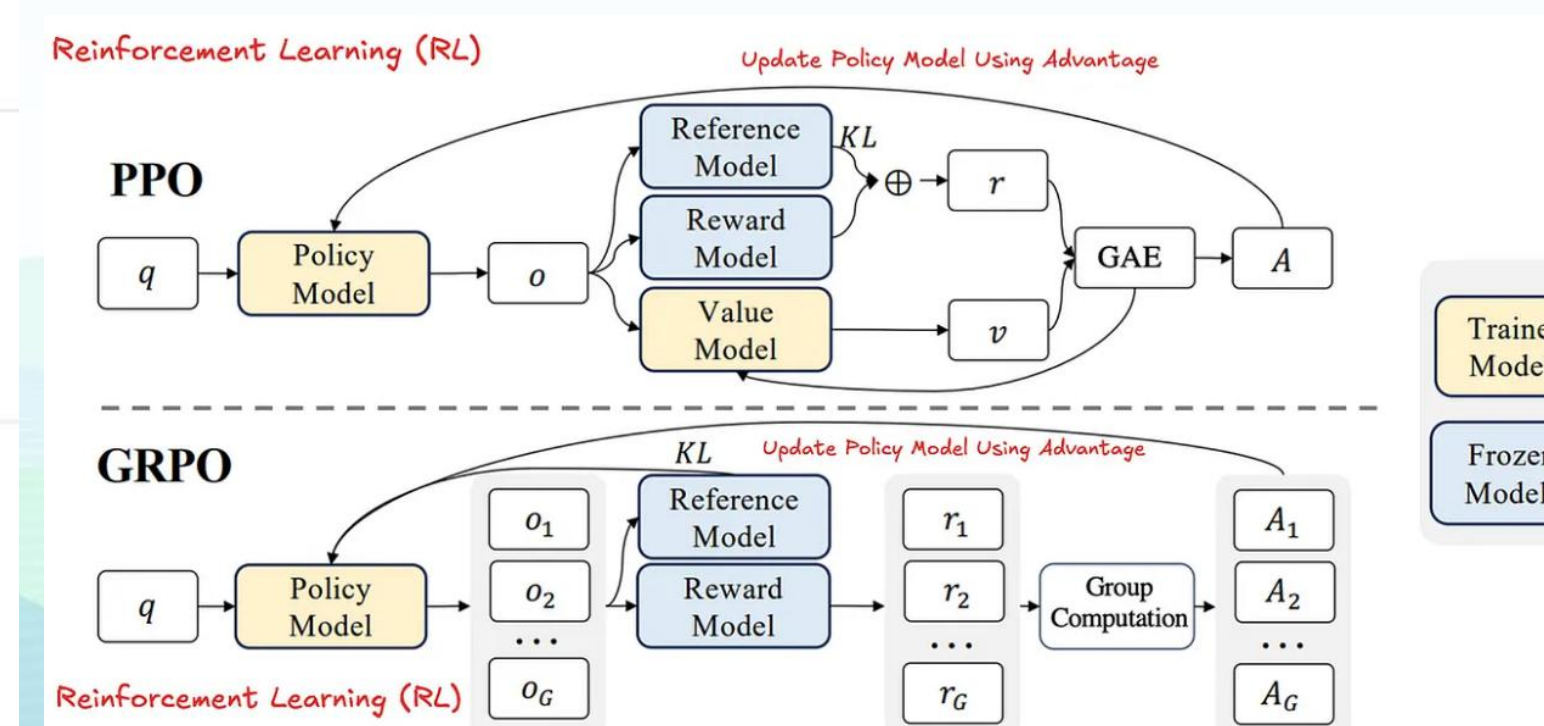
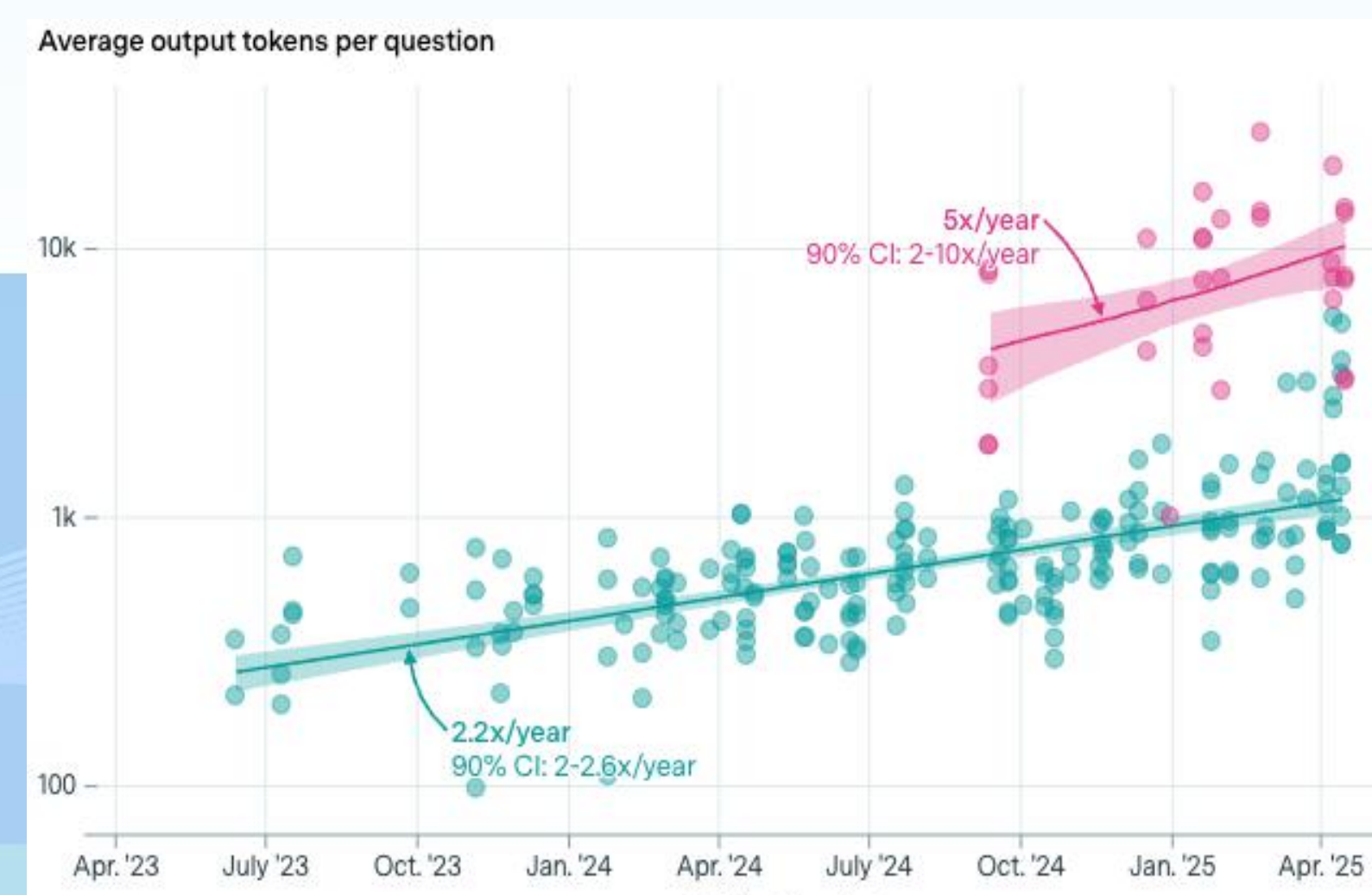
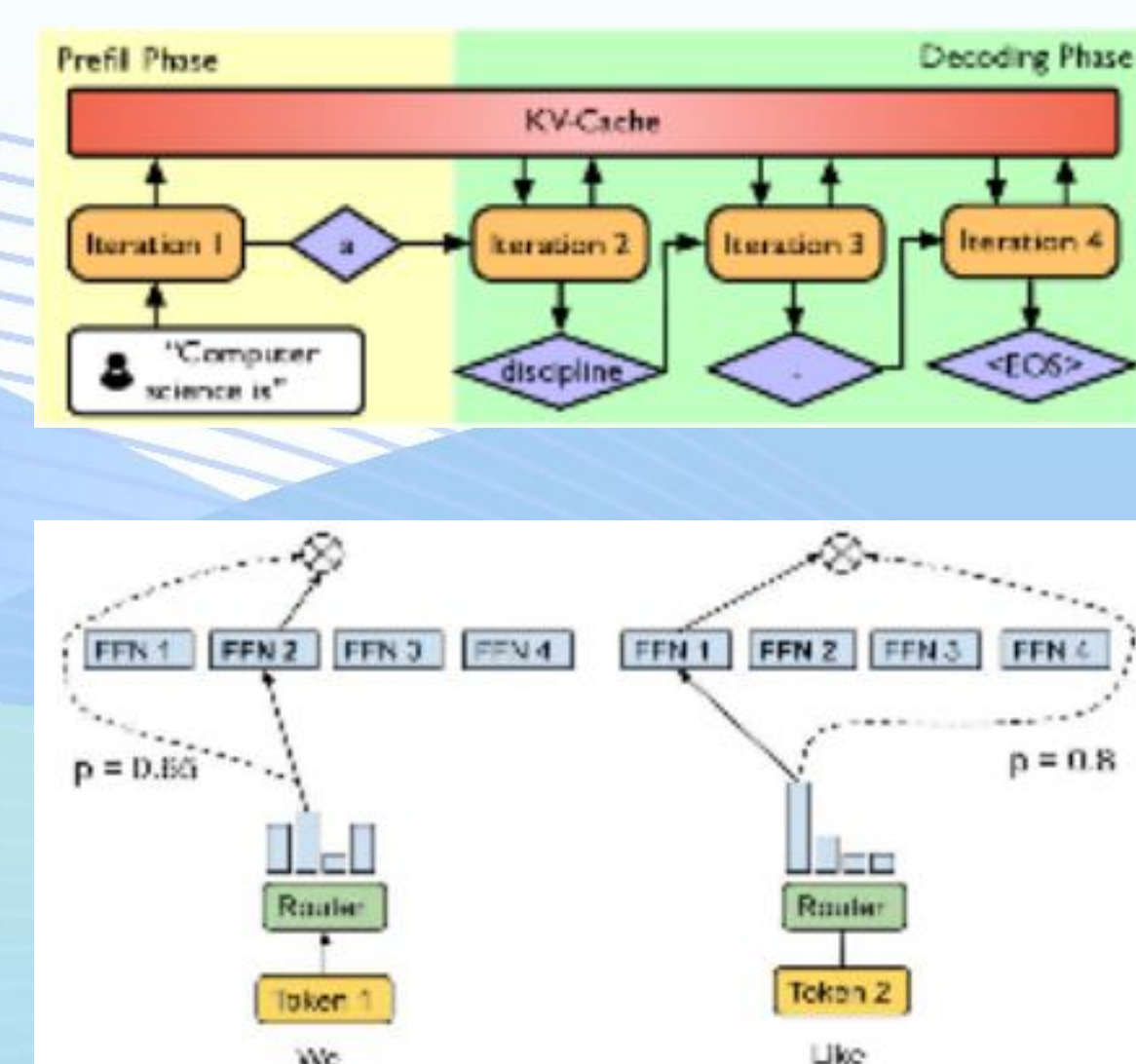
Basics

Features: Automation, Abstraction, Backend Portability
Programming Model: Eager Execution v.s. Static Graph



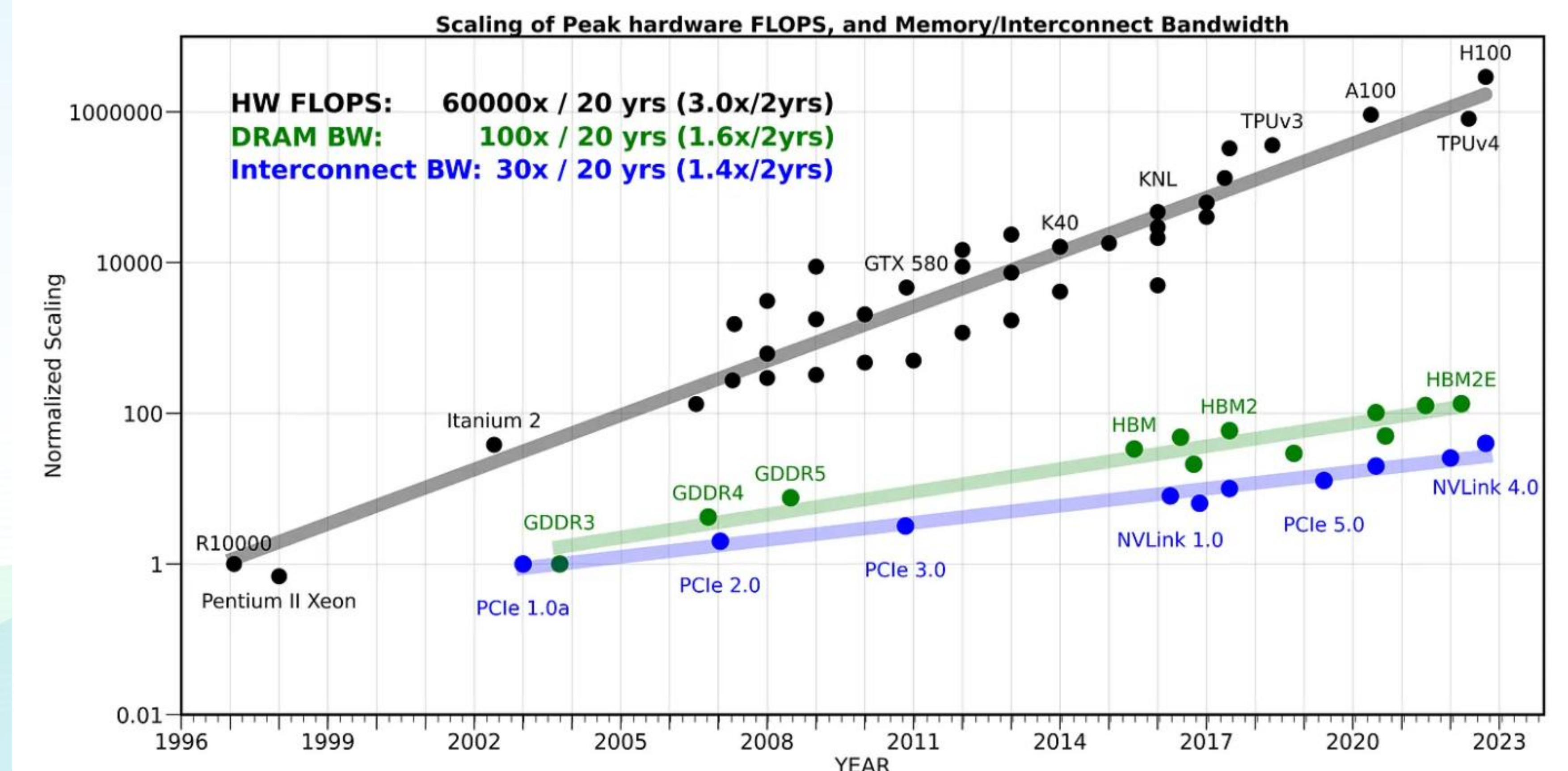
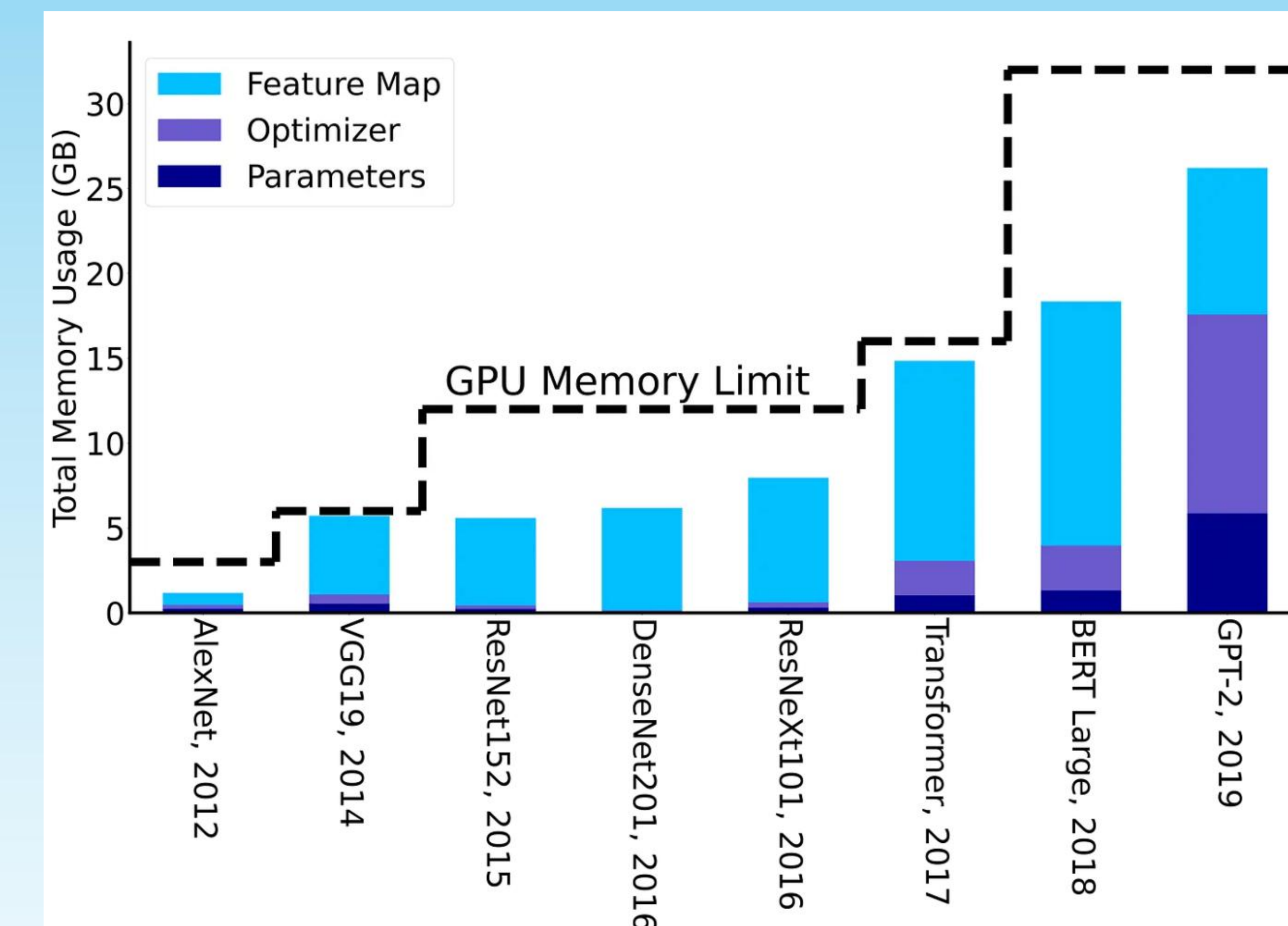
Algo.

Runtime: Dynamicity at the inference time.
Post-training: DPO, PPO, GRPO, and etc.



System

Computation, Memory, and Communication Wall



第3讲预览 (3/3): SGLang (Serving)

Programming Model

Abstraction with structured representations.

```
dimensions = ["Clarity", "Originality", "Evidence"]
@function
def multi_dimensional_judge(s, path, essay):
    s += system("Evaluate an essay about an image.")
    s += user(image(path) + "Essay:" + essay)
    s += assistant("Sure!")

    # Return directly if it is not related
    s += user("Is the essay related to the image?")
    s += assistant(select("related", choices=["yes", "no"]))
    if s["related"] == "no": return

    # Judge multiple dimensions in parallel
    forks = s.fork(len(dimensions))
    for f, dim in zip(forks, dimensions):
        f += user("Evaluate based on the following dimension:" +
            dim + ". End your judgment with the word 'END'")
        f += assistant("Judgment:" + gen("judgment", stop="END"))

    # Merge the judgments
    judgment = "\n".join(f["judgment"] for f in forks)

    # Generate a summary and a grade. Return in the JSON format.
    s += user("Provide the judgment, summary, and a letter grade")
    s += assistant(judgment + "In summary," + gen("summary", stop=".")
        + "The grade of it is" + gen("grade"))

    schema = r'\{"summary": "[\w\d\s]+\.", "grade": "[ABCD][+-]?"\}'
    s += user("Return in the JSON format.")
    s += assistant(gen("output", regex=schema))

state = multi_dimensional_judge.run(...)
print(state["output"])
```

Handle chat template and multi-modal inputs

Select an option with the highest probability

Fetch result; Use Python control flow

Runtime optimization: KV Cache Reuse (Sec. 3)

Multiple generation calls run in parallel

Fetch generation results

Runtime optimization: API speculative execution (Sec. 5)

Runtime optimization: fast constrained decoding (Sec. 4)

Run an SGLang program

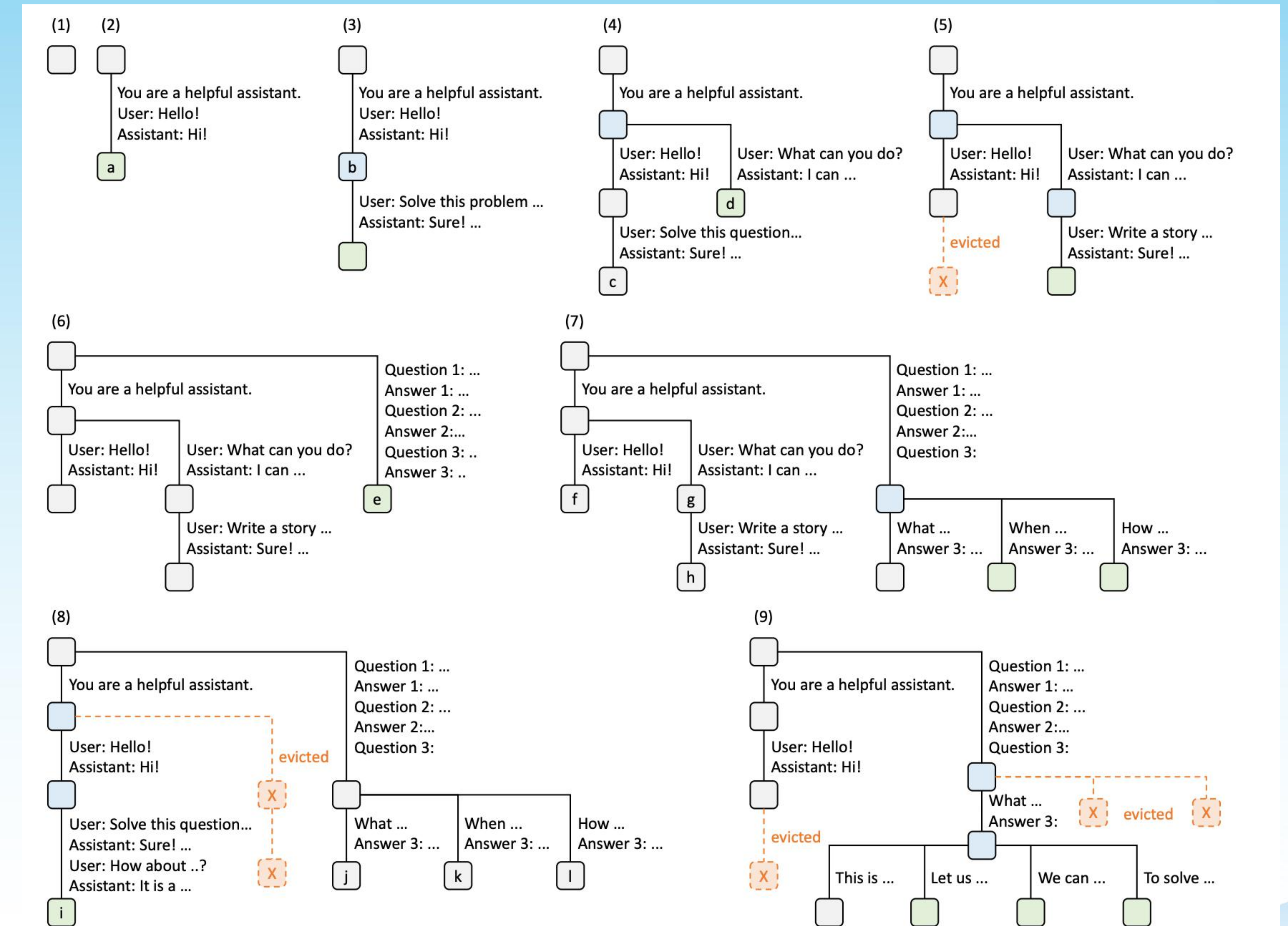
Table 1: Comparison among LMQL, Guidance, and SGLang.

System	Syntax	Language Primitives	Runtime Backends
LMQL	Custom	extend, gen, select	HF Transformers, llama.cpp, OpenAI
Guidance	Python	extend, gen, select, image	HF Transformers, llama.cpp, OpenAI
SGLang	Python	extend, gen, select, image, video, fork, join	SGLang Runtime (SRT), OpenAI

An example of the programming model in SGLang.

KV Cache Management

Optimize the computation wall & memory wall.

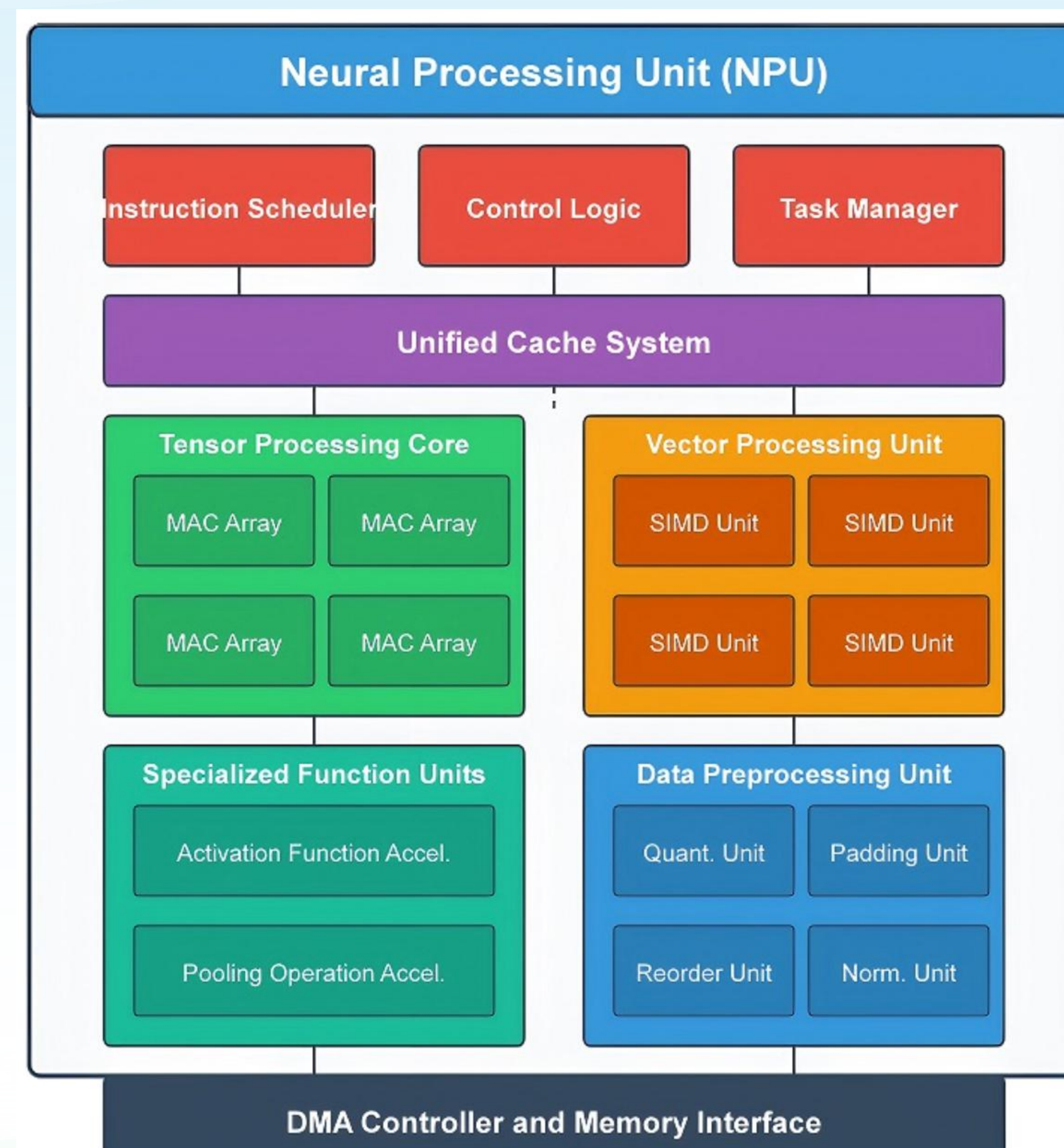


Theorem 3.1. For a batch of requests, we can achieve an optimal cache hit rate by visiting the radix tree of the requests in the depth-first search order, with a cache size $\geq$ the maximum request length. The longest-shared-prefix-first order is equivalent to a depth-first search order.

An example of the RadixAttention with LRU policy.*

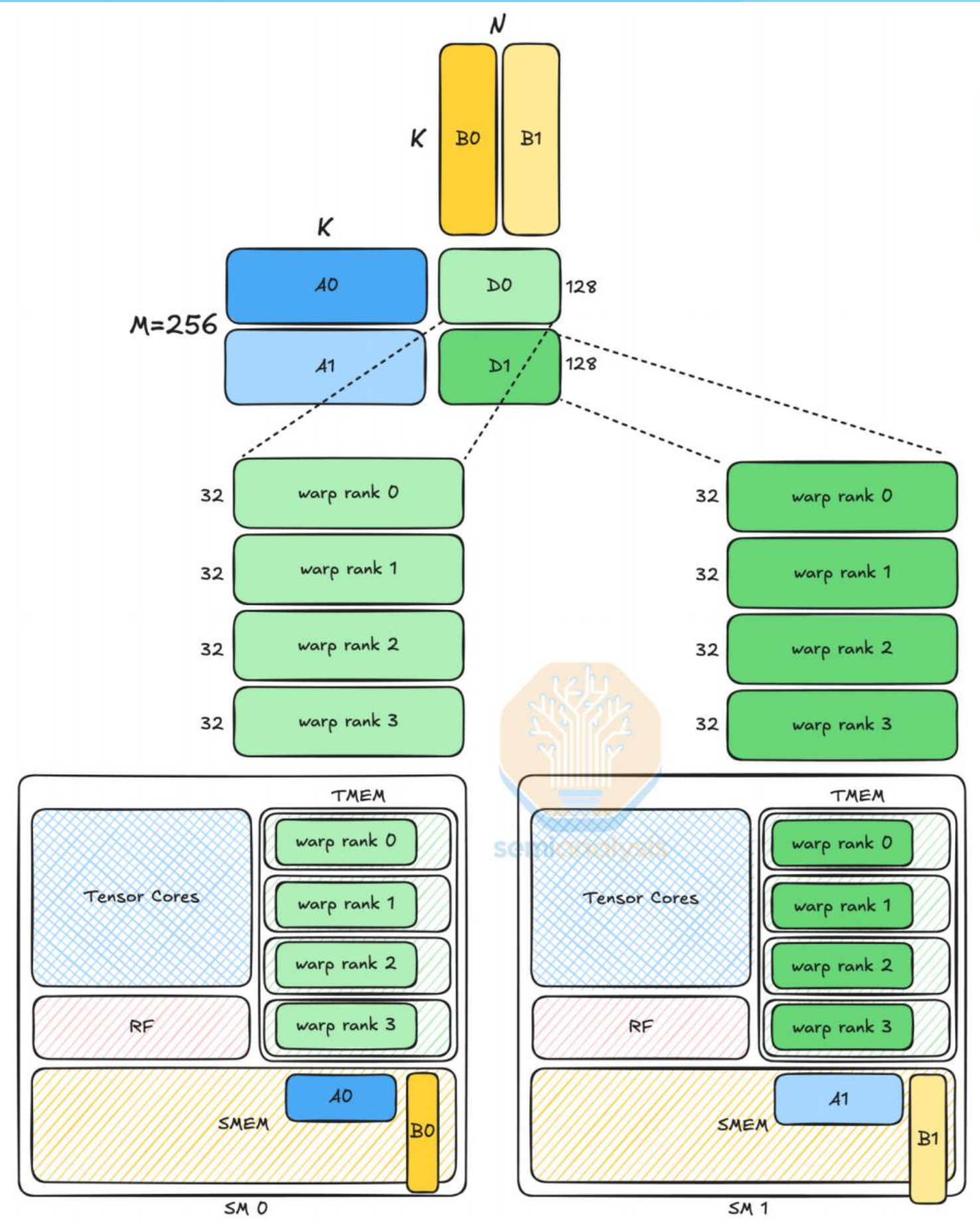
第4讲预览 (1/2): ML Compute Subsystem

- Hardware Demands of AI
 - Characteristics of AI workloads
 - Core compute and memory primitives
- The Evolution of AI Processors
 - Emergence of heterogeneous computing
- Foundational Principles
 - Key architectural components
- Representative NPU Architectures
 - NVIDIA Tensor Core, Intel Advanced Matrix Extensions (AMX), AMD AI Engine (AIE), Huawei DaVinci Architecture, NVDLA, Tenstorrent, ...



第4讲预览 (2/2): Core Technologies in Modern NPUs

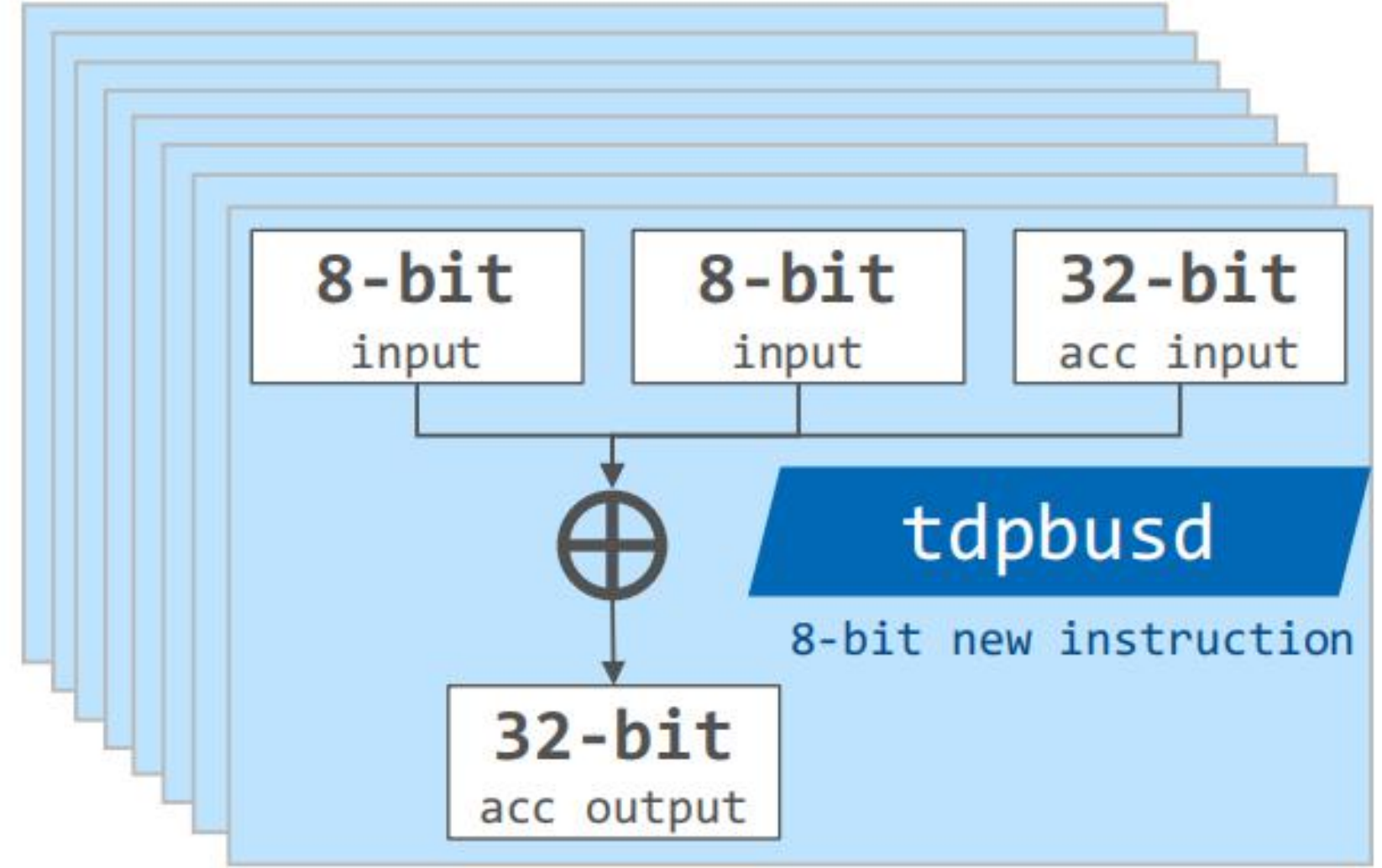
Architecture	Core Technology
NV Tensor Core	Mixed-Precision MAC, Systolic Array
Intel AMX	Tile Matrix Multiplication (TMUL)
Intel/AMD ACE	
AMD AIE	VLIW/SIMD Vector Processor Array
HUAWEI DaVinci	1D Scalar, 2D Vector, 3D Cube
Tenstorrent	RISC-V, Explicit Data Movement



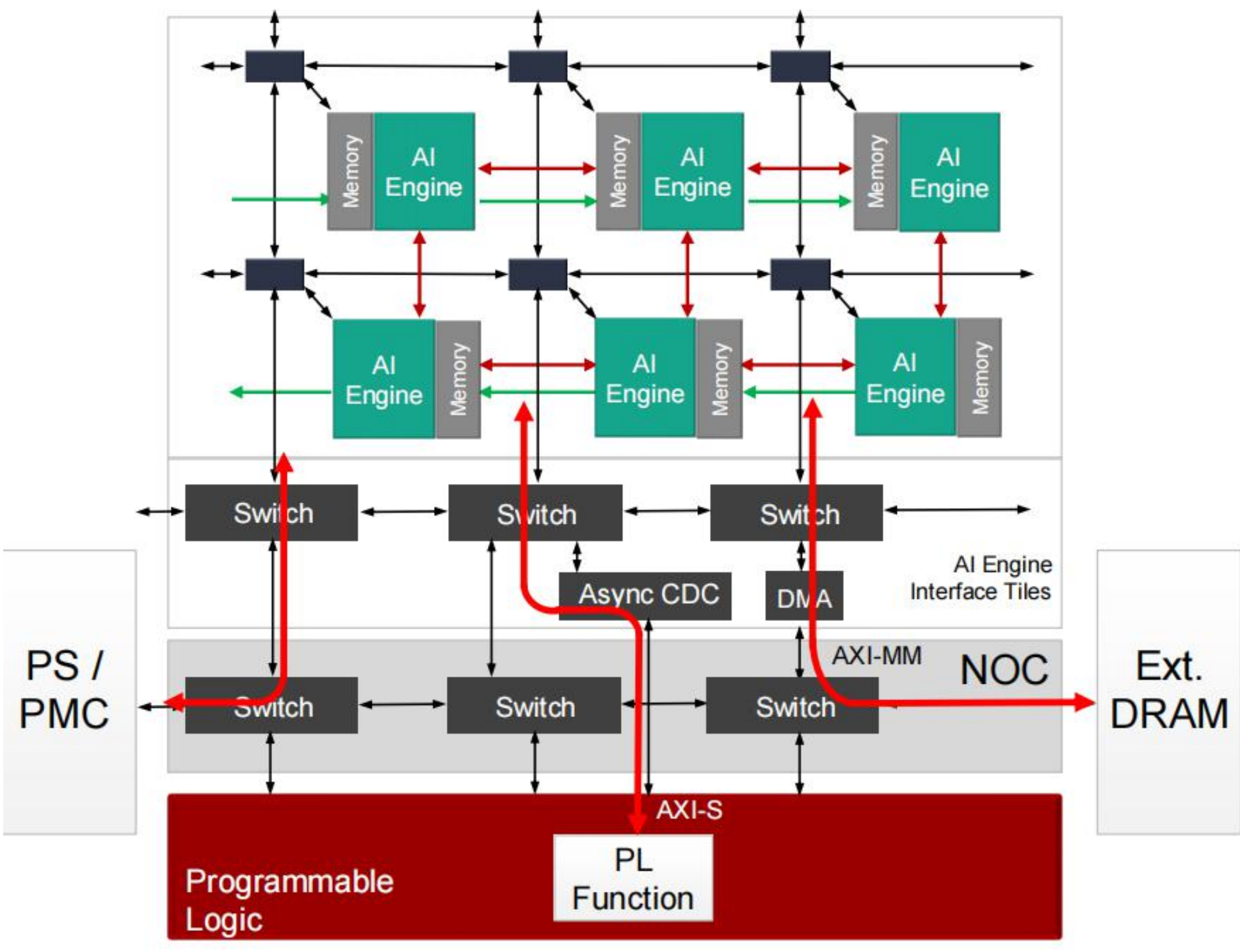
NV Tensor Core

Intel® AMX

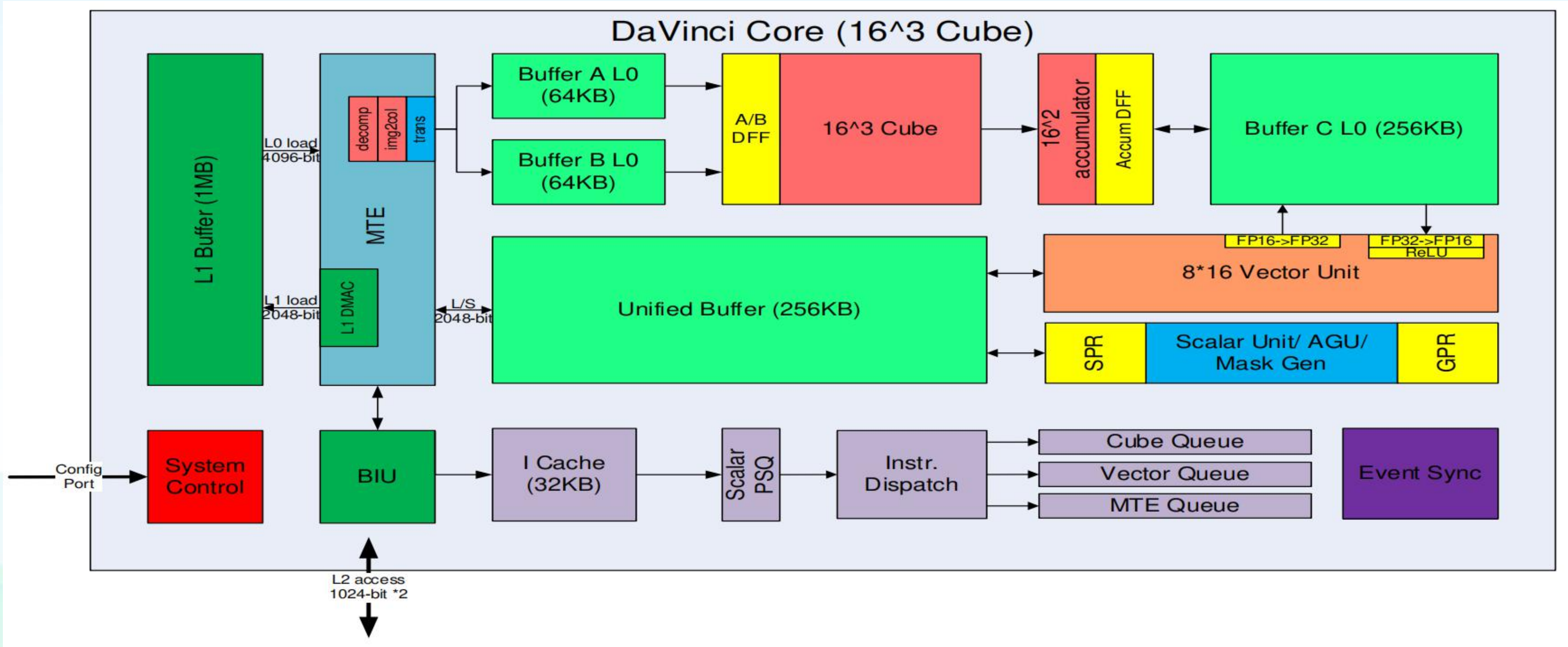
2048 INT8 ops/cycle/core
Multi-fold MACs in one instruction



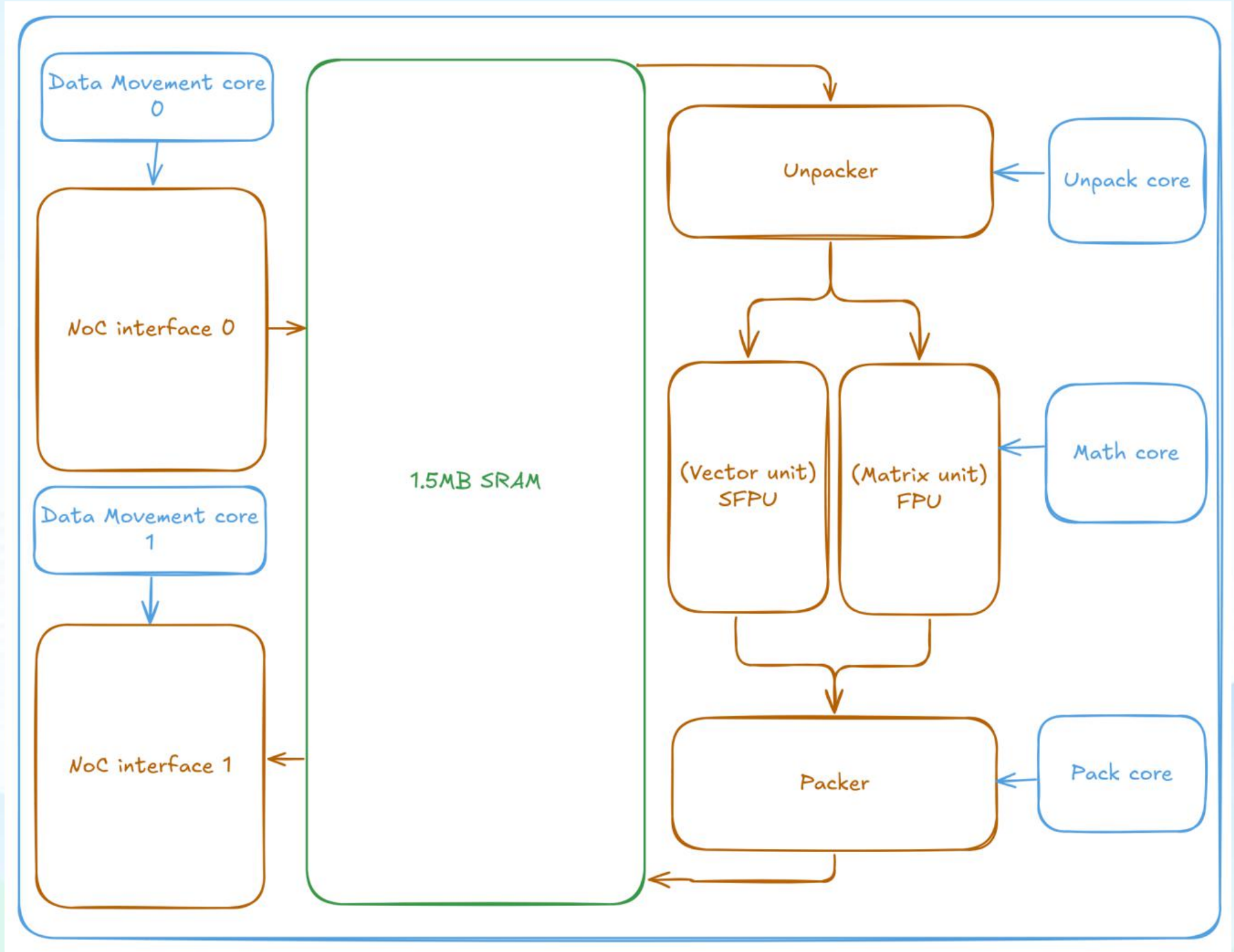
Intel AMX



AMD AIE



HUAWEI DaVinci



Tenstorrent



第5讲预览 (1/2): ML Compilation Techniques

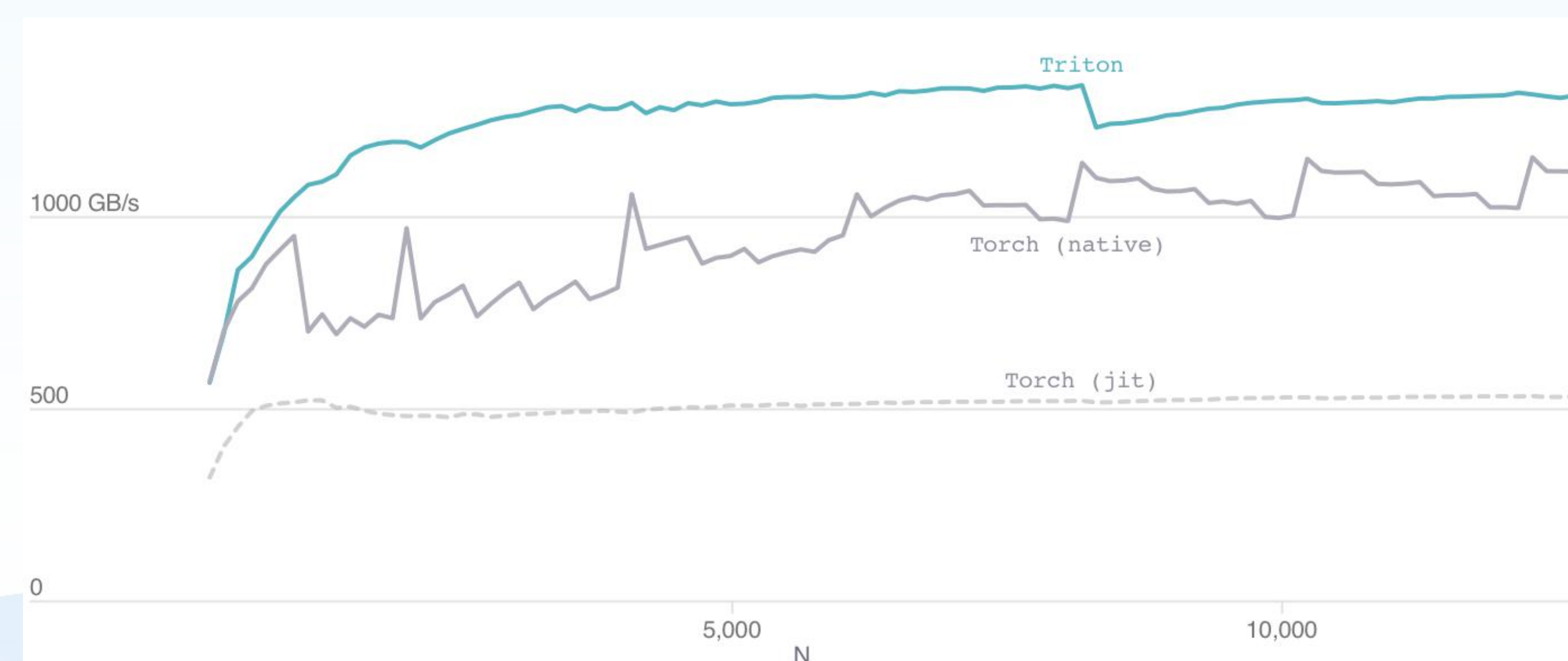
- General flow of AI compilation
 - From high-level framework to hardware instructions
 - Torch.compile and triton examples
- Intermediate representations
 - MLIR
 - TVM Relay IR and TIR
- Optimization techniques
 - Graph optimization (Control Data Graph)
 - Operator merging (CUDA)
 - Tensor program optimization: vectorization, loop transformation

第5讲预览 (2/2): Triton Compilation Flow

• Comparison with CUDA

	CUDA	TRITON
Memory Coalescing	Manual	Automatic
Shared Memory Management	Manual	Automatic
Scheduling (Within SMs)	Manual	Automatic
Scheduling (Across SMs)	Manual	Manual

• Softmax optimization effect



• Write GPU code in a Numpy-like style

```
@jit
def add(X, Y, Z, N):
    pid = program_id(0)
    idx = pid * 512 + arange(512)
    mask = idx < N
    x = load(X + idx, mask=mask)
    y = load(Y + idx, mask=mask)
    store(Z + idx, x + y, mask=mask)
```

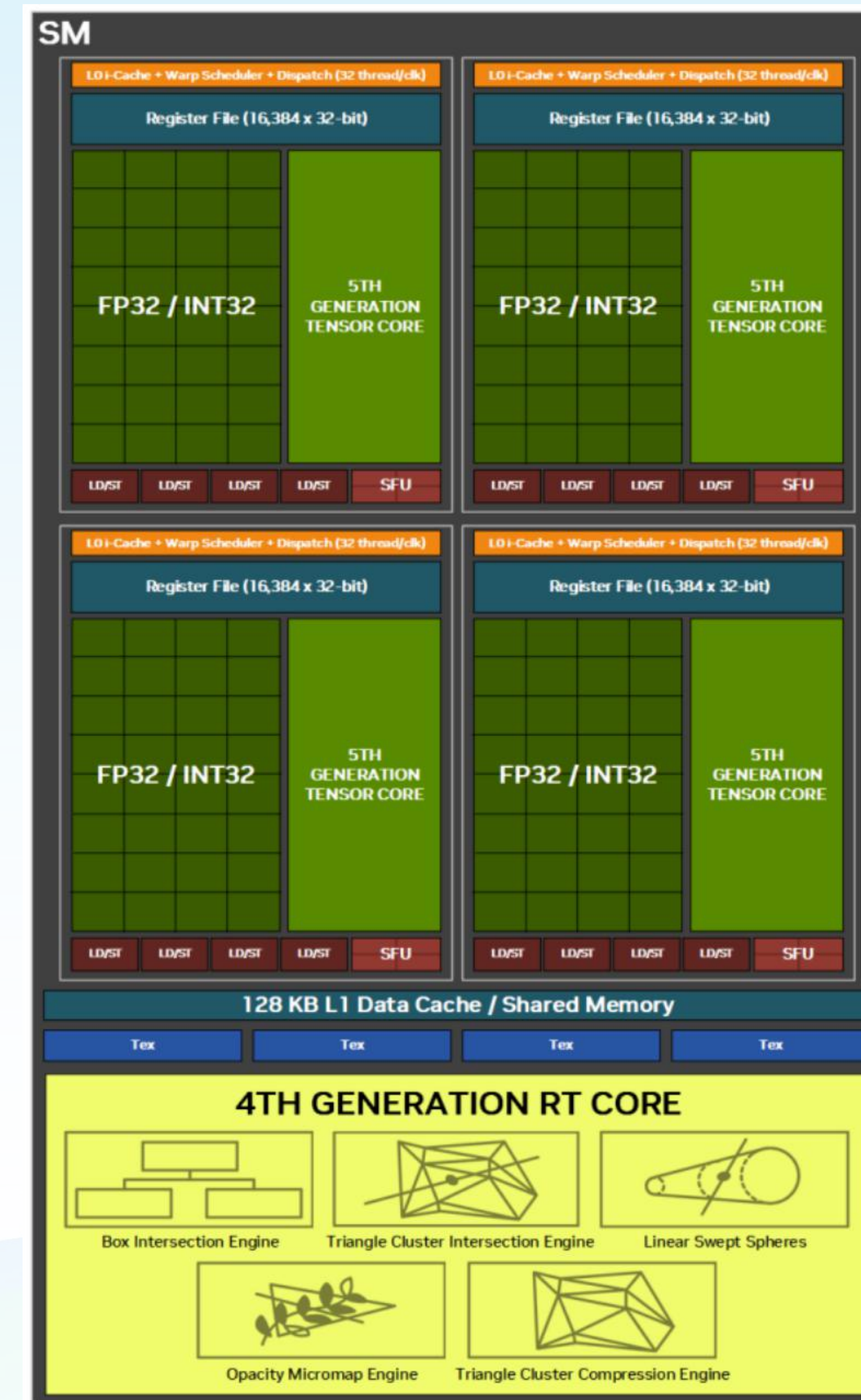
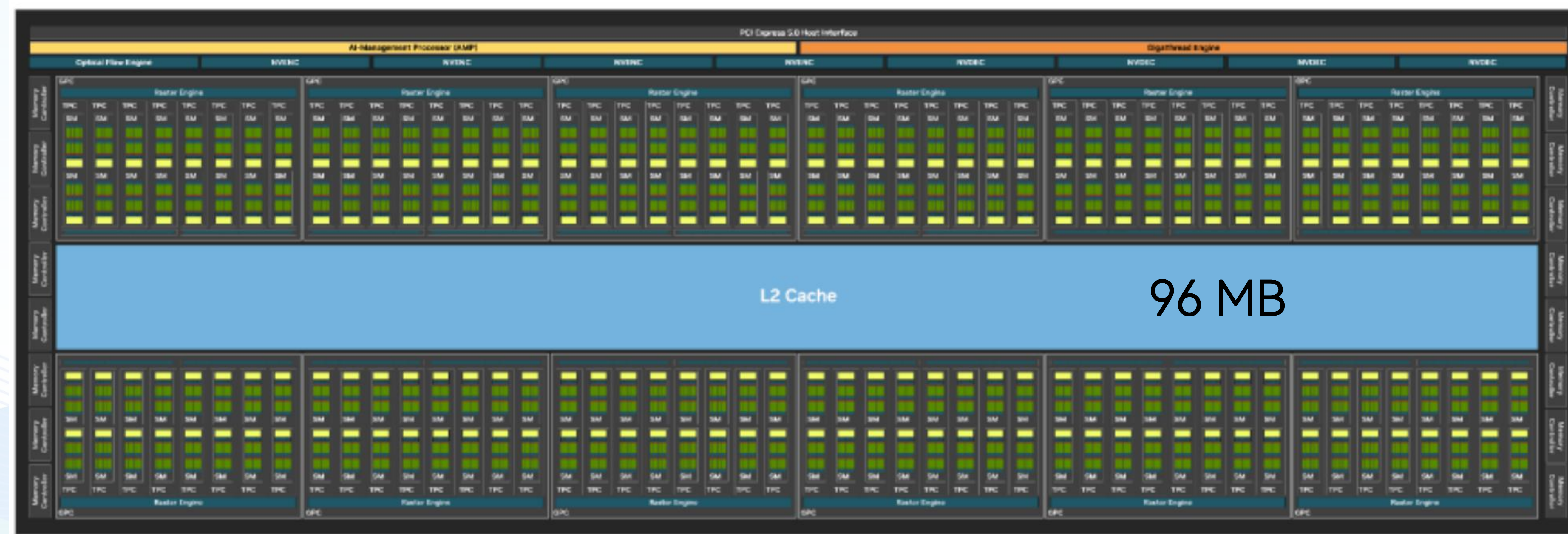


```
def void add(i32* X .aligned(16) , i32* Y .aligned(16) , i32*
Z .aligned(16) , i32 N .multipleof(2) ) {
entry:
    %0 = get_program_id[0] i32;
    %1 = mul i32 %0, 512;
    %3 = make_range[0 : 512] i32<512>;
    %4 = splat i32<512> %1;
    %6 = add i32<512> %4, %3;
    %9 = splat i32<512> N;
    %11 = icmp_slt i1<512> %6, %9;
    %14 = splat i32*<512> X;
    %16 = getelementptr i32*<512> %14, %6;
    %19 = broadcast i1<512> %11;
    %21 = splat i32<512> undef;
    %22 = masked_load i32<512> %16, %19, %21;
    %26 = splat i32*<512> Y;
    %28 = getelementptr i32*<512> %26, %6;
    %31 = broadcast i1<512> %11;
    %33 = splat i32<512> undef;
    %34 = masked_load i32<512> %28, %31, %33;
    %38 = splat i32*<512> Z;
    %40 = getelementptr i32*<512> %38, %6;
    %43 = add i32<512> %22, %34;
    %46 = broadcast i32<512> %43;
    %48 = broadcast i1<512> %11;
    masked_store void %40, %46, %48;
    ret void;}
```

```
visible .entry add(
    .param .u64 add_param_0, .param .u64 add_param_1,
    .param .u64 add_param_2, .param .u32 add_param_3
)
.maxntid 128, 1, 1
{
    .reg .pred    %p<4>;
    .reg .b32    %r<18>;
    .reg .b64    %rd<8>;
    ld.param.u64    %rd4, [add_param_0];
    ld.param.u64    %rd5, [add_param_1];
    mov.u32    %r13, %tid.x;
    ld.param.u32    %r14, [add_param_3];
    shl.b32    %r15, %r13, 2;
    mov.u32    %r16, %ctaid.x;
    mad.lo.s32    %r17, %r16, 512, %r15;
    setp.ge.s32    %p3, %r17, %r14;
    setp.lt.s32    %p1, %r17, %r14;
    mul.wide.s32    %rd7, %r17, 4;
    add.s64    %rd2, %rd4, %rd7;
    @%p1 ld.global.cg.v4.b32 {%r5,%r6,%r7,%r8}, [ %rd2 + 0];
    add.s64    %rd3, %rd5, %rd7;
    @%p1 ld.global.cg.v4.b32 {%r9,%r10,%r11,%r12}, [ %rd3 + 0];
    @%p3 bra    LBB0_2;
    ld.param.u64    %rd6, [add_param_2];
    add.s64    %rd1, %rd6, %rd7;
    add.s32    %r1, %r5, %r9;
    add.s32    %r2, %r6, %r10;
    add.s32    %r3, %r7, %r11;
    add.s32    %r4, %r8, %r12;
    st.global.v4.u32    [%rd1], {%r1, %r2, %r3, %r4};
LBB0_2:
    ret;
}
```

第6讲预览 (1/2): ML Mem. Subsystem

- Memory System
 - Mem Hierarchy & Mem Walls
 - DDR vs HBM; CXL
- Quantization, KV cache





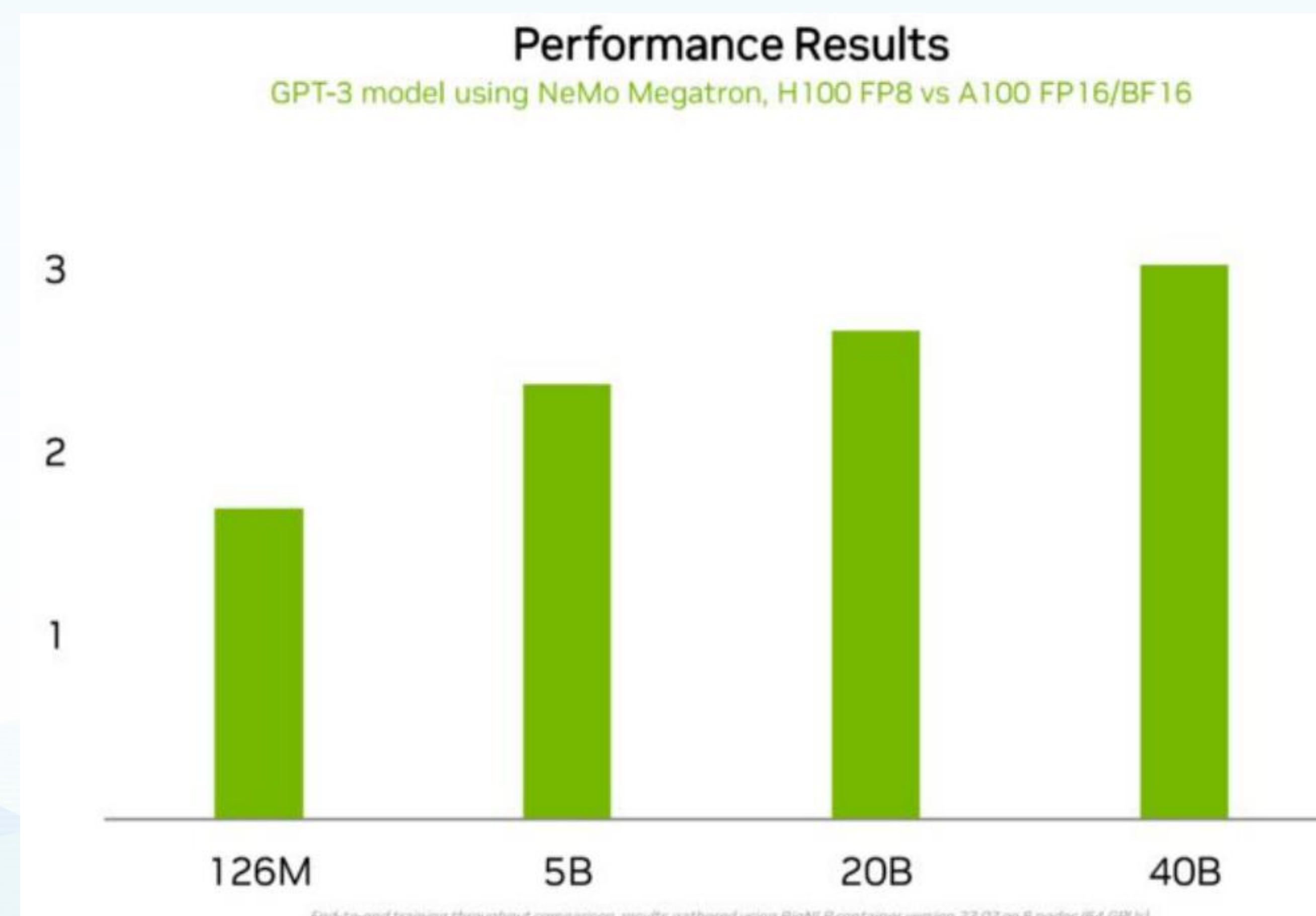
第6讲预览 (2/2): FP8 Training

- A100 → H100
 - TFLOPS 3.2X
 - Bandwidth 1.6X
- Keep ~3X Perf. using FP8

NVIDIA H100 specifications (vs. NVIDIA A100)

Data type	H100-SXM5 (TFLOPS)	A100-SXM4 (TFLOPS)	Difference
TF32	494	156	3.2x
BF16	989	312	3.2x
FP16	989	312	3.2x
FP8	1979	-	6.3x (vs BF16)
Bandwidth (GB/s)	3350	2039	1.6x

Table 1: FLOPS and memory bandwidth comparison between the NVIDIA H100 and NVIDIA A100. While there are 3x-6x more total FLOPS, real-world models may not realize these gains.



Model	System	Precision	#GPUs	Speedup over H100-BF16
126M 5B 20B 40B 175B	H100	FP8	64	1.01
	H100	FP8	64	1.21
	H100	FP8	64	1.28
	H100	FP8	64	1.24
	H100	FP8	64	1.29

Compare with FP8 and BF16 on *H100



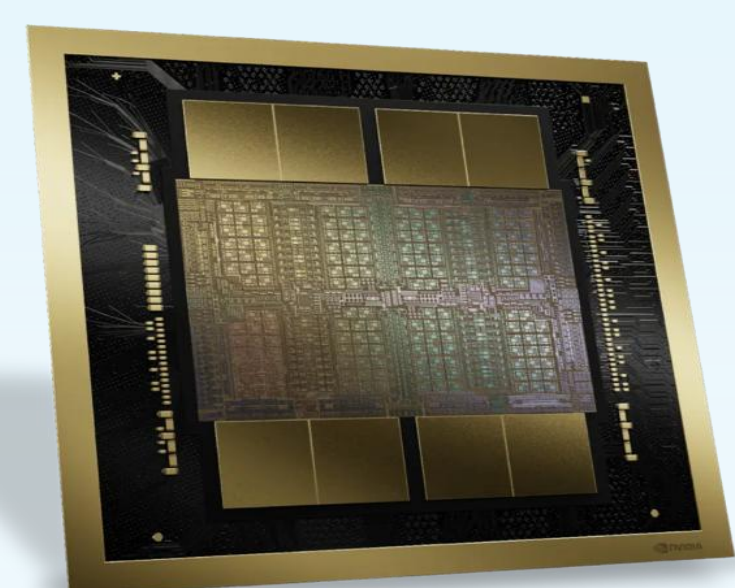
第7讲预览 (1/2): ML Comm. Subsystem

- Intra-node Communication
 - PCIe
 - NVLink / UALink / **CLink**
- Inter-node Communication
 - InfiniBand RDMA
 - Ethernet RoCE RDMA
- Communication Programming Model
 - Nvidia Collective Comm. Library (NCCL)
 - **Huawei Collective Comm. Library (HCCL)**
- AI Compute Cluster System
 - Huawei UB-Mesh

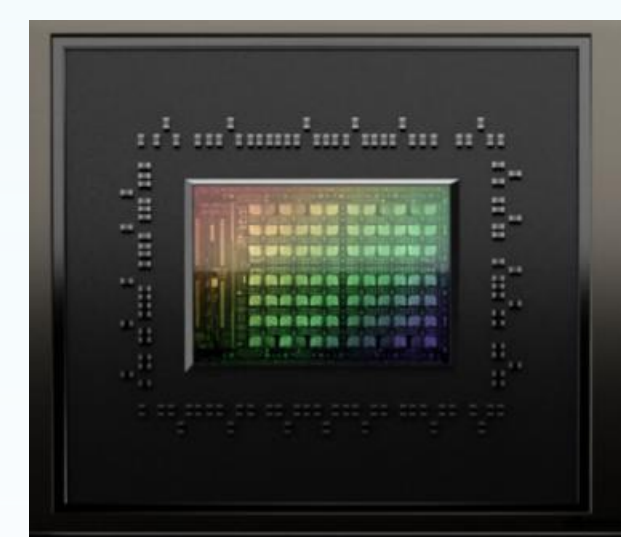


第7讲预览 (2/2): NVIDIA DGX GB200 NVL72

- DGX GB200 NVL72 System (launched in March 2024)

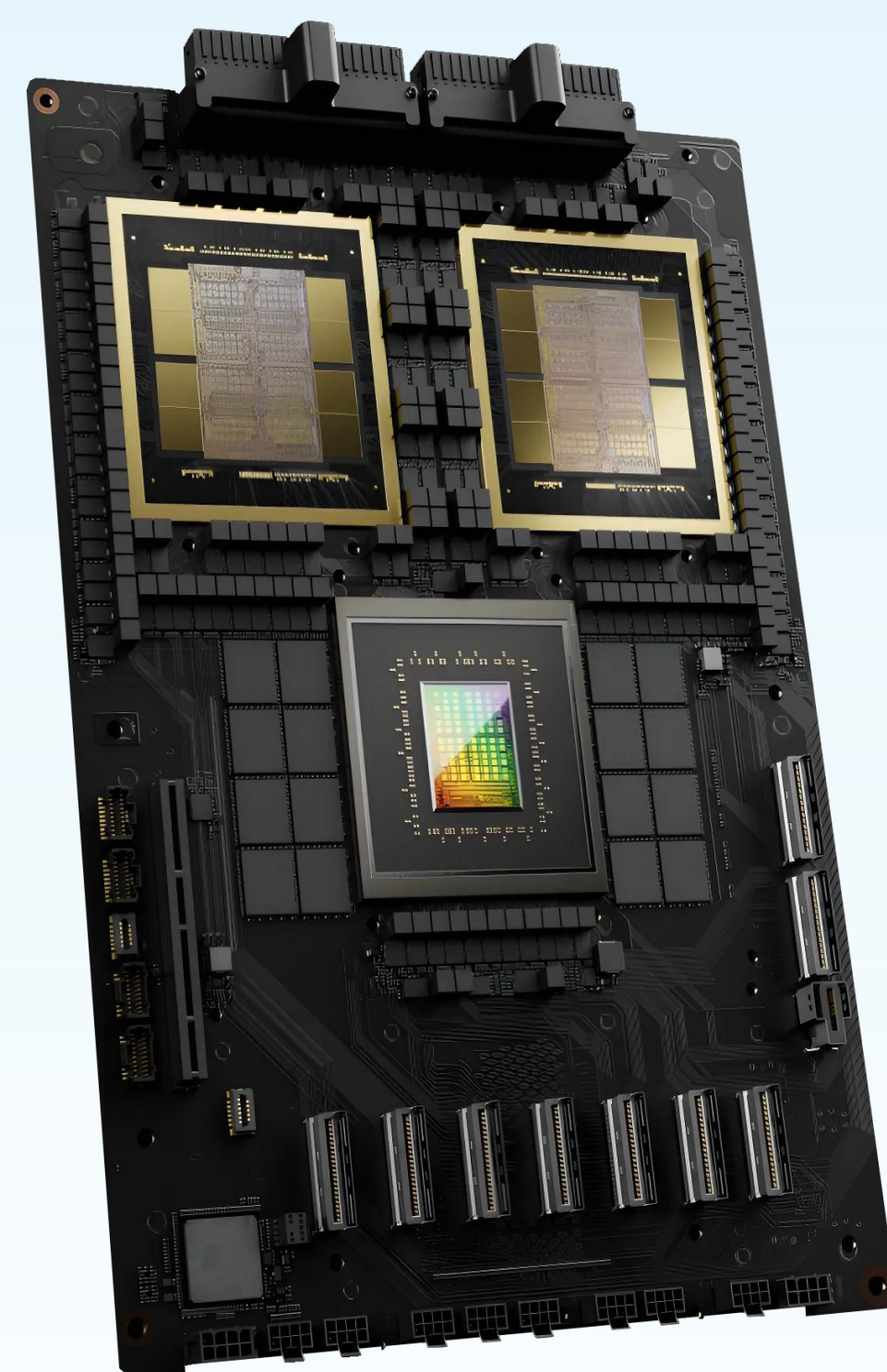
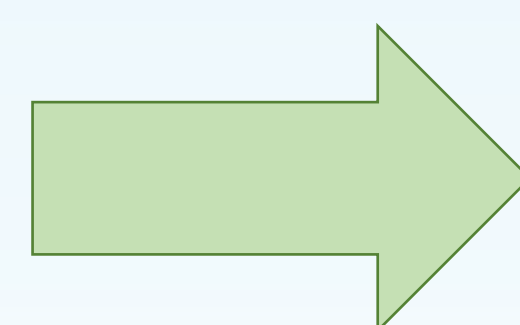


B200 GPU



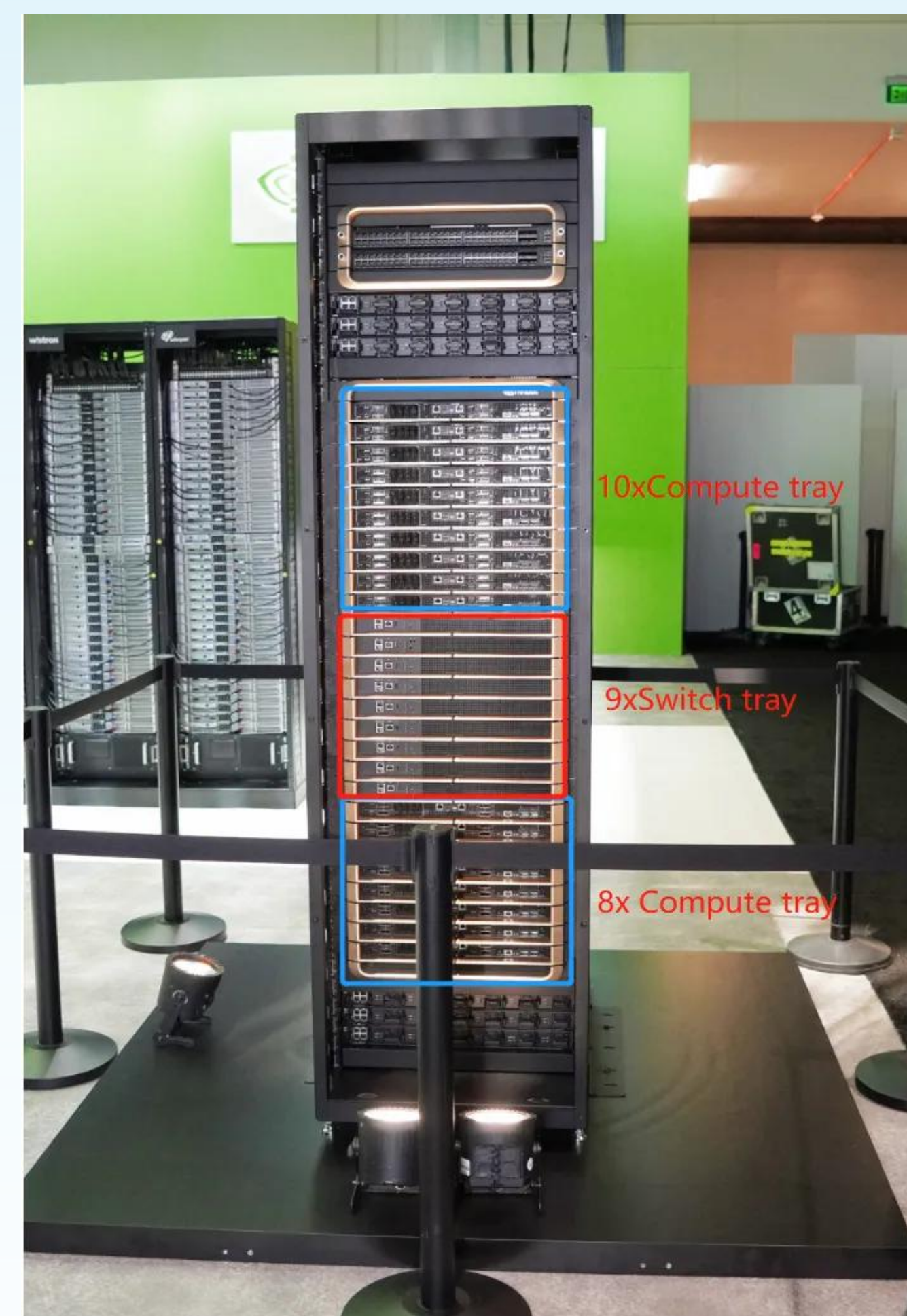
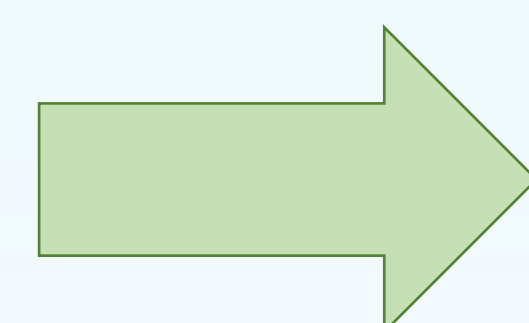
Grace CPU

*NVLink
Chip-to-Chip*



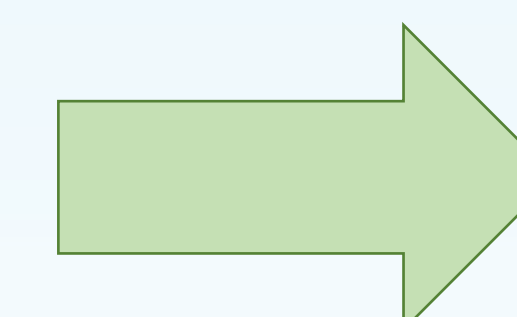
GB200 Grace Blackwell Superchip
2x B200 superchip
1x Grace superchip

*NVLink
&
NVLink Switch*



DGX GB200 NVL72
36x GB200 (18x compute tray)
9x switch tray

*InfiniBand
RDMA*



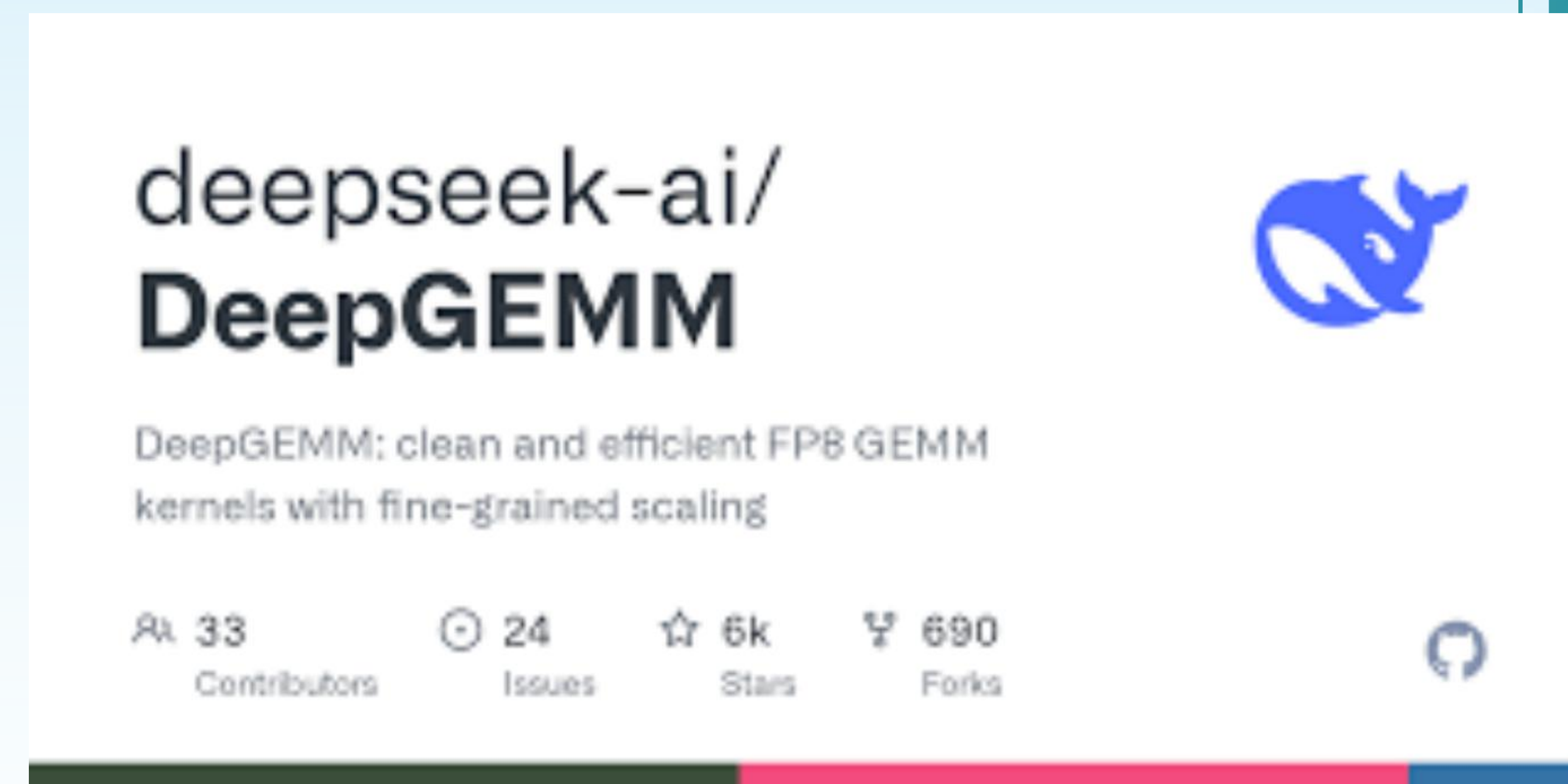
GB200 NVL72 Compute Rack
8x DGX GB200 NVL72

Scale-up Communication

Scale-out Communication

第8讲预览: Co-opt. in Practice

- DeepGEMM: A Co-optimized GEMM Library
- What is it?
 - An open-source library for highly efficient General Matrix Multiplication (GEMM).
 - Uses Just-in-Time (JIT) compilation to generate optimized code at runtime.
- Co-optimization Details:
 - Specifically targets NVIDIA Hopper GPUs.
 - Optimizes for low-precision formats like FP8.
 - Achieves performance superior to general-purpose libraries by leveraging hardware features like the Tensor Memory Accelerator (TMA).



第9讲预览: Miscellaneous Topics and Discussions

- Alop
- AI P & S
- Sustainable AI
- Scaling law





FCES

2026 CCF未来计算机教育峰会

FUTURE COMPUTER EDUCATION SUMMIT

未来

Grading

50%	Mini-projects
25%	Course engagements
25%	Other h/w and written reports

noemotiovn / llama.cpp

CodeIssuesPull requests5AgentsProjectsSecurity and qualityInsights

llama.cpp需求 #1

Closed

noemotiovn opened on Sep 15, 2025Last edited by noemotiovnOwner

ROPE算子: MROPE模式支持

ROPE算子: VISION模式支持

ROPE算子: 支持部分headSize旋转 (n_dims <= src0->ne0)

CONV_TRANSPOSE_1D算子: 支持 (op->src[0]->ne[0] - 1) > 255 场景

OUT_PROD算子: 新算子支持

GATED_LINEAR_ATTN算子: 新算子支持

L2_NORM算子: 新算子支持

CROSS_ENTROPY_LOSS算子: 新算子支持

RWKV_WKV6算子: 新算子支持

RWKV_WKV7算子: 新算子支持

SSM_CONV算子: 新算子支持

SSM_SCAN算子: 新算子支持

重构: acl graph中, 将图命中的校验沉淀至lru cache中

针对某些特殊模型, 支持使用aclnnRopeWithSinCosCache融合算子 (工作量较大)

优化set_device

Assignees

No one assigned

Labels

stale

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Subscribe

flagos-ai / FlagGems

CodeIssues316Pull requests396AgentsDiscussionsActionsProjectsSecurity and qualityInsights

Code Contribution: 【Hard】【Operator Development】
index_reduce #893

Open

0x45f opened on Aug 12, 2025Collaborator

Description 任务介绍

Develop forward function for index_reduce operator.
开发index_reduce算子的前向功能。

Requirements 任务要求

Interface 接口
index_reduce(Tensor self, int dim, Tensor index, Tensor source, str reduce, *, bool include_self=True) -> Tensor

Function reference 功能参考
https://docs.pytorch.org/docs/stable/generated/torch.index_reduce.html

Implementation reference 实现参考
https://github.com/FlagOpen/FlagGems/blob/master/src/flag_gems/ops/index_add.py

The operator should support all optional arguments defined in the interface.
算子应支持接口中定义的所有参数选项。

DDL 提交时间

Assignees

luohj29

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

FlagGems: 高性能通用AI 算子库
第1讲《算子结构与GPU并行编程原理》
第2讲《高性能探索：从 Triton 算子到高性能通用算子 FlagGems》
第3讲《Triton 语言编程与优化实战》
案例讲解：某个 FlagGems 的应用和效果
课后作业：请学生开发一个算子，并在主流模型上应用

FlagTree: 统一 AI 编译器
第1讲《Triton 语言与 Triton 编译器》
第2讲《增强版 Triton编译器：FlagTree 编译器的设计理念》
案例讲解：在 FlagTree 上如何接入一种 AI 芯片架构？
课后作业：替换Wrapper

FlagScale: 面向多元算力的并行训推一体框架
第1讲《大规模分布式训练优化技术》
第2讲《大模型高效推理部署优化技

FlagCX: 统一通信库
第1讲《集合通信库设计与实现》
第2讲《跨芯片高效通信算法》
第3讲《拓扑感知通信优化》

mini-projects from 北京智源研究院开源生态



FCES

2026 CCF未来计算机教育峰会
FUTURE COMPUTER EDUCATION SUMMIT

未来教育:颠覆还是改良?

北京智源“中非人工智能教师人才联合培养计划”



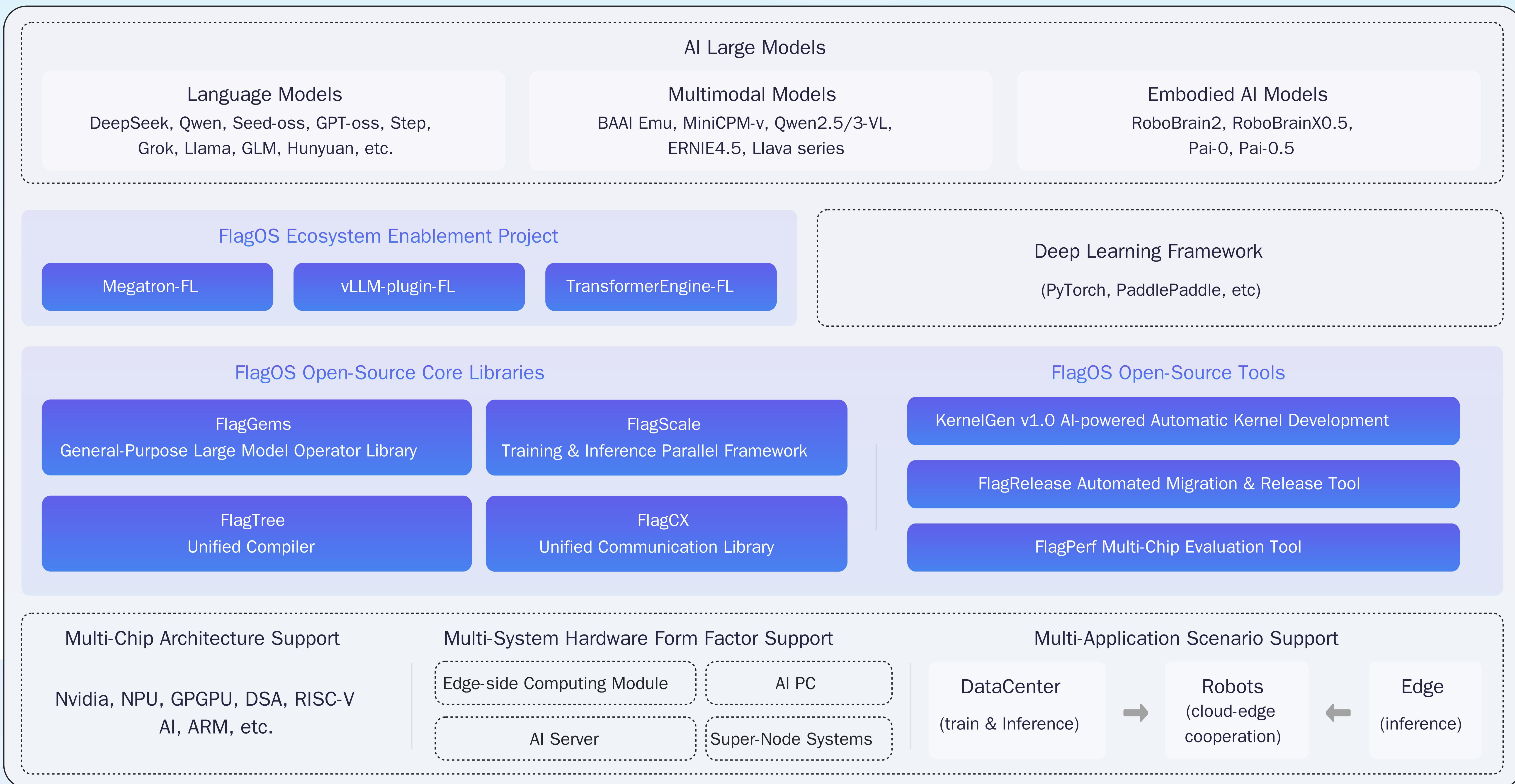


FCES

2026 CCF未来计算机教育峰会
FUTURE COMPUTER EDUCATION SUMMIT

未来教育:颠覆还是改良?

北京智源“中非人工智能教师人才联合培养计划”



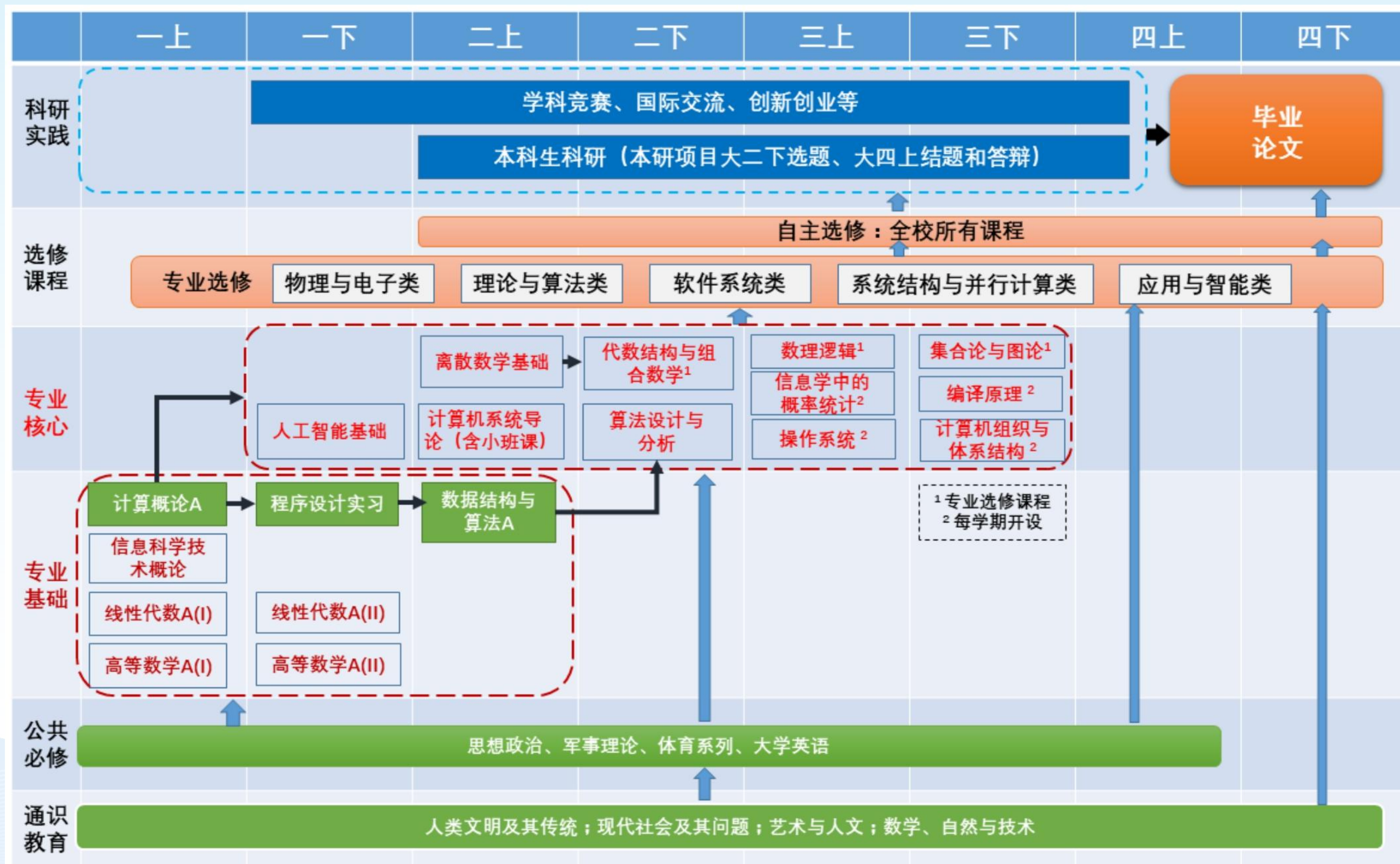


FCES

2026 CCF未来计算机教育峰会
FUTURE COMPUTER EDUCATION SUMMIT

未来教育:颠覆还是改良?

北大“计算机科学与技术专业课程地图”





本科生“并行与分布式计算”课程主要内容

- 模块 1: AI 系统与异构计算基础
 - GPU 或 NPU (“设备”) 内部的并行, 以及“主机”和“设备”之间的并行
- 模块 2: 高性能 AI 算子与算子工程
 - DNN/LLM 算子内部的并行, 以及它们的典型优化方式
- 模块 3: AI 编译器的原理与优化
 - 通过编译实现自动或半自动的并行化
- 模块 4: 分布式并行训练与推理及通信
 - 计算卡和计算节点之间的并行
- 模块 5: 性能评估与AI辅助的内核生成
 - 下一代“自动”并行化



Model Parameter vs GPU Memory Capacity

- NLP Moore's Law
 - 参数量: 7年 **>10,000** 倍

Time	Model	Size
2018	BERT-Large	0.34 B
2019	GPT-2	1.5 B
2020	GPT-3	175 B
2023	GPT-4	≈1.8 T (MoE)
2024	Llama-3.1	405 B
2024	Gemini 1.5 Pro	≈1.5 T
2024	DeepSeek-V3	671 B (MoE)
2025	GPT-4.5	≈12.8 T (MoE)

- GPU Memory Growth
 - 内存容量: 7年 **<10** 倍

Time	GPU	Memory Capacity	Memory Bandwidth
2018	V100	32 GB	900 GB/s
2020	A100	80 GB	2.0 TB/s
2022	A800	80 GB	2.0 TB/s
2023	H100	94 GB	3.9 TB/s
2024	H200	141 GB	4.8 TB/s
2025	B200	192 GB	8 TB/s
2025	B300	288 GB	10 TB/s

Massive-scale, high-bandwidth interconnects are mandatory



FCES

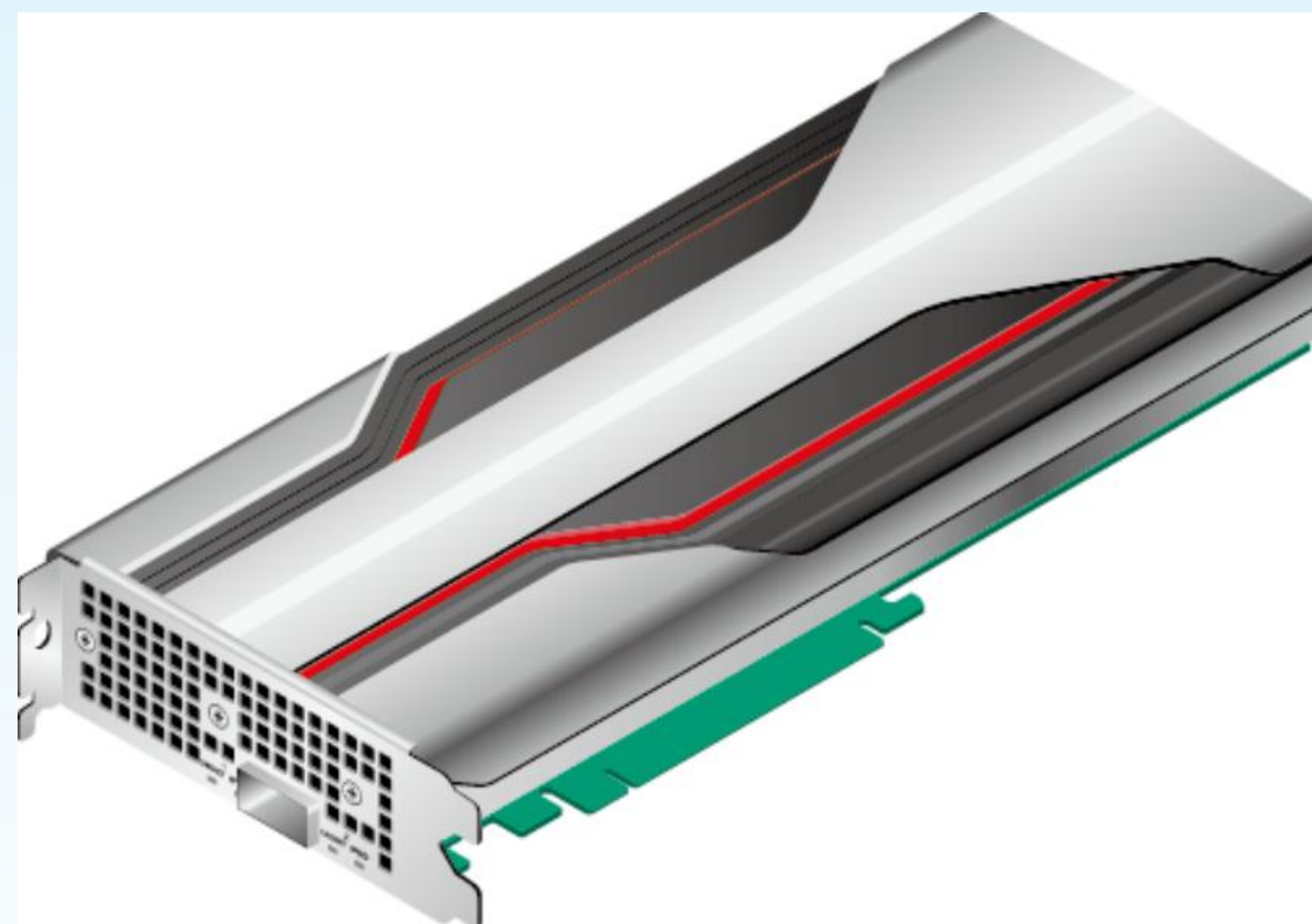
2026 CCF未来计算机教育峰会
FUTURE COMPUTER EDUCATION SUMMIT

未来教育:颠覆还是改良?

模块1：AI 硬件（板级）谱系

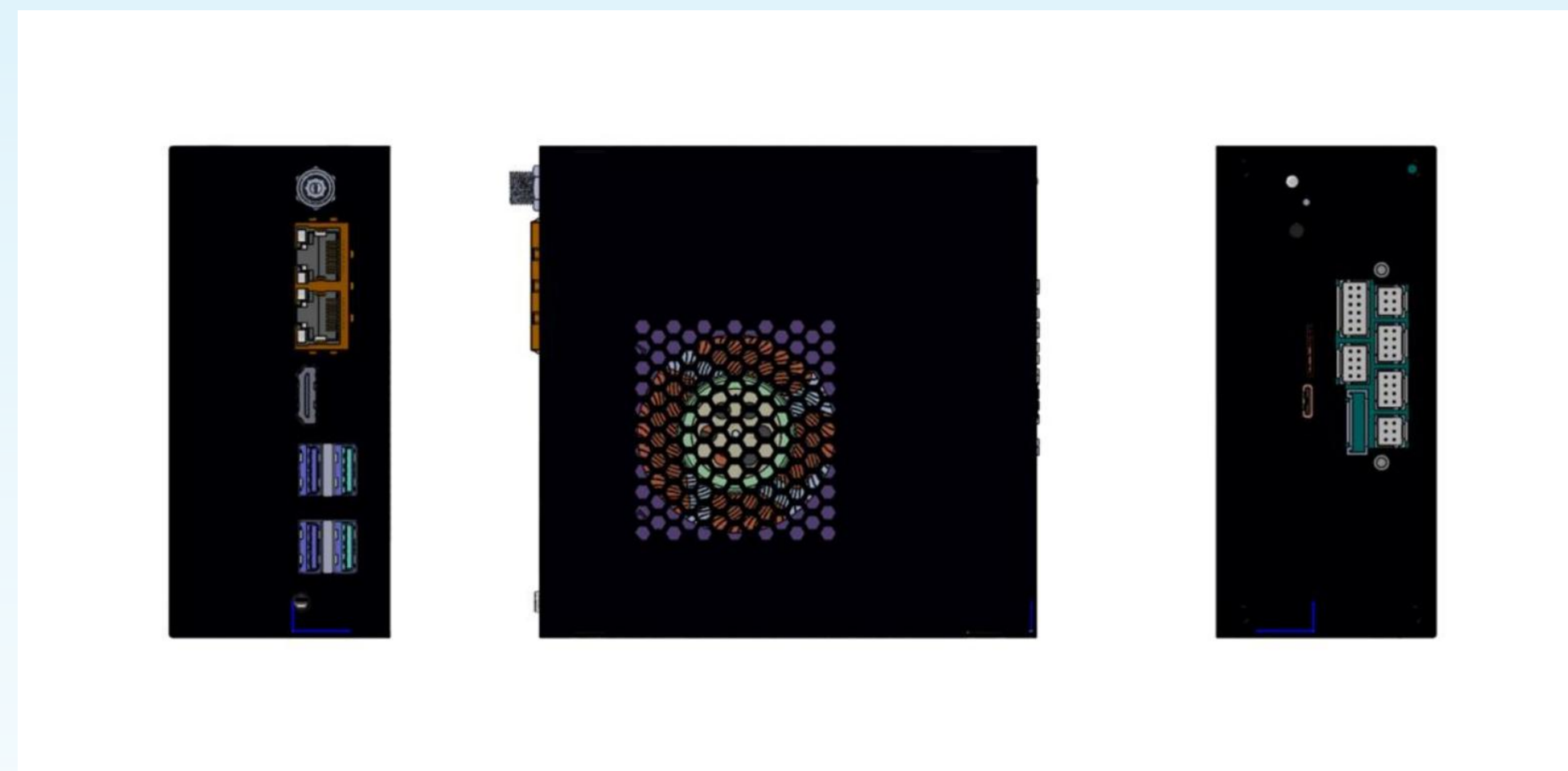
云端机器学习

华为昇腾
寒武纪 MLU370
等等



边缘机器学习

Iluvatar
TY1200



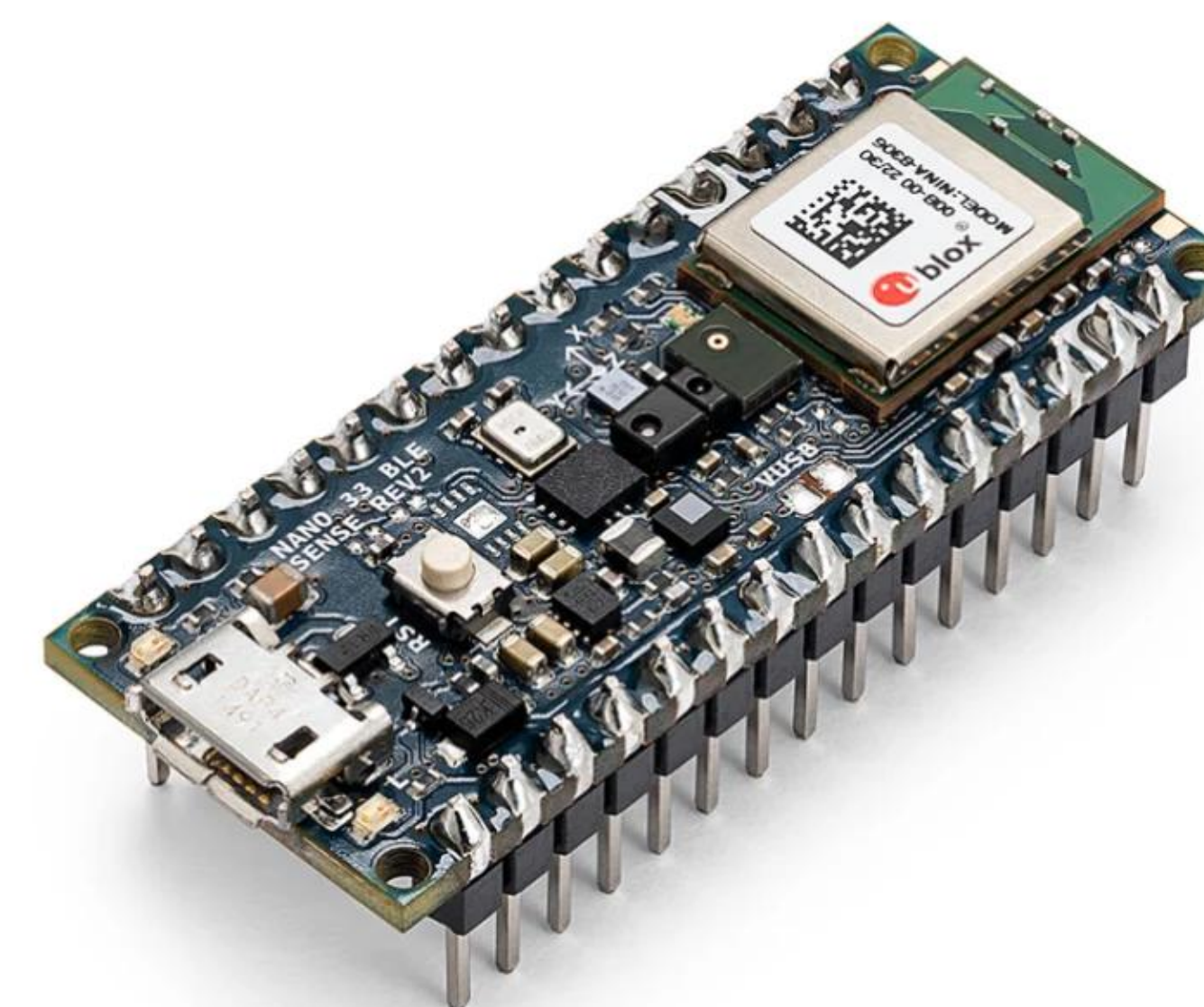
移动端机器学习

Kirin 9030 Pro



微型机器学习

Arduino Nano 33
BLE Sense



模块1：AI 硬件 (芯片级) 示例

Architecture

Core Technology

NV Tensor Core

Mixed-Precision MAC,
Systolic Array

Intel AMX

Tile Matrix Multiplication
(TMUL)

Intel/AMD ACE

AMD AIE

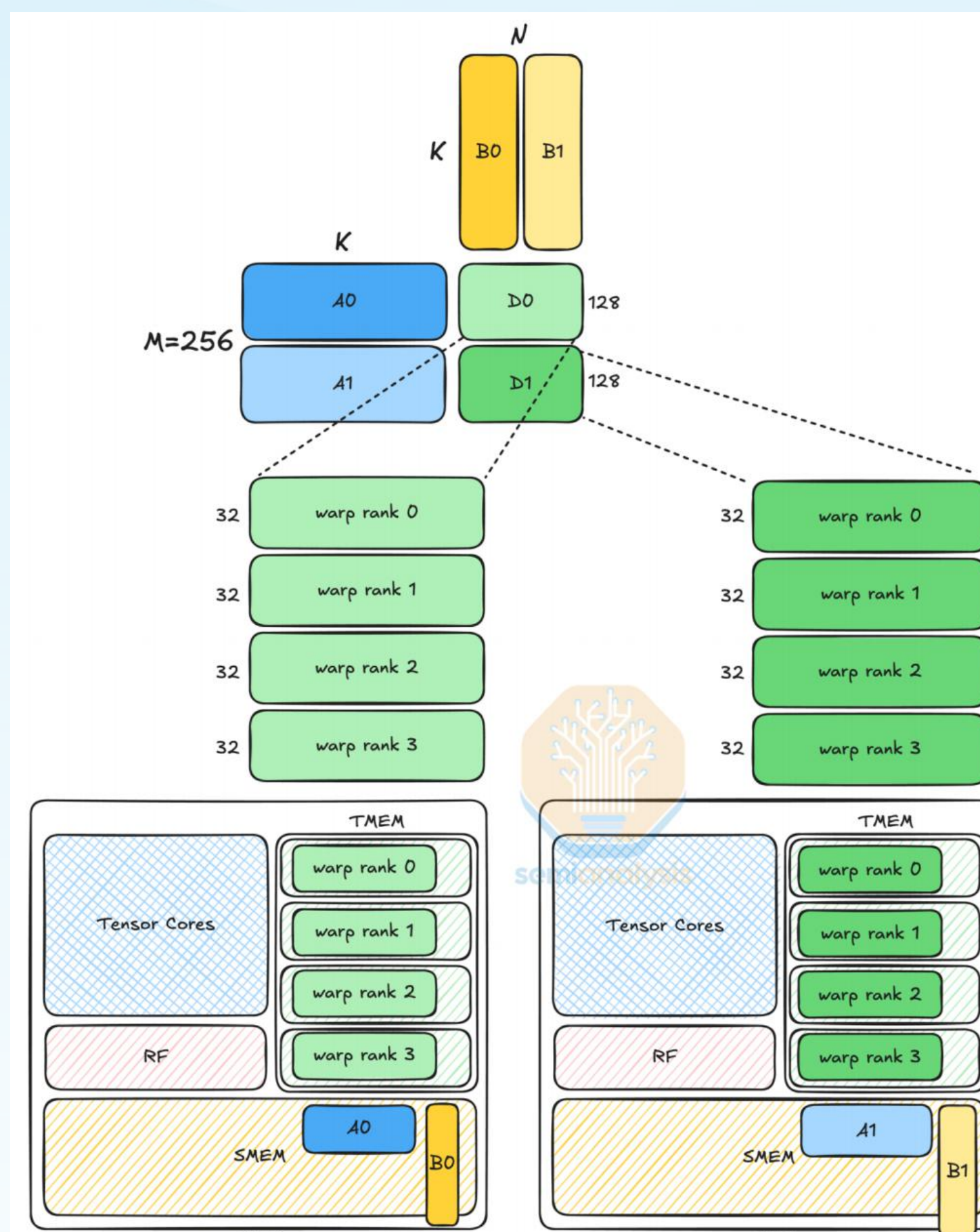
VLIW/SIMD Vector
Processor Array

HUAWEI DaVinci

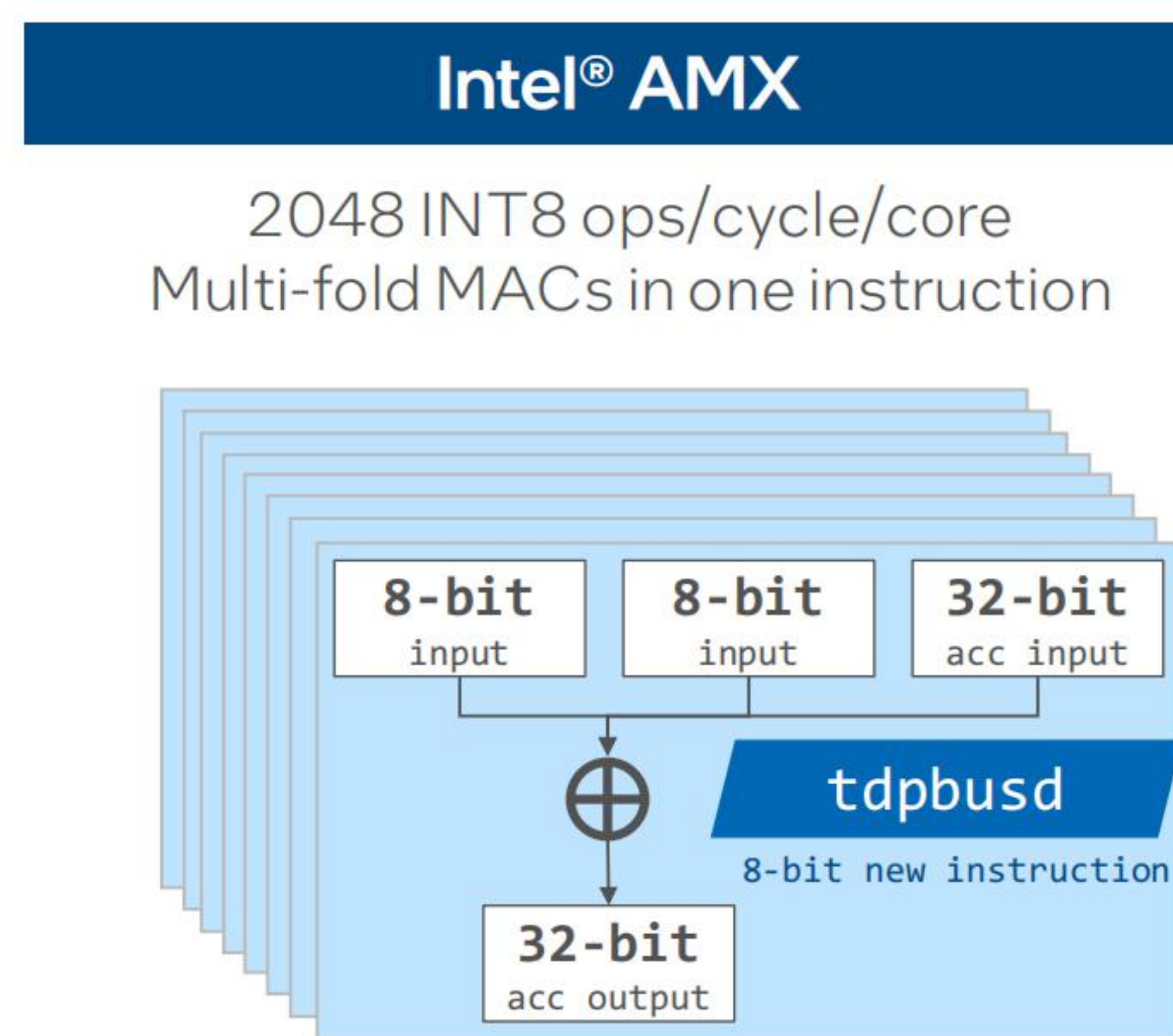
1D Scalar, 2D Vector, 3D
Cube

Tenstorrent

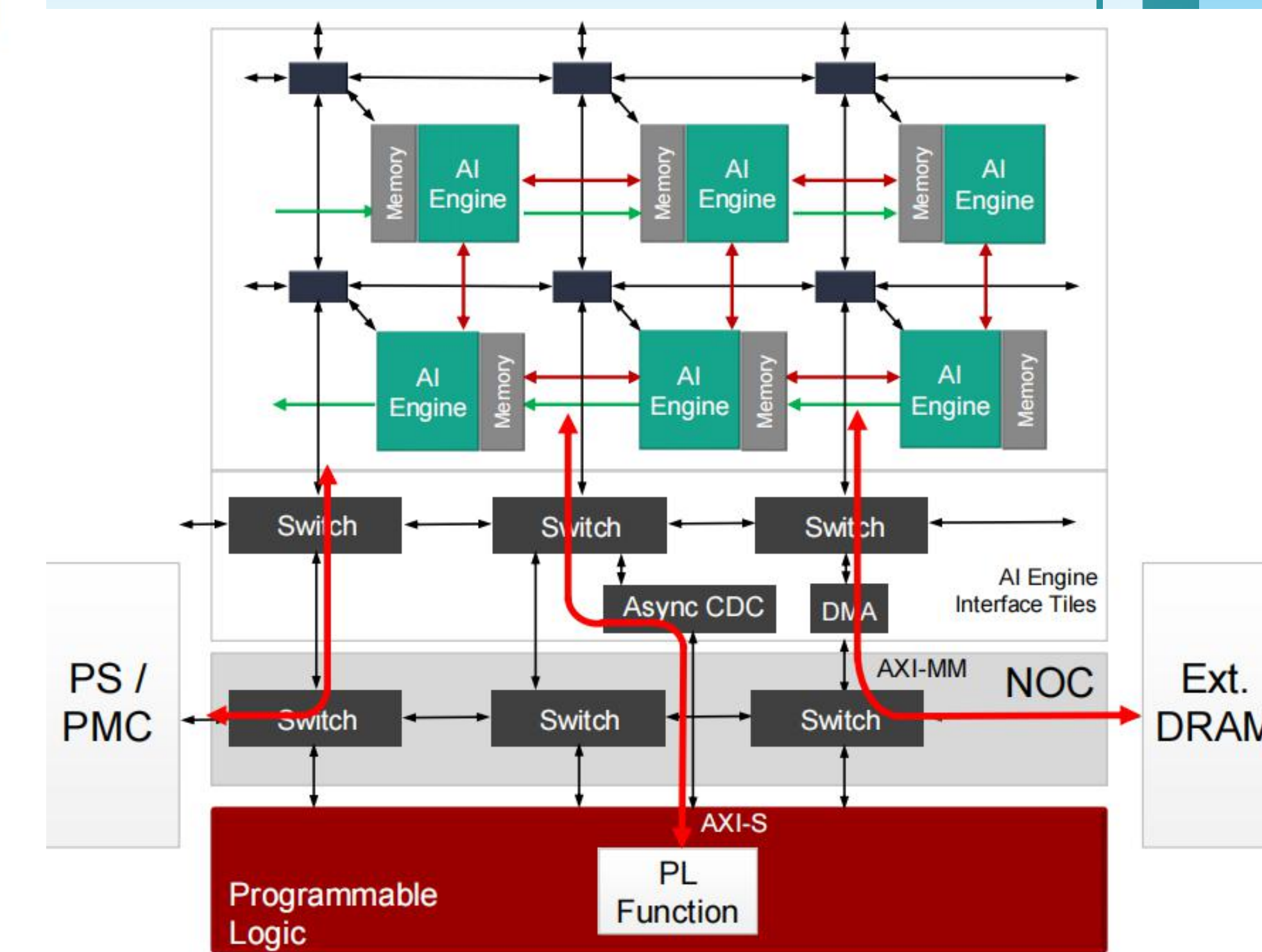
RISC-V, Explicit Data
Movement



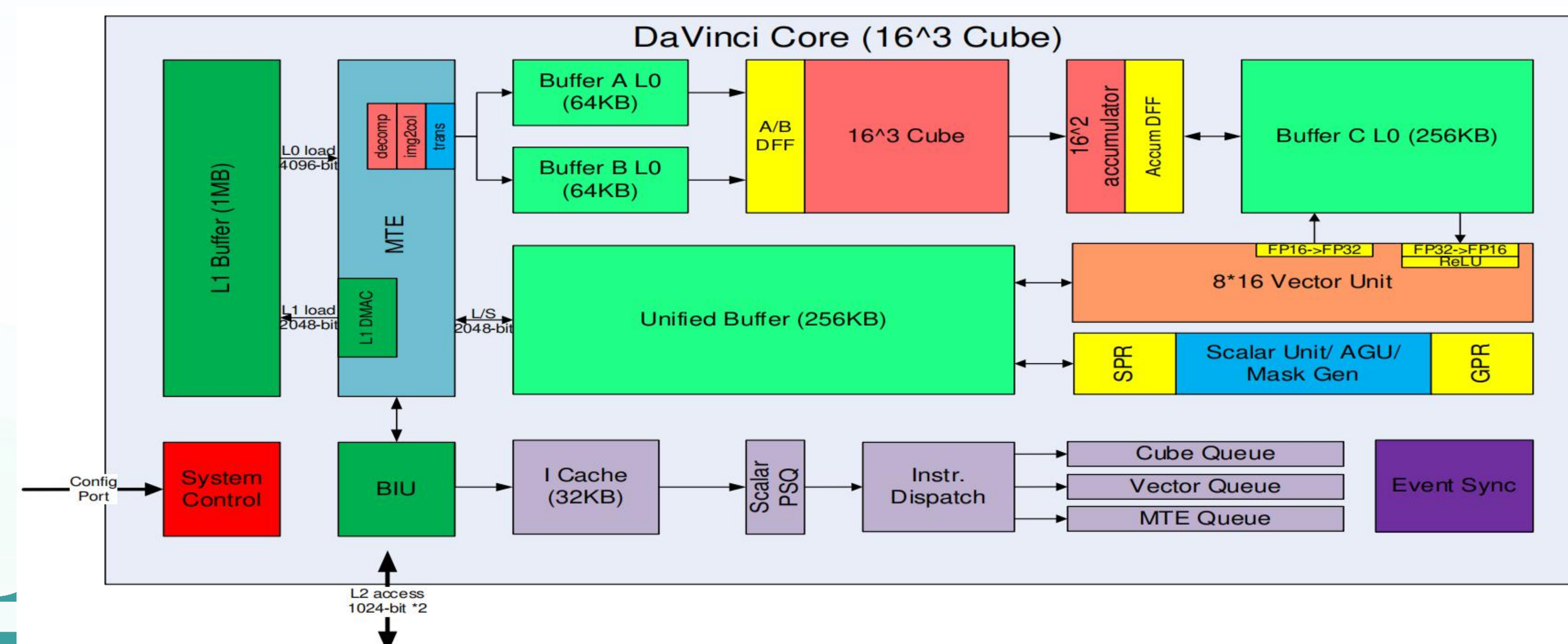
NV Tensor Core



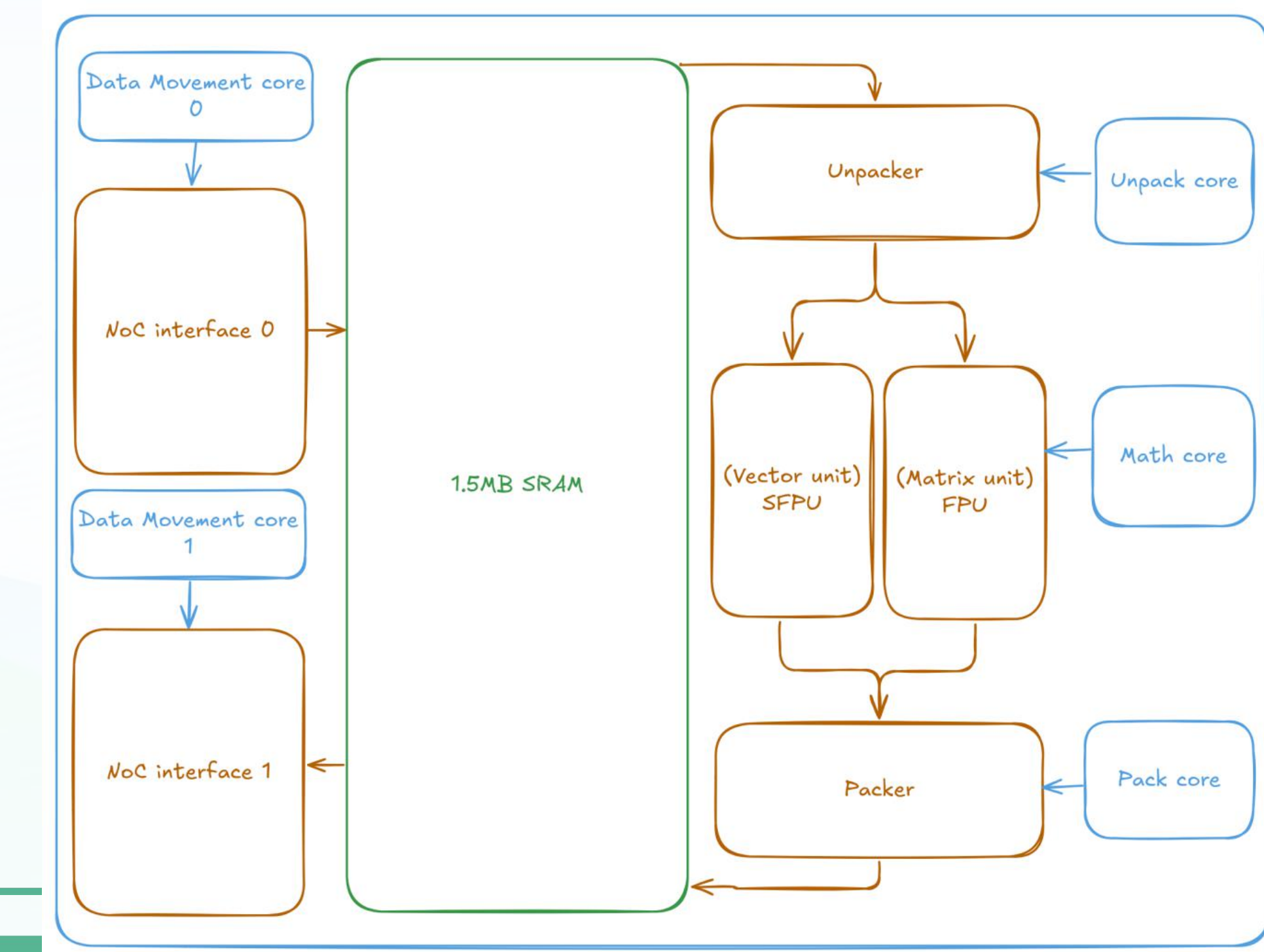
Intel AMX



AMD AIE



HUAWEI DaVinci



Tenstorrent



模块 2：高性能 AI 算子与算子工程

CUDA	异构编程	* Asynchronous Execution * Unified and System Mem
	核编程	* SIMT kernels * Tile kernels
	优化样例	https://github.com/NVIDIA/cuda-samples
CANN	异构编程	CANN 应用开发 (AscendCL)
	核编程	* Ascend C 算子开发 * Ascend C 算子开发接口
	优化样例	https://gitcode.com/cann/cann-samples

- Tensor 核使 CUDA 抽象 break
 - wmma::fragment
 - wmma::load_matrix_sync
 - wmma::mma
 - wmma::store_matrix_sync
- 思维模型转变
 - grid / block / thread
 - grid / block
- 基于 block/tile 编程
 - Triton, cuTile
 - TileLang, PTO





模块 3: AI 编译器的原理与优化

- torch.compile
 - Python → FX Graph → Triton IR
- Triton compilation
 - eDSL → TTIR → TTGIR → LLVM IR
- (TO-DO: TileLang compilation)

```
@jit
def add(X, Y, Z, N):
    pid = program_id(0)
    idx = pid * 512 + arange(512)
    mask = idx < N
    x = load(X + idx, mask=mask)
    y = load(Y + idx, mask=mask)
    store(Z + idx, x + y, mask=mask)
```

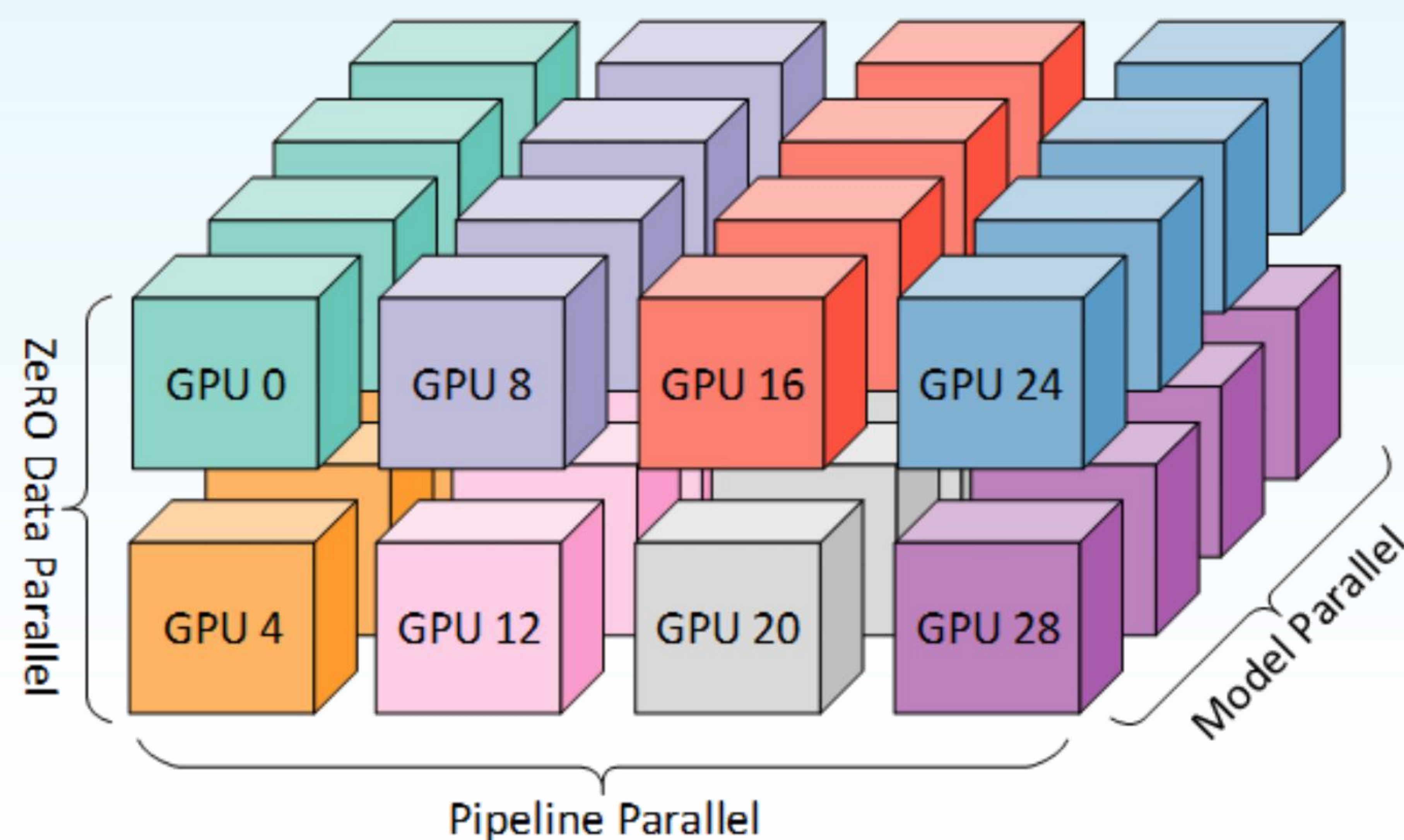
↓

```
def void add(i32* X .aligned(16), i32* Y .aligned(16), i32*
Z .aligned(16), i32 N .multipleof(2)) {
entry:
    %0 = get_program_id[0] i32;
    %1 = mul i32 %0, 512;
    %3 = make_range[0 : 512] i32<512>;
    %4 = splat i32<512> %1;
    %6 = add i32<512> %4, %3;
    %9 = splat i32<512> N;
    %11 = icmp_slt i1<512> %6, %9;
    %14 = splat i32*<512> X;
    %16 = getelementptr i32*<512> %14, %6;
    %19 = broadcast i1<512> %11;
    %21 = splat i32<512> undef;
    %22 = masked_load i32<512> %16, %19, %21;
    %26 = splat i32*<512> Y;
    %28 = getelementptr i32*<512> %26, %6;
    %31 = broadcast i1<512> %11;
    %33 = splat i32<512> undef;
    %34 = masked_load i32<512> %28, %31, %33;
    %38 = splat i32*<512> Z;
    %40 = getelementptr i32*<512> %38, %6;
    %43 = add i32<512> %22, %34;
    %46 = broadcast i32<512> %43;
    %48 = broadcast i1<512> %11;
    masked_store void %40, %46, %48;
    ret void;}
```

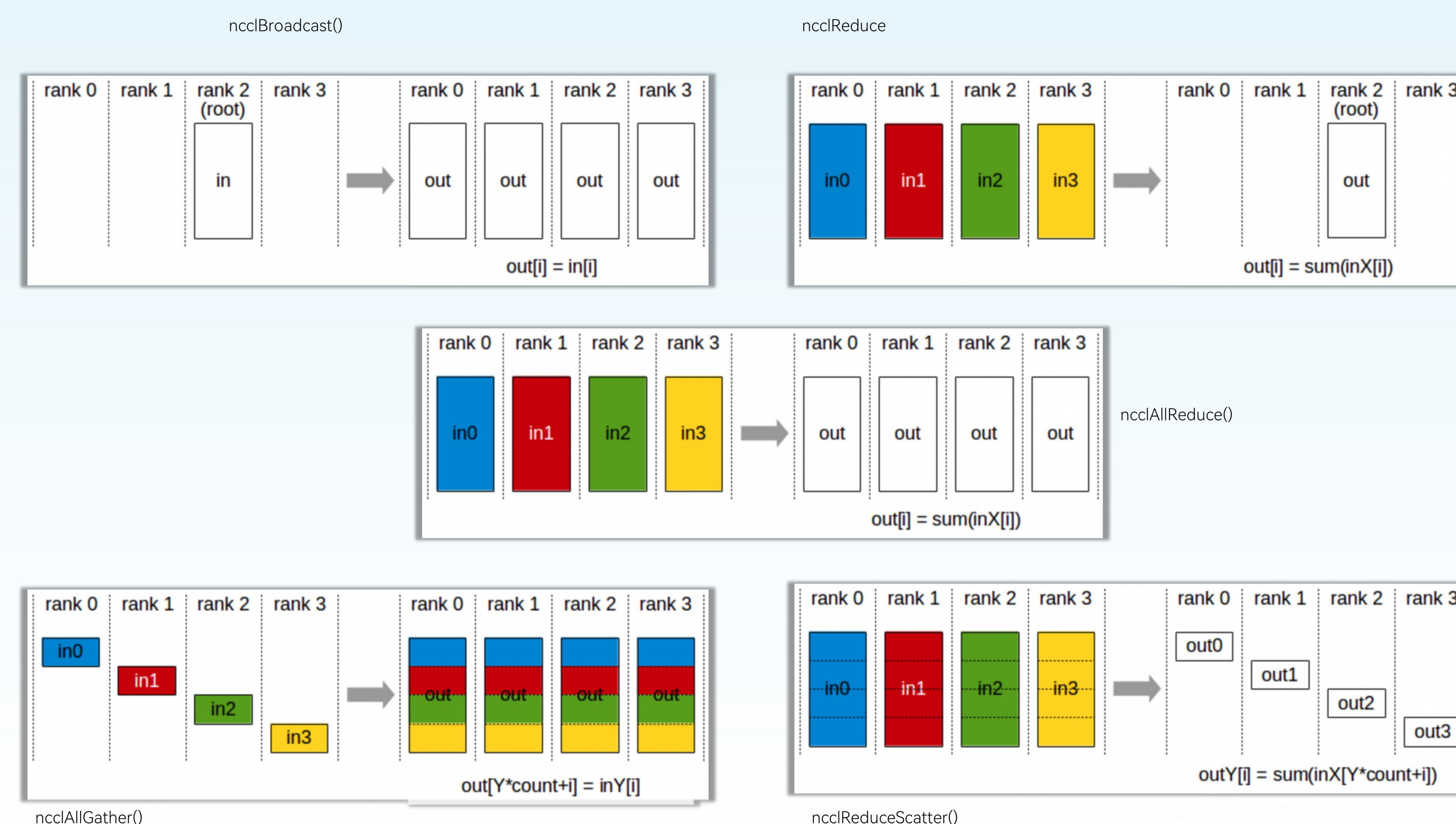
```
visible .entry add(
    .param .u64 add_param_0, .param .u64 add_param_1,
    .param .u64 add_param_2, .param .u32 add_param_3
)
.maxntid 128, 1, 1
{
    .reg .pred    %p<4>;
    .reg .b32    %r<18>;
    .reg .b64    %rd<8>;
    ld.param.u64    %rd4, [add_param_0];
    ld.param.u64    %rd5, [add_param_1];
    mov.u32    %r13, %tid.x;
    ld.param.u32    %r14, [add_param_3];
    shl.b32    %r15, %r13, 2;
    mov.u32    %r16, %ctaid.x;
    mad.lo.s32    %r17, %r16, 512, %r15;
    setp.ge.s32    %p3, %r17, %r14;
    setp.lt.s32    %p1, %r17, %r14;
    mul.wide.s32    %rd7, %r17, 4;
    add.s64    %rd2, %rd4, %rd7;
    @%p1 ld.global.cg.v4.b32 {%r5,%r6,%r7,%r8}, [ %rd2 + 0];
    add.s64    %rd3, %rd5, %rd7;
    @%p1 ld.global.cg.v4.b32 {%r9,%r10,%r11,%r12}, [ %rd3 + 0];
    @%p3 bra    LBB0_2;
    ld.param.u64    %rd6, [add_param_2];
    add.s64    %rd1, %rd6, %rd7;
    add.s32    %r1, %r5, %r9;
    add.s32    %r2, %r6, %r10;
    add.s32    %r3, %r7, %r11;
    add.s32    %r4, %r8, %r12;
    st.global.v4.u32    [%rd1], {%r1, %r2, %r3, %r4};
LBB0_2:
    ret;
}
```

模块 4：分布式并行训推与集合通信

- DP, TP, PP; (CP, EP)



- Collective Communications





模块 5：性能评估与AI辅助的内核生成

gitcode.com/cann/cannbot-skills

cannbot-skills / docs / feature-list.md

Preview

Code

修改追溯

15.0 KB

<> 源码

复制

Ascend C 算子开发

Skills

Skill	功能	使用样例
ascendc-api-best-practices	API 使用最佳实践、参数限制	—
npu-arch	NPU 架构知识、芯片型号映射	—
ascendc-docs-search	API 文档索引 + 在线搜索	—
ascendc-env-check	NPU 设备查询、CANN 环境验证	—
ascendc-tiling-design	Tiling 和 Kernel 设计方法论，按算子类别分类	—
ascendc-precision-debug	精度调试，症状-原因速查、常见陷阱	—
ascendc-runtime-	运行时错误调试，错误码解析	—

github.com/flagos-ai/KernelGen

子技能	用途
kernelgen-generate	为任意 Python/Triton 仓库生成 GPU 算子
kernelgen-generate-for-flaggems	FlagGems 专用 (pointwise_dynamic、_FULL_CONFIG、分类测试)
kernelgen-generate-for-vllm	vLLM 专用 (SPDX 头、init_logger、@triton.autotune、自定义算子注册)
kernelgen-optimize	通用 Triton 内核优化，通过 MCP 迭代循环进行多轮优化
kernelgen-optimize-for-flaggems	FlagGems 专用优化 (3 种模式：内置/外部/实验性算子)
kernelgen-optimize-for-vllm	vLLM 专用优化，含 CustomOp 注册和集成
kernelgen-specialize	平台特化，将 Triton 算子迁移到目标平台 (如 GPU → 昇腾 NPU)
kernelgen-specialize-for-flaggems	平台特化 + FlagGems 框架集成，支持 4 种集成模式



PKU “并行与分布式计算导论” lab1

第一部分：平台基础练习（80 分）

步骤一：注册并加入小组

1. 访问 <https://cannjudge.cn/>，登录后将昵称修改为真实姓名（用于成绩核对）。
2. 在“小组空间”加入 PKU算子 小组。

步骤二：完成算子题目

在基础练习题单中，任选 4 道算子题目，编写算子实现并通过平台测试。

- 每道题目占本次作业总分的 20 分，共 80 分。
- 提交后请截图保存通过记录，在本次大作业的报告展示。
- 注：cann算子创建规范.zip压缩包在第二部分自行设计算子中使用，平台基础练习直接在平台上通过测试即可。

第二部分：自行设计算子（20 分）

请参考压缩包中提供的示例文件（TensorSum.json、TensorSum.py、说明文档），自行设计一个新的算子，并提交对应的 JSON 文件和 Python 文件。

参考文件说明

压缩包中包含以下文件：

文件名	说明
TensorSum.json	算子描述文件示例，定义算子输入输出、数据类型及测试点
TensorSum.py	算子标准实现示例（Python）
TensorSum算子案例.txt	对上述示例的文字说明

- <https://tensoroj.cn/cann/>
- （80分）平台基础练习
 - 给定范围内，完成4道算子题目
- （20分）算子设计练习
 - 设计新的算子评测题目



TensorOJ平台

- 域名为 <https://tensoroj.cn/cann>
- 类似于openjudge，平台支持以小组形式管理班级，小组内开设比赛和题单
- 平台同时设有开放题库和开放比赛
- 每道题目提交时需要编辑多个文件，提交后反馈每个测试点的状态
 - Pass 通过
 - Fail 运行时错误
 - Wrong Answer 输出错误或精度不够
 - Time Limit Exceeded 超时，如算法很慢或陷入死循环
- 提交的代码根据运行时间打分，设有排行榜

Addcmul

芯片类型: 910B

自定义算子工程

通过率: 17% 通过人数: 2 尝试人数: 12

开始答题

[题目内容](#)[提交排名](#)

题目内容

描述

一、赛题背景

Addcmul是深度学习框架中的核心计算算子，实现了"加法与乘法组合"的高效计算，广泛应用于优化器参数更新、梯度计算、特征融合等深度学习场景。

本题要求基于PyTorch原生torch.addcmul算子的核心业务逻辑，采用Ascend C编程语言进行算子原生开发，在昇腾NPU硬件上实现一款高性能、高精度的Addcmul算子。

二、算子功能描述

实现的Addcmul算子需对输入张量执行"加法与乘法组合"计算。算子计算公式为： $y = \text{input_data} + x1 \times x2 \times \text{value}$ 。

算子支持多种数据类型，输出类型与输入类型保持一致，需兼容非32整倍数的维度非对齐场景，并支持NumPy风格的广播语义。

三、核心定义与约束

3.1 参考算子

PyTorch原生算子：torch.addcmul

我的提交记录

暂无提交记录

题目内容

芯片类型: 910B

自定义算子工程

描述

一、赛题背景

Addcmul是深度学习框架中的核心计算算子，实现了"加法与乘法组合"的高效计算，广泛应用于优化器参数更新、梯度计算、特征融合等深度学习场景。

本题要求基于PyTorch原生torch.addcmul算子的核心业务逻辑，采用Ascend C编程语言进行算子原生开发，在昇腾NPU硬件上实现一款高性能、高精度的Addcmul算子。

二、算子功能描述

实现的Addcmul算子需对输入张量执行"加法与乘法组合"计算。算子计算公式为： $y = \text{input_data} + x1 \times x2 \times \text{value}$ 。

算子支持多种数据类型，输出类型与输入类型保持一致，需兼容非32整倍数的维度非对齐场景，并支持NumPy风格的广播语义。

三、核心定义与约束

3.1 参考算子

PyTorch原生算子：torch.addcmul

工程目录

按目录展开查看工程文件

 CMakeLists.txt op_host addcmul.cpp CMakeLists.txt op_kernel addcmul_tiling.h addcmul.cpp CMakeLists.txt tiling_key_addcmul.h

CMakeLists.txt ×

op_host/addcmul.cpp ×

op_kernel/addcmul.cpp ×

addcmul

🔄 📄

```
6
7  template <class DT_INPUT_DATA>
8  class KernelAddcmul {
9  public:
10     __aicore__ inline KernelAddcmul() {}
11     __aicore__ inline void Init(GM_ADDR input_data, GM_ADDR x1, GM_ADDR
12 x2, GM_ADDR value, GM_ADDR y, uint32_t length) {
13
14     }
15     __aicore__ inline void Process() {
16
17     }
18 private:
19
20 };
21
22 template <typename DT_INPUT_DATA>
23 __global__ __aicore__ void addcmul(GM_ADDR input_data, GM_ADDR x1,
24 GM_ADDR x2, GM_ADDR value, GM_ADDR y, GM_ADDR workspace, GM_ADDR
25 tiling) {
26     REGISTER_TILING_DEFAULT(AddcmulTilingData);
27     GET_TILING_DATA_WITH_STRUCT(AddcmulTilingData, tiling_data,
28 tiling);
29     KernelAddcmul<DT_INPUT_DATA> op;
30     op.Init(input_data, x1, x2, value, y, tiling_data.length);
31     op.Process();
32 }
```



FCES

2026 CCF未来计算机教育峰会
FUTURE COMPUTER EDUCATION SUMMIT

未来教育:颠覆还是改良?

开放题库 / Addcmul

Addcmul

芯片类型: 910B 自定义算子工程

通过率: 17% 通过人数: 2 尝试人数: 12

开始答题

题目内容 提交排名

提交排名

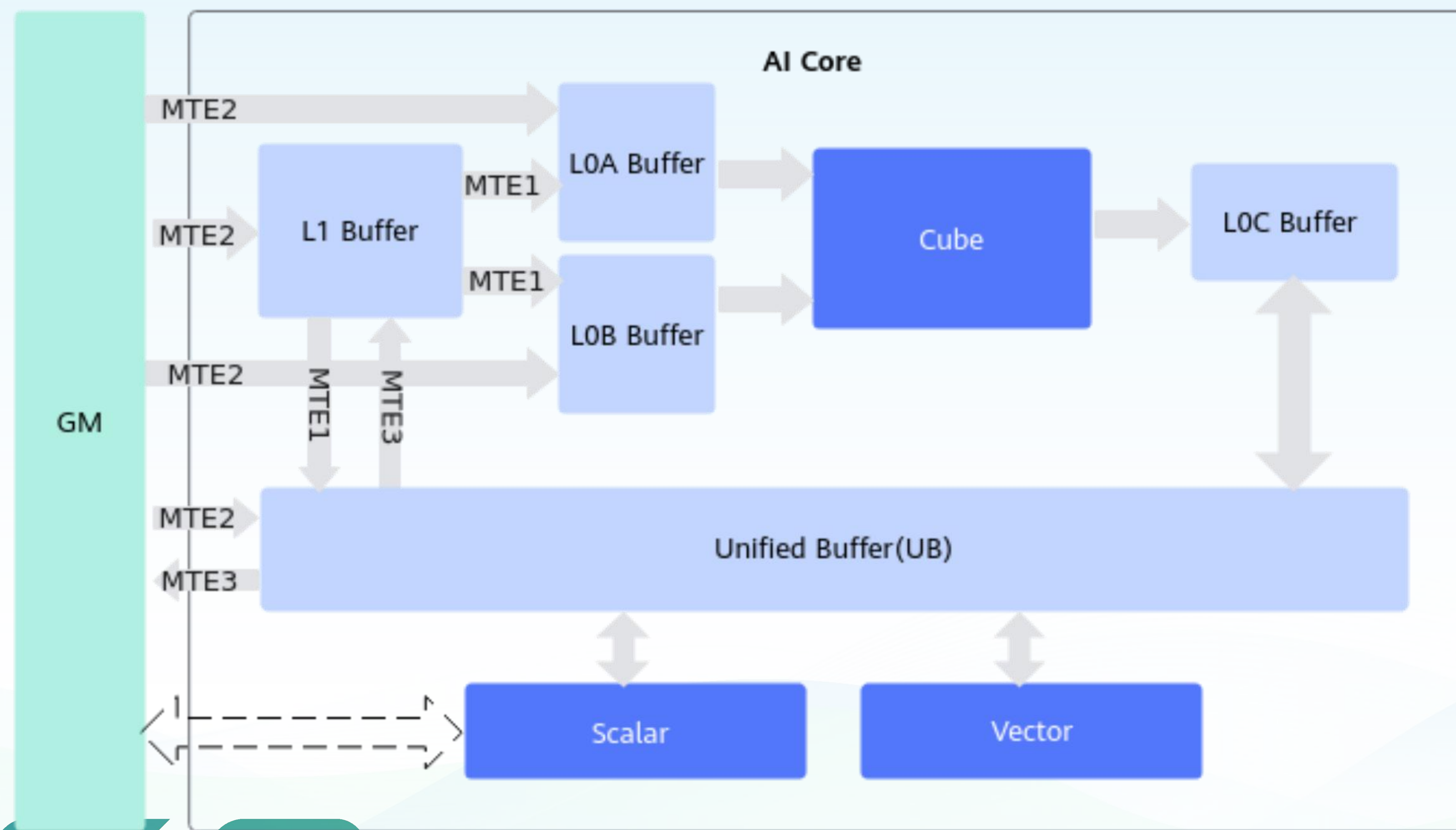
排名	提交人	提交时间	分数	测试点 1	测试点 2	测试点 3	测试点 4	测试点 5
1	fuyangchenghu	2026/04/25 18:41:56	95.66	5μs	6μs	5μs	16μs	21μs
2	Huang51966	2026/04/07 12:37:33	88.47	5μs	6μs	6μs	16μs	22μs
3	mlewis	2026/03/30 17:57:34	0.00	-	-	-	-	-
4	desperateW	2026/03/31 10:30:43	0.00	-	-	-	-	-

DaVinci 耦合架构

- 每张计算卡包含多个核心，每个核心具有自己的内存和计算单元
 - 不同型号的计算卡包含的核心数量、核心类型不一样

• (1) 耦合架构

- 所有核心的结构相同
- 每个核心里同时包含了
 - 标量运算单元 (Scalar)
 - 向量运算单元 (Vector)
 - 矩阵运算单元 (Cube)
 - 多个内存单元
 - 多个内存搬运单元



图示说明

MTE1/MTE2/MTE3:搬运单元

存储单元

计算单元

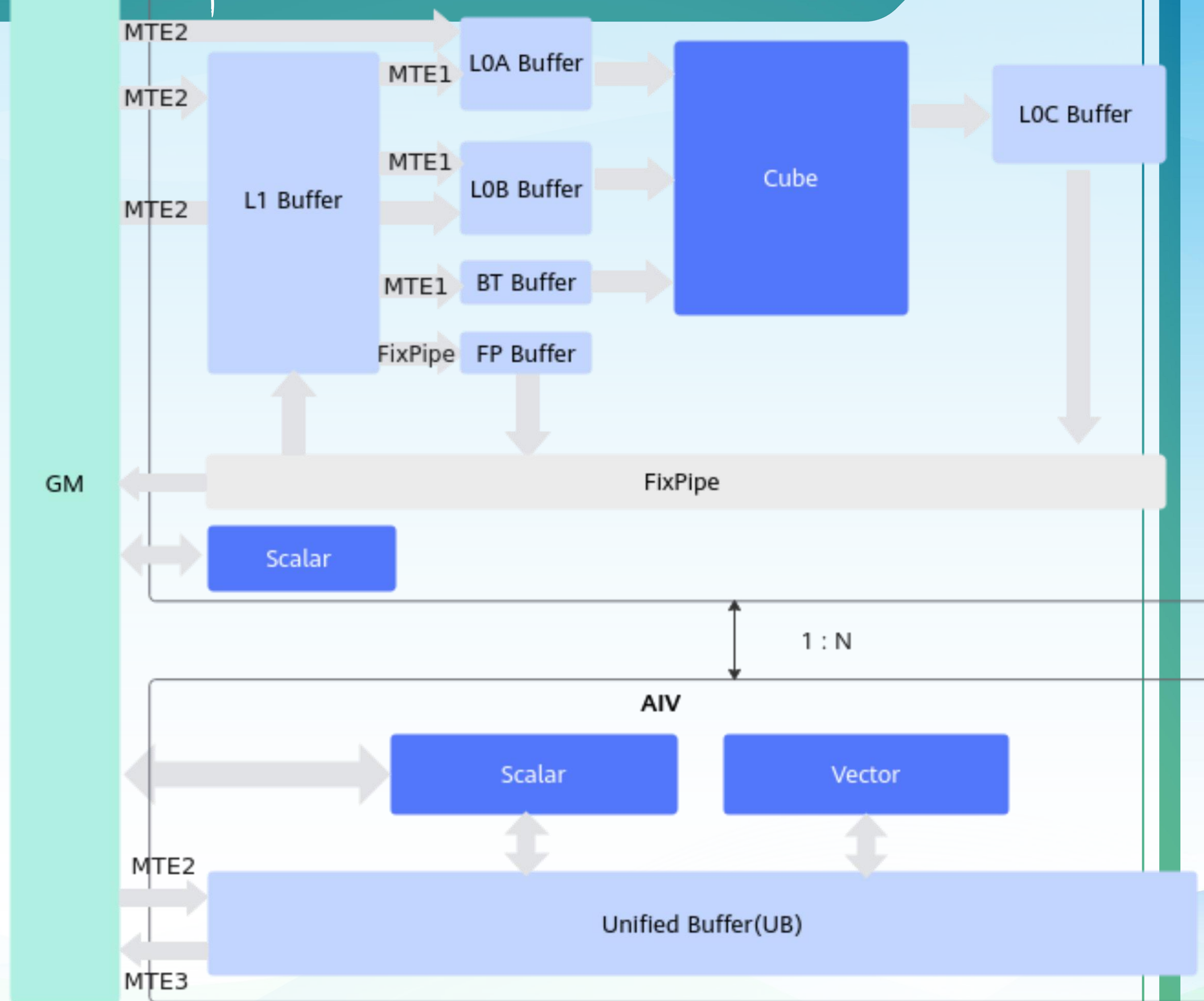
DaVinci 分离架构

• (2) 分离架构

- 包含两种核心
- Vector核
 - Scalar和Vector计算单元
- Cube核
 - Scalar和Cube计算单元

• 昇腾910B计算卡:

• 分离架构, 20AIC+40AIV



图示说明

MTE1/MTE2/MTE3/FixPipe:搬运单元 存储单元 计算单元

PKU “并行与分布式计算导论” lab2

并行与分布式导论 — Lab2: 昇腾 NPU 微基准测试设计

截止日期: 2026年6月20日 23:59

提交方式: 将 lab2_学号_姓名.zip 提交至教学网

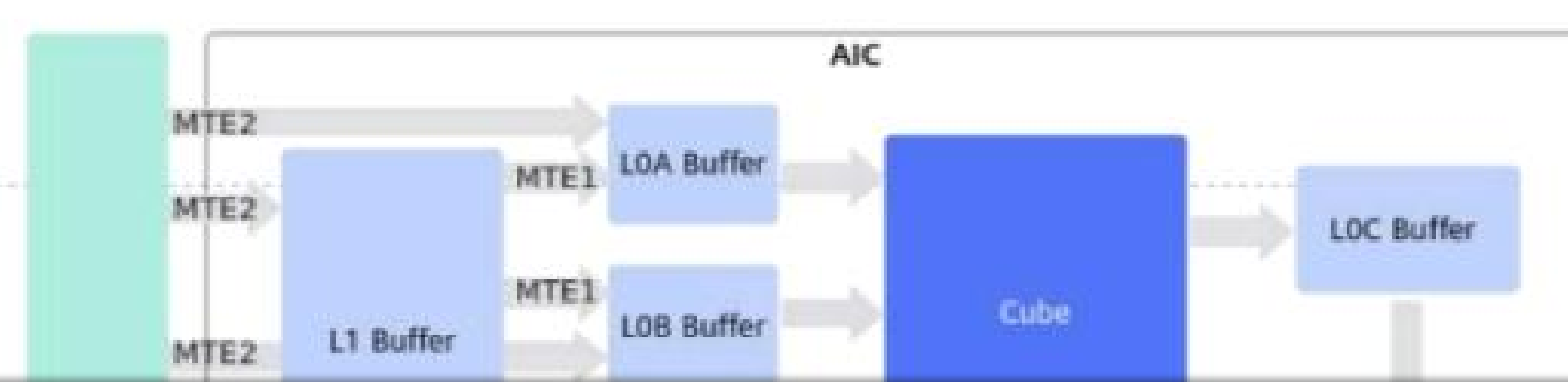
1. 背景

昇腾 (Ascend) 是华为自研的 AI 处理器, 基于 DaVinci 架构设计, 其微架构概览见教学网: TensorOJ平台与芯片算子开发.pdf 第8页。

芯片架构

(2) 分离架构

包含两种核心



8.在AI工具日益普及的背景下，您希望本课程在教学或考核方式上做出哪些调整？(评估人数:3)

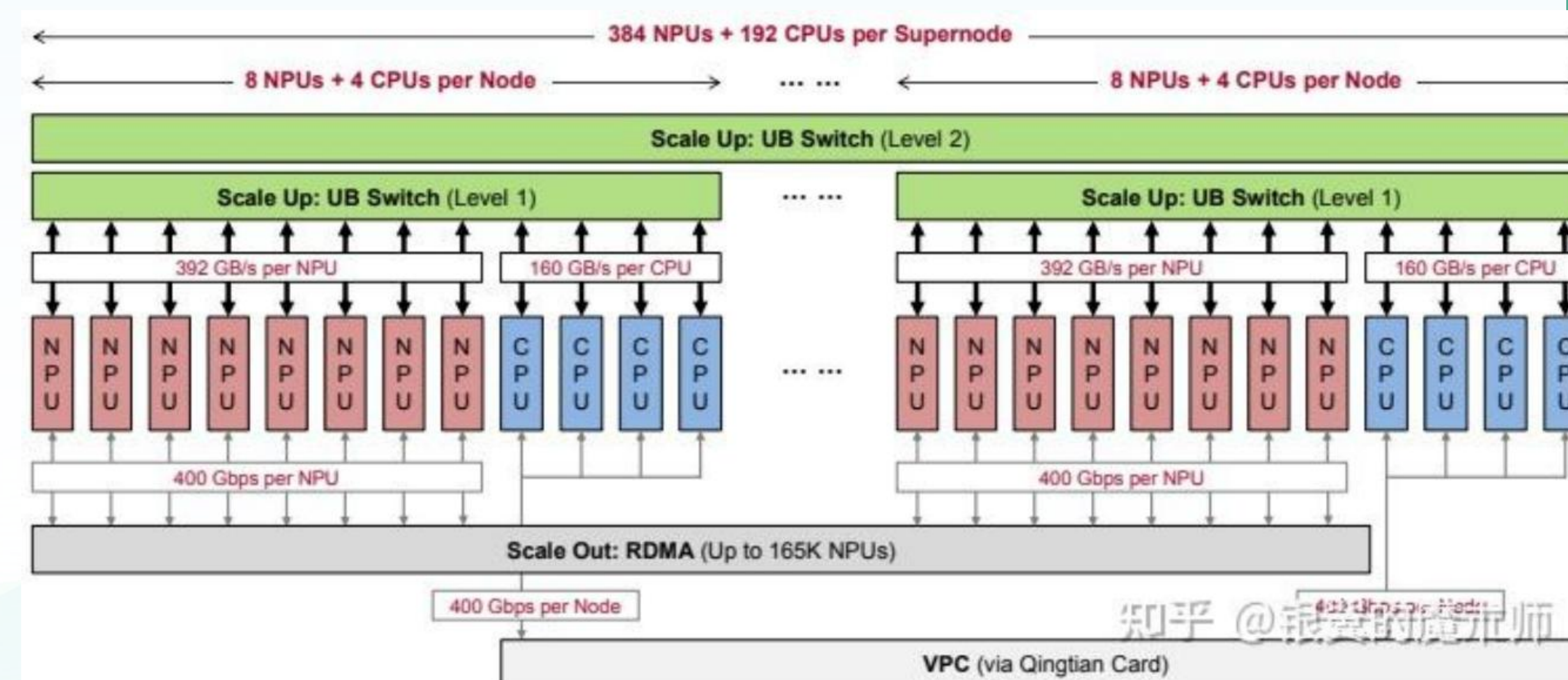
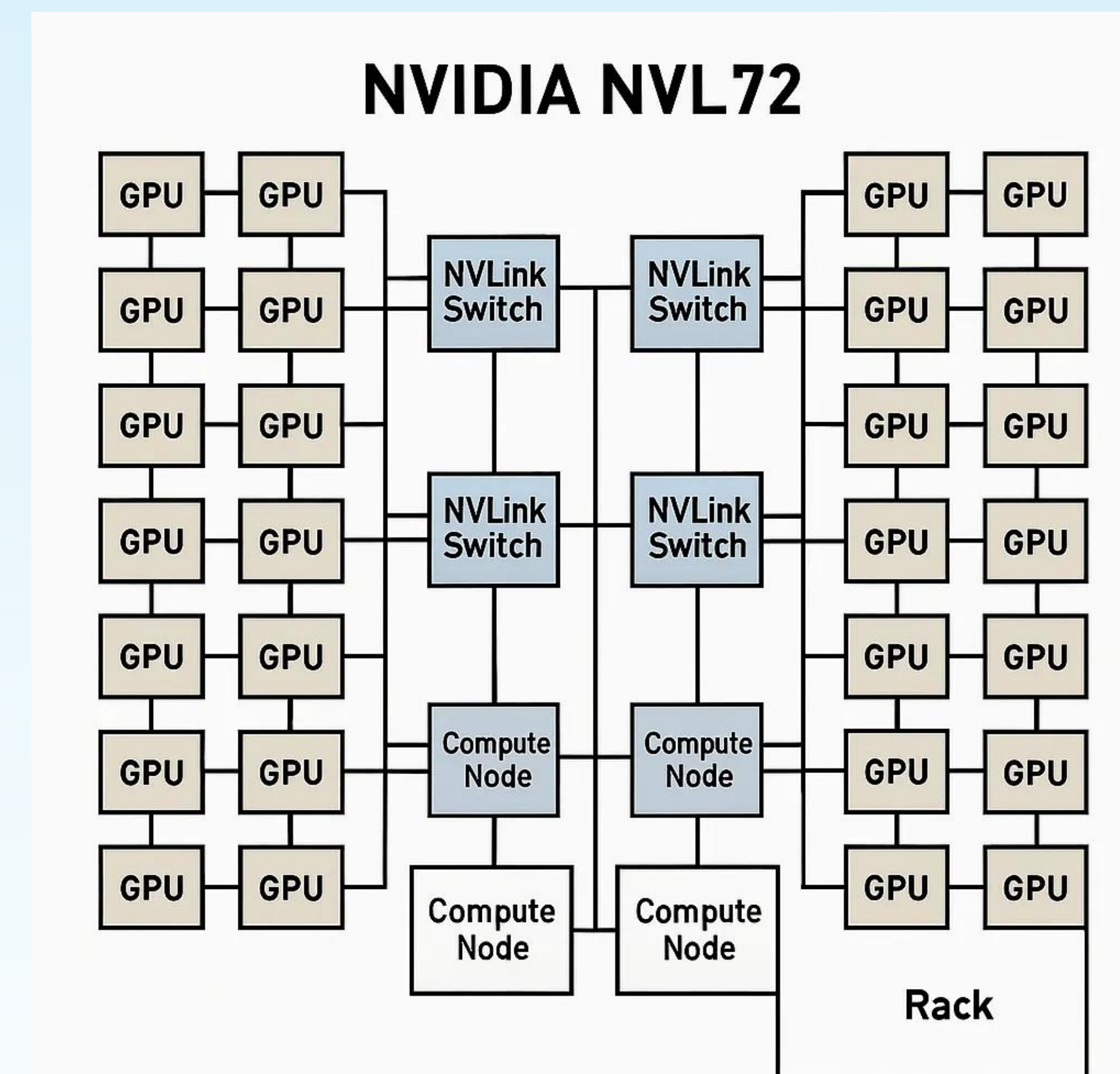
序号	内容
1	非常好 我们是过度的一代，老师真正在探索AI时代我们真正需要学习到什么的教师
2	emm能不能用中文的课件哇www，英文课件看得好难受喔 不过老师能在未名平台上教学配置qwen也蛮好的嘿 嘿！解决了很多烦恼！
3	无

本实
设计
吐率等)

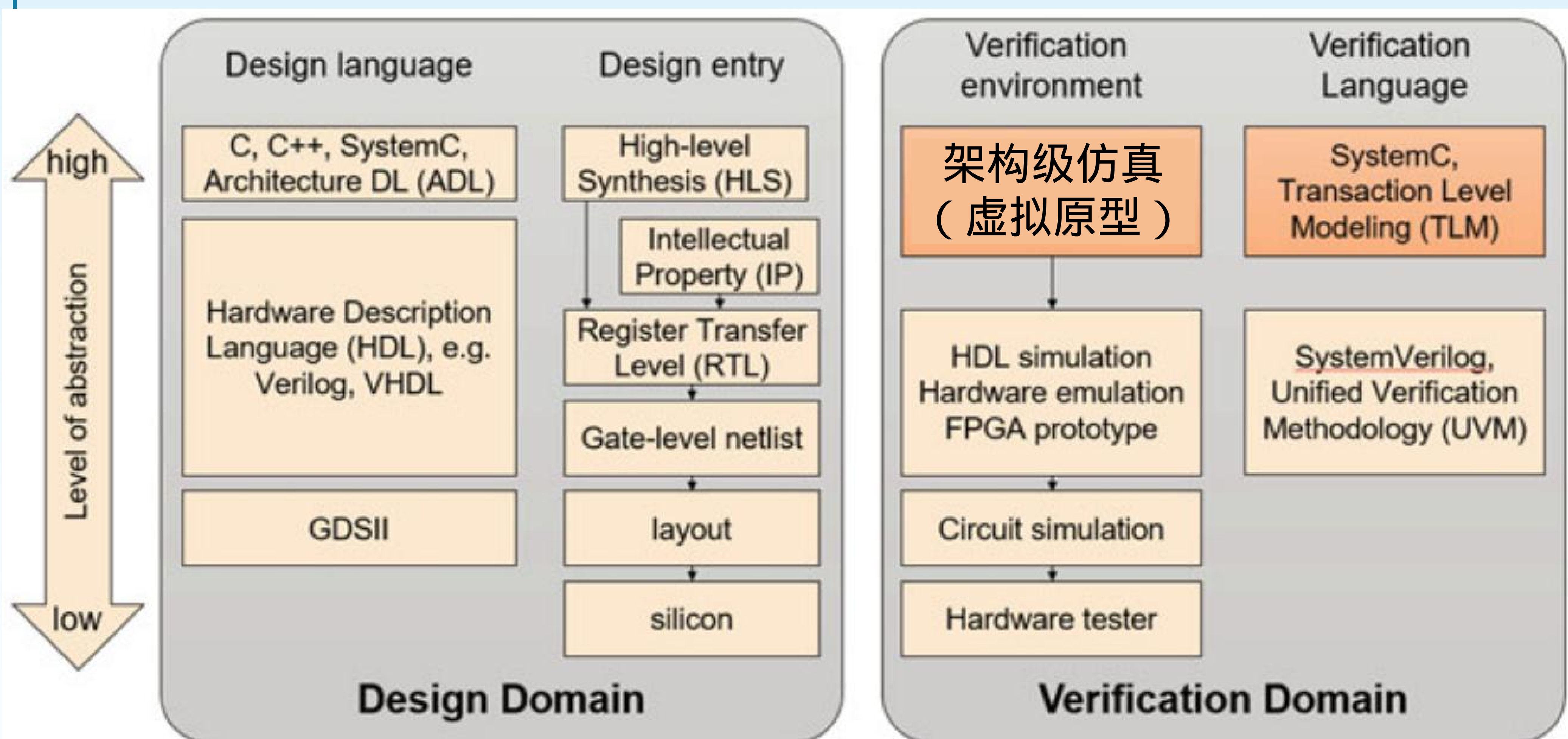
- 构造微基准测试 (μ bench)
 - 指标：时延、吞吐
 - 计算元件：AIC, AIV, Scala
 - 存储元件：L0X, BT, FixPipe, L1, UB, GM
- Simulation as a Source of New Knowledge
 - predicting how an assemblage of such (well-known) components will behave
 - simulation of poorly understood systems

课程提及但未覆盖内容：超节点架构

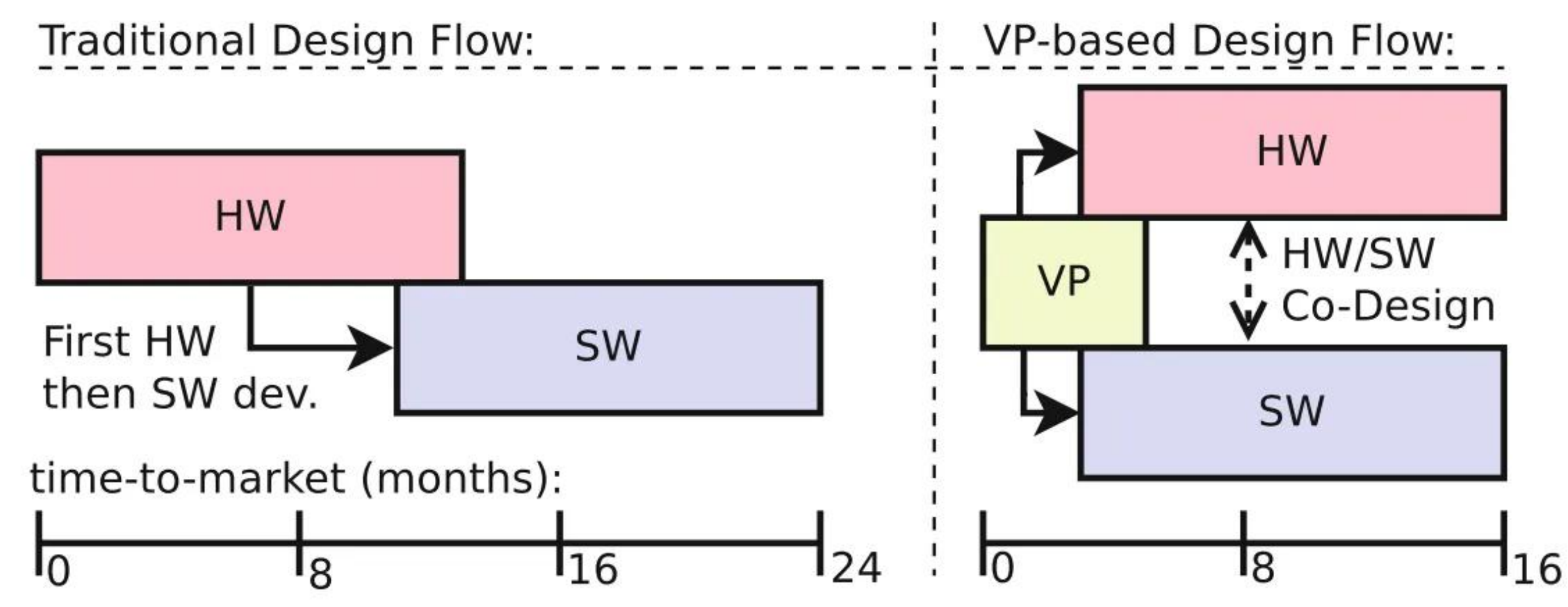
- Nvidia NVL72：机架即芯片
 - 芯片级、计算优先的超节点
 - 设计：将72颗Blackwell GPU塞入一个机架，通过铜缆背板（NVLink Switch）全互连。
 - 目标：用传统组件逼近WSE-3的效果，利用铜缆连接实现机架内All-to-All高带宽。
- 华为 Cloud-Matrix 384
 - 系统级、通信优先的单体超级节点
 - 逻辑：通过HCCS私有协议互连数百颗Ascend处理器，构建统一内存池。
 - 通用性优势：兼顾大规模训练与推理，拥有成熟的CANN生态。



TO-DO: 基于“虚拟原型”设计相关的作业和lab



	SDK, OS emu.	Virt. proto.	HDL Sim.	H/W Sim.	FPGA proto.	Eval. board
仿真速度	++	+ / -	--	0	+	++
周转时间	++	+	0	-	--	n/a
早期可用性	+	+	0	-	--	n/a
时序准确性	--	- / +	+	+	0	++
调试与分析	+	++	+	0	0	0
可用性 & 协作	++	++	+	-	-	0



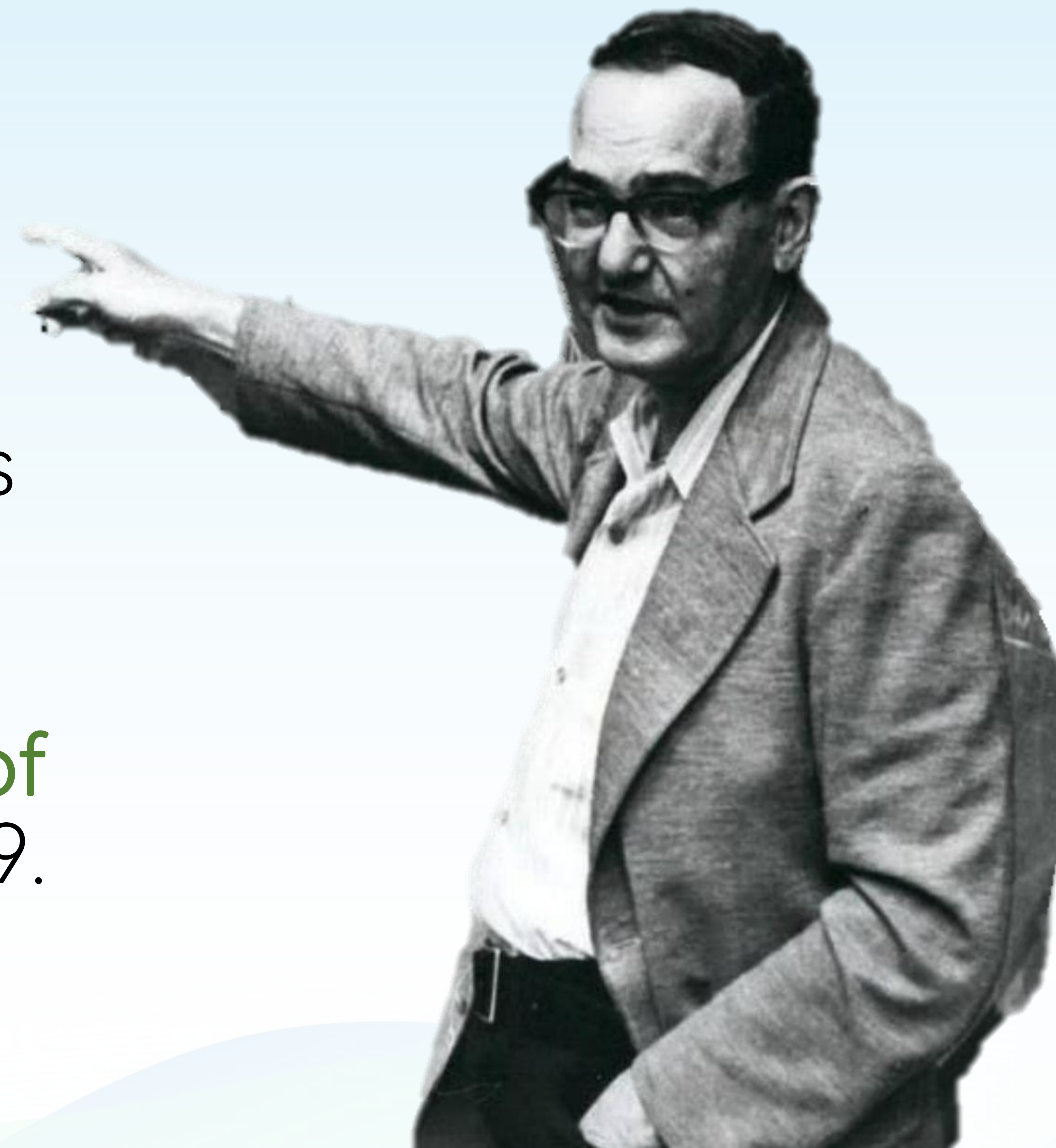
架构级仿真
(虚拟原型)

- Simulation as a Source of New Knowledge
 - predicting how an assemblage of such (well-known) components will behave
 - simulation of poorly understood systems

Chapter 1: The Natural and Artificial Worlds," in Herbert Simon, "**The Sciences of the Artificial (3rd ed.)**," The MIT Press, 2019.

Herbert Simon

- 诺贝尔奖、图灵奖获得者
- “有限理性”理论的提出者





课程大纲和未来展望

- 研究生课程（当代计算机组成）
 - L01: Introduction
 - L02: ML System Foundations
 - L03: Frameworks
 - L04: Comp. Subsys
 - L05: AI Compilation
 - L06: Mem. Subsystem
 - L07: Comm. Subsystem
 - L08: Mod.-Sys. Co-opt.
 - L09: Conclusions
- 本科生课程（五模块）
 - 模块 1：AI 系统与异构计算基础
 - 模块 2：高性能 AI 算子工程
 - 模块 3：AI 编译器的原理与优化
 - 模块 4：分布式并行训推及通信
 - 模块 5：性能评估与AI辅助生成
- TO-DO
 - 丰富 hw 和 lab 的内容
 - 以仿真建模牵引对计算机的理解